

OSDP 3.0 Proposed PIV Enhancements

Security Industry Association

Redline: dc80bdf149e8 to b00ab36ea976 working tree
September 16, 2026

Contents

Changelog	5
1 Introduction	10
2 Changes to OSDP 2.2	11
2.1 Updated Command Definitions	11
2.1.1 Authentication Challenge (osdp_CRAUTH)	11
2.1.2 Response to Challenge (osdp_CRAUTHR)	13
2.1.3 Get PIV Data (osdp_PIVGETDATA)	15
2.1.4 PIV Data Reply (osdp_PIVDATAR)	19
2.1.4 <u>PIV Data Reply (osdp_PIVGETDATAR)</u>	19
2.2 PD-PIV Error Codes for PIV	21
2.2.1 PD-Application-Error-Reply (osdp_PDERROR)	21
2.2.1 <u>PIV Error Reply (osdp_PIVERROR)</u>	21
2.3 Updated Function Codes	22
2.3.1 Function Code 12 – Smart Card Support	22
2.3.2 Function Code 13 – Reader Interfaces	23
2.3.3 Function Code 14 – Biometric Support	23
3 New or Enhanced PIV Commands	24

3.1	<u>Credential preparation and Secure Messaging</u>	24
3.2	PIV Put Data (osdp_PIVPUTDATA)	32
3.3	PIV Application Select (osdp_PIVSELECT)	33
3.4	Card Status (osdp_CARDSTATUS)	34
3.5	Card Status Reply (osdp_CARDSTATUSR)	34
3.6	PIV Secure PIN Entry (osdp_PIVSPE)	39
3.7	PIV Secure PIN Entry Reply (osdp_PIVSPER)	44
3.8	PIV Biometric Match (osdp_PIVBIOMATCH)	47
3.9	PIV Biometric Match Reply (osdp_PIVBIOMATCHR)	47
3.10	TWIC Privacy Key Load (osdp_PIVLOADTPK)	49
3.11	PIV VCI Pairing Code Transmit (osdp_PIVXMITPAIRING)	49
3.12	Set PIV Mode (osdp_PIVMODE)	51
3.13	PIV Auto Status Reply (osdp_PIVSTATUS)	54
3.14	VCI Trust Anchor Load (osdp_PIVVCILOADTA)	57
3.15	VCI Trust Anchor Status (osdp_PIVVCITASTATUS)	62
A	References	64
A.1	Normative References	64
A.2	Informative References	64
B	Sample-osdp_PIVDATA Requests <u>PIVGETDATA Worked Examples</u> (Informative)	65
B.1	<u>Reading the exchanges</u>	65
B.2	<u>Source fixtures</u>	66
B.3	<u>Complete Discovery Object</u>	66
B.4	<u>Discovery PIN Usage Policy (0x5F2F)</u>	67
B.5	<u>Read the FASC-N element (0x30)</u>	68
B.6	<u>Read the FASC-N value</u>	69
B.7	<u>Read the UUID element (0x34)</u>	70
B.8	<u>Read the UUID value</u>	71

B.9	<u>Bounded CHUID prefix</u>	72
B.10	<u>Complete certificate object</u>	73
B.11	<u>Read the certificate element (0x70)</u>	74
B.12	<u>Read a compressed certificate element</u>	76
B.13	<u>Read a range within the certificate element</u>	77
B.14	<u>Complete Key History object</u>	78
B.15	<u>BIT Group with no biometric templates</u>	79
B.16	<u>Empty 0x53 container</u>	80
B.17	<u>Zero-byte successful response</u>	81
B.18	<u>Existing child with an empty value</u>	82
B.19	<u>Missing direct child</u>	83
B.20	<u>Element request against an empty response</u>	84
B.21	<u>Ambiguous direct-child selection</u>	85
C	High-Speed Autonomous PIV Data Templates (Informative)	87
C.1	Vector Sources	87
C.2	Supported Profiles	87
C.3	Fixed-Width KMAC Challenge Template	88
C.4	Dynamic Authentication Templates	88
C.4.1	RSA Key Challenge Input	88
C.4.2	ECC Signature Input	89
C.4.3	Worked Dynamic Authentication Examples	90
C.5	APDUs	98
D	<u>Credential SM and VCI Worked Examples (Informative)</u>	101
D.1	<u>Credential SM over contact</u>	102
D.2	<u>PIN before credential presentation</u>	103
D.3	<u>PIV Auto needs commanded pairing</u>	104
D.4	<u>SM failure diagnostics</u>	105

D.5	<u>PIN cache expires while idle</u>	106
D.6	<u>Expiration while waiting for a credential</u>	107
D.7	<u>Expiration before VERIFY submission</u>	108
D.8	<u>Expiration after VERIFY submission</u>	109
D.9	<u>PIN reuse preserves the original deadline</u>	110
D.10	<u>Credential failure status across two fragments</u>	111
D.11	<u>Incorrect PIN preserves SM and VCI</u>	112
D.12	<u>Recovering trust for contactless PIV Auto</u>	113
E	<u>CRAUTH Worked Examples (Informative)</u>	115
F	Deprecated osdp_GENAUTH	117

Changelog

2026-09-16 Changes from OSDP 2.2:

- Renamed `osdp_PIVDATA/osdp_PIVDATAR` to `osdp_PIVGETDATA/osdp_PIVGETDATAR`, retaining command code `0xA3`. Added identifier and tag lengths, two-byte offsets and requested lengths, and whole-object or unique direct-child selection with the original tag and length bytes preserved. Defined OSDP 3.0 multi-part delivery and an all-zero first reply for empty success. Command data length distinguishes the new request from the fixed-length OSDP 2.2 form.
- Added Format Marker `0xFF` to the reassembled `osdp_CRAUTH` request, retaining command code `0xA5`. Multi-part lengths and offsets cover the marker, algorithm, key reference, and challenge. OSDP 3.0 ACUs use the new `CRAUTH` and `PIVGETDATA` forms with supporting PDs; PDs may additionally support the OSDP 2.2 forms, with format-specific response handling.
- Renamed `osdp_PDERROR` to `osdp_PIVERROR` and added GET DATA selection and transfer errors. The new `CRAUTH` and `PIVGETDATA` forms return one terminal `PIVERROR` on failure, replacing `CRAUTH`'s empty failure reply and optional follow-up error. Renumbered tentative PIV-specific errors consecutively from `0x1020` through `0x1029`; credential removal is now `0x1026`.
- Separated credential SM setup, VCI pairing, and PIN verification. Clarified automatic preparation over contact and contactless, PIV Auto failure reporting, and ACU-directed recovery. Authenticated PIN and pairing rejections preserve SM; SM processing and integrity failures end the session.
- Added `osdp_CARDSTATUS` Type `0x02` diagnostics containing the SM outcome and credential CVC hash. Every `CARDSTATUSR` now includes Status Type after reply code `0x85`: `0x01` for standard status and pairing completion, `0x02` before the multi-part fields in each diagnostic fragment. Extended `osdp_PIVSTATUS` with SW1/SW2 when a verified credential failure status is available; the status word follows the signed response in the reassembled payload.
- Defined cached-PIN reuse without extending the original deadline, silent idle expiration, and completion of requests affected by expiration. Bounded PD timeout defaults to 1-255 seconds. Reassigned SPE flags `0x01` for retention across removal and `0x02` for Auto Execute; added result `0x0B` for credential communication failure or timeout, requiring preparation to restart after an unconfirmed PIN VERIFY. Applied the same restart rule to pairing VERIFY timeouts, reported through existing `PIVERROR 0x0002` (smart-card timeout).
- Added workflow diagrams and worked packet examples for selection, SM/VCI recovery, PIN-cache timing, and multi-part replies. Standardized table sizes and protocol values on hexadecimal notation.

2026-09-02 Made proposal builds self-contained by vendoring the SIL-licensed Carlito typeface, replacing system Calibri and Arial dependencies with real regular, bold, italic, and bold-italic faces, and documenting the XeLaTeX and latexmk requirements. Completed the Enhanced PIV message-name cleanup by applying the compact OSDP naming convention consistently to `osdp_PIVLOADTPK`, `osdp_PIVXMITPAIRING`, `osdp_PIVVCILOADTA`, `osdp_PIVVCITASTATUS`, and `osdp_PIVVCITASTATUSR`; command and reply codes, payloads,

and behavior are unchanged. Standardized multi-part messages on the generic OSDP *MpSizeTotal*, *MpOffset*, and *MpFragmentSize* field names and moved the multi-part fields ahead of the re-assembled `osdp_PIVMODE` and `osdp_PIVVCILOADTA` data. The current `osdp_PIVVCILOADTA` definition supersedes the anchor-only multi-part sizing described in the 2026-05-30 entry: its total now includes the 12-byte metadata block. Updated the PIV Auto simulator, generated fixtures, proposal PDF, and redline PDF to exercise and publish the same framing and naming.

2026-08-12 Proposed Secure PIN Entry changes and consistency cleanup:

- Proposed clear, mode-specific SPE behavior for status, PIN acquisition and verification, cached-PIN verification, cache/abort, and attempts-remaining queries. The proposal also defines finite default timeouts, cache and PIN-verified lifecycles, and ACK/poll completion behavior.
- Proposed a one-byte `osdp_PIVSPER` reply whose value is interpreted by the requested mode. The OSDP 2.2 `osdp_PR01SPER` reply remains unchanged.
- Standardized Enhanced PIV names, including renaming the autonomous PIV Auto reply to `osdp_PIVSTATUS`. Corrected tentative code conflicts by assigning `osdp_PIVSELECT` to 0xB0 and `osdp_PDERROR` to 0x8A.

2026-06-18 PIVSELECT, PDERROR, and VCI consistency updates:

- Added `osdp_PIVSELECT` as an OSDP-native direct AID selection command using tentative command code 0xA7; clarified its ISO/IEC 7816 SELECT by DF name mapping, selected-AID status update, and application-state clearing rules.
- Removed stale `osdp_CRAUTHR` `osdp_PDERROR` wording so codes 0x0006/0x0007 retain their common meanings for secure-channel and multi-part errors.
- Clarified that VCI pairing codes are transmitted from the ACU to the PD only over OSDP Secure Channel 2.0 and are used by the PD to establish VCI secure messaging with the credential.
- Tightened and clarified verbiage by replacing imprecise “legacy” terminology throughout normative text with neutral wording such as unchanged, OSDP 2.2-format, and poll-based; retained “legacy” only for withdrawn algorithms such as RSA-1024 test material.

2026-05-30 Added source-backed VCI and PIV Auto examples, clarified PIV Auto interoperability requirements, and improved reviewability:

- Added NIST-derived VCI and PIV APDU fixture coverage, GSA-backed demonstration keys, and reproducible worked examples for PIV Auto and VCI validation. RSA-1024 material is retained only as informative legacy/non-current test-vector material.
- Clarified PIV Auto cryptographic inputs and response handling, including KMAC-derived challenge construction, RSA and ECC GENERAL AUTHENTICATE templates, nonzero UUID requirements, and complete returned 0x7C Dynamic Authentication Templates in `osdp_PIVSTATUS`.
- Aligned related command semantics for capability advertising, `osdp_CARDSTATUS` errors, selective `osdp_PIVDATA` replies, `osdp_PIVSPE` status/abort modes, and VCI trust-anchor inventory.

- Reworked Appendix C and the references section for standards review: examples are explicitly informative, vector provenance is documented, source fixtures are framed as public test material, and normative references now include the needed APDU, KMAC, X.509, and requirement-language specifications.
- Added supporting scripts and tests for deterministic simulation, fixture replay, VCI import/validation, Appendix C generation, YubiKey material loading, live APDU capture, and reproducible PDF/table builds.

2026-05-27 Version renamed from OSDP 2.3 to OSDP 3.0 throughout. Consolidated PIV Auto, biometric-result, VCI trust-anchor, and SPE updates:

- Removed `osdp_PIVSPE` flag 0x01 (retry continuation); each `osdp_PIVSPE` command now initiates exactly one PIN-entry session and `osdp_PIVSPER` terminates the session unconditionally, including on PIN mismatch. ACUs wishing to allow additional attempts **MAY** re-issue `osdp_PIVSPE`, using the *Attempts Remaining* count from the preceding `osdp_PIVSPER` to guide retry policy. Bit 0x01 is now reserved.
- Aligned `osdp_PIVSPER` and `osdp_PIVBIOMATCHR` result codes for credential removal, changed `osdp_PIVBIOMATCHR` 0x03 to PIN verification required, reserved `osdp_PIVBIOMATCHR` 0x07, clarified biometric assurance limits, and specified that only off-card biometric match success returns the 32-byte SHA-256 hash of tag 0xBC.
- Clarified the Function Code 14 biometric-support reference for `osdp_PIVLOADTPK` and added CAC as PIV mode data model 0x03.
- Added PIV Auto KMAC challenge derivation, required series and sequence counters, fixed-width supplemental entropy, and the `osdp_PIVSTATUS` poll reply for reporting credential identity and signed challenge responses. PIV Auto is configured through `osdp_PIVMODE` and is restricted to Secure Channel 2.0 sessions that are not using factory keys.
- Replaced the SP 800-73 Intermediate CVC anchor model with an OSDP-defined VCI trust anchor record (tag 0x7F50) carrying a Profile Identifier (0x80 = 0x01), an eight-byte Issuer Identifier Number (0x42), and a DER-encoded SubjectPublicKeyInfo (0x7F49). Maximum conforming record size is 576 bytes (0x0240), derived from the RSA-4096 key case.
- Added normative IIN derivation rule: the IIN in `osdp_PIVVCILOADTA` and in the trust anchor record **SHALL** be the leftmost eight bytes of the subjectKeyIdentifier from the content-signing X.509 certificate.
- Tightened `osdp_PIVVCILOADTA` multi-part encoding: Anchor Length and Mp-SizeTotal ranges corrected to 0x0000-0x0240; added normative constraint that MpOffset + MpFragmentSize **SHALL NOT** exceed Anchor Length; PD **SHALL** verify the reassembled byte string is exactly *Anchor Length* bytes and a single 0x7F50 record before committing.
- Documented variable-length PD storage accounting: PDs **SHALL** account for anchors by their actual *Anchor Length*, not a fixed maximum; replacement is atomic (prior anchor remains if replacement fails); delete releases actual stored bytes.
- Specified the two-tier PD verification path during VCI establishment: direct (trust anchor IIN matches secure messaging CVC IIN, anchor SPKI verifies CVC signature) and intermediate (no

direct match triggers retrieval of PIV data object 0x1017/5FC122; Intermediate CVC Subject Identifier must equal CVC IIN, Role 0x5F4C must be 0x12, anchor IIN must match Intermediate CVC IIN, anchor SPKI verifies Intermediate CVC, Intermediate CVC verifies secure messaging CVC). PDs **SHALL NOT** perform X.509 path validation or revocation for VCI trust anchors.

- Added normative ACU post-VCI validation requirements: after `osdp_CARDSTATUSR` reports VCI established, the ACU **SHALL** obtain the content-signing certificate (from cache or via `osdp_PIVDATA` for 0x1017/inner tag 0x70), verify SKI matches *VCI Anchor IIN*, verify SPKI matches the stored anchor record, verify PIV or PIV-I content-signing ECU OID, verify validity period (expired certificate **SHALL** cause session rejection and anchor unload), and perform X.509 path validation and revocation per deployment policy.
- Restored *VCI Trust Anchor ID* to `osdp_CARDSTATUSR` with normative language; added *VCI CVC IIN* and *VCI Anchor IIN* fields (each 8 bytes) so the ACU receives the IIN pair needed to distinguish direct and intermediate verification paths.
- Updated `osdp_PIVVCITASTATUSR`: clarified that *Available Bytes* reflects variable-length accounting and that the per-record IIN is the anchor selector from the stored trust anchor record, not necessarily the secure messaging CVC IIN reported in `osdp_CARDSTATUSR`.
- Added VCI security preconditions: all VCI-related commands **SHALL** be rejected unless OSDP Secure Channel 2.0 is active and not using factory/default keys; failures reported via `osdp_PDERROR` 0x0006.

2026-01-21 Error-handling cleanup and code assignments: **(1)** `osdp_PIVPUTDATA` now **SHALL** report failures via `osdp_PDERROR` (status word optional) so writes can't silently succeed. **(2)** `osdp_PIVSPE` adds result code 0x09 in `osdp_PIVSPER` for "VCI not established." **(3)** `PDERROR` is credential/global only; PD capability gaps stay `osdp_NAK`. **(4)** OSDP 2.2-format replies (e.g., `osdp_CRAUTHR`, `osdp_PIVDATAR`) remain unchanged, with optional follow-up `osdp_PDERROR` detail if the PD has it. **(5)** Documented tentative command/reply codes for the new PIV set: commands 0xA6 (VCI trust-anchor status) through 0xAF (VCI trust-anchor load), and replies 0x85-0x89 for the new responses.

2026-01-07 Added a PIV Mode interface-mask table aligned to Function Code 13 bits, clarified `osdp_CRAUTHR` reply structure (full GENERAL AUTHENTICATE response without SW1|SW2; status words reported via `osdp_PDERROR` only), tightened `osdp_CARDSTATUS/osdp_CARDSTATUSR` behavior (solicited-only after PD preparation; error on credential removal), remapped the PIV error codes to the 0x10xx block per subcommittee guidance, added PIV APDU references, aligned `osdp_PIVDATA` tag selection to allow Discovery (0x7E)/BIT (0x7F61) subtags in line with SP 800-73-4, clarified that `osdp_PDERROR` conveys a single error per reply, and clarified `osdp_PIVXMITPAIRING` handling so pairing codes can be provided before VCI is established when the credential requires them.

2025-12-10 Consolidated application-level errors on `osdp_PDERROR` (two-byte codes, optional SW1|SW2 for APDU failures, multi-error support); moved the PIV error set to 0x0100-0x01FF and updated PIV commands to use `osdp_PDERROR` (transport faults remain `osdp_NAK`); made the Global Interface Mask in `osdp_PIVMODE` a required byte (no TLV tag) with an updated field layout; clarified that

osdp_CARDSTATUSR is solicited-only after PD preparation with poll-based osdp_RAW as the aggregated card-present indication; and noted the deprecation of osdp_GENAUTH with a recommendation to migrate to osdp_CRAUTH.

2025-12-04 Clarified that osdp_CRAUTHR returns the full GENERAL AUTHENTICATE data including the Dynamic Authentication Template (excluding only the ISO/IEC 7816 status word).

2025-11-12 Added PIV Biometric Match request/reply and tables (one-bit type values only); moved Enhanced PIV NAKs to a 0x20+ base (added 0x27 Credential Not Present/Removed, 0x28 Biometric Type Not Supported; clarified 0x23 Credential Security Status Not Satisfied); aligned CRAUTHR error handling to reference the enhanced NAK set; relocated Enhanced PIV (0x04) and PIV Auto (0x08) indicators to Function Code 12 and cleaned Function Code 13 to interface-only bits (removed non-interface flags; removed PIV Auto from Function Code 14); updated CARDSTATUS gating to Function Code 12 bit 0x04; added a PIV Mode Flags table and clarified PIV Mode AID encoding; enabled PDF TOC hyperlinks and improved table/caption pagination.

2025-10-30 Clarified CRAUTH command/reply behaviour (including RSA, AES, and ECC support), overhauled PIV Mode configuration with TLV-based application and interface controls (including PIV Auto), added card status request/reply messages, expanded PIV data retrieval semantics, introduced PIV Put Data, PIV Secure PIN Entry, TWIC privacy-key load, VCI pairing transmit, and VCI trust-anchor load/status commands, and documented the enhanced error-code set (including VCI/PIV Auto conditions).

2025-09-17 Major PIV Mode and SPE revisions.

2025-06-25 Added multi-AID usage and VCI support notes.

2025-05-27 New function code definitions for 3.0 biometric handling.

1 Introduction

This document captures proposed enhancements to the SIA Open Supervised Device Protocol (OSDP) needed to carry FICAM-compliant Personal Identity Verification (PIV) workflows. The material consolidates draft content, reviewer comments, and interoperability feedback collected during the 2025 working sessions and re-expresses it as normative guidance for command authors and device implementers.

The chapters that follow highlight changes required for existing OSDP 2.2 commands, introduce new transactions where no equivalent existed, and provide supplemental examples that reflect current federal credentialing practices. Wherever the proposal diverges from the baseline specification, the text records both the behavioral requirements and the error handling expectations so that access control units (ACUs) and peripheral devices (PDs) can adopt the profile without relying on vendor extensions.

~~Backward compatibility: when a PD encounters a credential-side error on an OSDP 2.2-format reply (for example `osdp_CRAUTHR` or `osdp_PIVDATAR`), it continues to return the unchanged reply shape (often with `MpFragmentSize = 0`) and may optionally send an `osdp_PDERROR` with additional detail on a subsequent `osdp_POLL`. ACUs are expected to tolerate or ignore the optional `osdp_PDERROR`; The key words **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are interpreted as described in RFC 2119 and RFC 8174 when they appear in capitals.~~

Tables express protocol sizes, lengths, offsets, and numeric values in hexadecimal with the prefix `0x`. Packet and APDU dumps use hexadecimal byte pairs without repeated prefixes. Standard names, bit positions, and elapsed-time annotations retain their usual notation.

OSDP 3.0 introduces a new PIVGETDATA request format at command code `0xA3`. PDs support the new format and may also accept the OSDP 2.2 ~~ACUs may log the unknown reply code and proceed~~format; the command data length distinguishes the two. OSDP 3.0 ACUs use the new format with PDs that support it. The revised CRAUTH and PIVGETDATA transactions return data on success and one terminal PIVERROR on failure.

2 Changes to OSDP 2.2

This section documents the normative deltas that apply when an OSDP 2.2 implementation adopts the Enhanced PIV profile. Each subsection references the original command definition and records the precise behavioral updates, encoding clarifications, and new error handling rules introduced by this proposal.

2.1 Updated Command Definitions

2.1.1 Authentication Challenge (osdp_CRAUTH)

~~OSDP 2.2 transports the ISO/IEC 7816-4 command through osdp_CRAUTH. The osdp_CRAUTH command, code 0xA5, transports a GENERALAUTHENTICATE operation to the credential. An OSDP 3.0 ACU SHALL use the new format with a PD that supports it. An OSDP 3.0 PD SHALL support the new format and MAY also support the OSDP 2.2 format.~~

Request format and recognition The new format uses the OSDP 3.0 ~~profile maintains the same framing fields, named MpSizeTotal, MpOffset, and MpFragmentSize as in the generic OSDP multi-part structure, but clarifies how the command payload maps to the PIV operation selected by the algorithm and key reference.~~ multi-part fields immediately after the command code. The reassembled data consists of Format Marker 0xFF, Algorithm, Reference Identifier, and Challenge, in that order. The three leading fields are each one byte. *MpSizeTotal* counts all of these bytes, including the marker, algorithm, and key reference.

The first data fragment **SHALL** start at offset 0x0000 and contain at least one data byte. The PD **SHALL** select the new format when that first byte is 0xFF. Otherwise, the PD **SHALL** select the OSDP 2.2 format if supported, or reject the request with osdp_NAK 0x09 (unable to process command). This proposal reserves 0xFF in that position as an OSDP format marker. A dual-format PD **SHALL NOT** interpret that byte as a legacy algorithm identifier. The marker is separate from the Algorithm field and is removed before the PD constructs the credential command.

The PD **SHALL** retain the selected format until completion or cancellation. Later fragments carry the next bytes of the same data record; the marker and other fields are not repeated. Fragment boundaries may split the leading fields or the challenge. The PD **SHALL** validate the complete record before sending the credential command. The minimum record length is 0x0003; the selected algorithm determines the required challenge length. A malformed new-format record **SHALL NOT** be retried as an OSDP 2.2 record. Multi-part framing errors use the applicable OSDP NAK rules. Invalid reassembled parameters return PIVERROR 0x0005 (invalid parameter) after acceptance.

A PD accepting the OSDP 2.2 form **SHALL** use that form's request and response rules for the transaction. The rules below apply to the new form. The worked example in Appendix E shows the marker, algorithm, and key reference divided between command fragments.

- ~~The Algorithm field continues to be sent in the first fragment only.~~
- Codes 0x05, 0x06, and 0x07 ~~select identify~~ RSA private-key operations ~~and preserve backwards compatibility with OSDP 2.2.~~ Codes 0x08, 0x0A, and 0x0C ~~select AES encryption using identify~~ AES

encryption with 128-, 192-, and 256-bit keys respectively. Codes 0x11 and 0x14 identify ECC P-256 and P-384 mechanisms; the selected **PIV**-key reference determines whether the **mechanism-is-used for operation is** ECDSA signature generation or ECDH key establishment. Algorithm selection remains subject to the applicable NIST requirements.

- The *Key* field retains its role as the card's reference identifier. PDs **SHALL NOT** reinterpret or remap the identifier.
- For RSA algorithms (0x05, 0x06, 0x07), the *Challenge* field carries the encoded datum for the RSA private-key operation. The PD forwards the value unmodified; padding and digest encoding are the responsibility of the ACU.
- For AES algorithms (0x08, 0x0A, 0x0C), the *Challenge* field contains the plaintext block(s) to be encrypted. PDs **SHALL** send the data to the credential without padding; ACUs **SHALL** provide data aligned to the block size dictated by the selected key length and card profile.
- For ECC signature operations using key references 0x9A, 0x9C, or 0x9E with algorithms 0x11 or 0x14, the *Challenge* field carries the pre-hashed input to be signed. The PD **SHALL** issue GENERALAUTHENTICATE with command data encoded as 7C Len 82 00 81 Len Challenge; the credential returns the ECDSA signature in tag 0x82. Exponentiation tag 0x85 **SHALL NOT** be used with 0x9A, 0x9C, or 0x9E.
- For ECC key-establishment operations using key reference 0x9D or retired key-management references, algorithms 0x11 and 0x14 use the *Challenge* field to convey the peer public key. The PD **SHALL** issue GENERALAUTHENTICATE with command data encoded as 7C Len 82 00 85 Len (04 || X || Y), where X and Y are the uncompressed coordinates for the selected curve and are left-padded with 0x00 as required. The credential performs elliptic-curve Diffie-Hellman in accordance with SP 800-73.

Table 1: Authentication Challenge (osdp_CRAUTH)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Command identifier.	0xA5
<u>0x02</u>	MpSizeTotal	Length of the complete message, <u>record, including Format Marker,</u> <u>Algorithm, Reference Identifier, and</u> <u>Challenge;</u> least-significant byte first.	0x0000–0xFFFF <u>0x0003–0xFFFF</u>
<u>0x02</u>	MpOffset	Offset of this fragment within the complete message, least-significant byte first.	0x0000–0xFFFF
<u>0x02</u>	MpFragmentSize	Length of the fragment payload, least-significant byte first.	0x0000–0xFFFF

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	<u>Format Marker</u>	<u>First byte of the reassembled record. Identifies the OSDP 3.0 format; removed before constructing the credential command.</u>	<u>0xFF</u>
<u>0x01</u>	Algorithm	Cryptographic mechanism. Supported values include 0x05/0x06/0x07 (RSA private-key operation), 0x08/0x0A/0x0C (AES encryption with 128/192/256-bit keys), and 0x11/0x14 (ECC P-256/P-384; operation depends on the key reference).	0x00–0xFF
<u>0x01</u>	Reference Identifier	Key reference passed to GENERALAUTHENTICATE.	0x00–0xFF
<u>0x00-n</u>	Challenge	For RSA algorithms, the encoded datum for the private-key operation. For AES algorithms, the plaintext block(s) to be encrypted, pre-padded by the ACU as required. For ECC 0x9A/0x9C/0x9E, the pre-hashed signature input in tag 0x81; for ECC 0x9D/retired key-management references, the peer public key in tag 0x85.	0x00–0xFF

When credential SM is established, the PD **SHALL** protect the resulting credential command using that context. The credential enforces the selected key's access conditions. An available card-authentication key (0x9E) may be used without SM or PIN verification when the credential permits it. Shared card-transport rules appear in Subsection 3.1.

Deprecated osdp_GENAUTH The osdp_GENAUTH command is deprecated and expected to be removed in a future revision of this specification. Using a PIV credential to decrypt or process arbitrary data is unsafe and unnecessary for credential verification; implementations **SHOULD** use osdp_CRAUTH flows instead.

2.1.2 Response to Challenge (osdp_CRAUTHR)

The reply continues to use the OSDP 3.0 uses the same completion rule for CRAUTH and PIVGETDATA: data replies carry successful results; one terminal PIVERROR reports a failed accepted operation. This replaces CRAUTH's OSDP 2.2 multi-part format. Successful execution yields a response containing the credential's entire data (minus the ISO/IEC 7816 status word); including the returned Dynamic Authentication Template; PDs failure behavior.

After accepting the complete CAUTH request, the PD **SHALL NOT** strip outer BER-TLV tags or lengths. If the credential reports an error, return the result on subsequent osdp_POLL commands. On success, the PD **SHALL** return the unchanged osdp_CAUTHR reply (for example with MpFragmentSize = 0) to preserve OSDP 2.2 behaviour. When available using the OSDP 3.0 multi-part format. On failure, the PD ~~SHOULD also return~~ **SHALL** return one osdp_PDERROR on a subsequent osdp_POLL with an applicable PIV code—for example (function not supported by credential), (insufficient credential storage), (credential security status not satisfied, including PIN-required conditions), (data object/tag not found), (VCI not established), or (credential not present/removed). Generic parameter errors may continue to use (incorrect parameter). PIN-entry outcomes are reported through command-specific result paths, such as osdp_PIVSPER, when secure PIN entry is used. ACUs **PIVERROR** with the applicable error code from Subsection 2.2. That reply ends the operation. The PD **SHALL NOT** precede the error with an empty CAUTHR. If CAUTHR fragments have already been returned, the ACU **SHALL** tolerate an optional osdp_PDERROR following that reply; OSDP 2.2 ACUs may log and ignore the unknown reply code; discard the partial response when PIVERROR arrives. OSDP retransmissions repeat the same reply without repeating the credential operation. Command rejection and transport faults continue to use the applicable OSDP NAK rules.

Reply structure

osdp_CAUTHR **SHALL** return the credential's full ISO/IEC 7816 GENERALAUTHENTICATE response payload, including the Dynamic Authentication Template, framed by the standard OSDP multi-part fields. PDs **SHALL NOT** include SW1|SW2 in osdp_CAUTHR; any status words from the credential **SHALL** be carried only in osdp_~~PDERROR~~**PIVERROR** ~~when a card/APDU failure occurs under the status-word rules in Subsection 2.2.1.~~ PDs **SHALL NOT** strip outer BER-TLV tags or lengths from the returned payload.

Table 2: Response to Challenge (osdp_CAUTHR)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Reply identifier.	0x82
<u>0x02</u>	MpSizeTotal	Length of the complete response, least-significant byte first.	0x0000–0xFFFF
<u>0x02</u>	MpOffset	Offset of this fragment within the complete response, least-significant byte first.	0x0000–0xFFFF
<u>0x02</u>	MpFragmentSize	Length of the fragment payload, least-significant byte first.	0x0000–0xFFFF
<u>0x00-n</u>	Response Data	Signature, encrypted block, or shared-secret material returned by the credential. The credential's BER-TLV structure is preserved exactly as received (status word excluded).	0x00–0xFF

2.1.3 Get PIV Data (osdp_PIVGETDATA)

OSDP 2.2 defined osdp_PIVDATA with a five-byte payload that selected a PIV object and optional element. The original text did not explicitly specify how to pad identifiers shorter than three bytes, how nested tags should be returned, or how to interpret the data offset. OSDP 3.0 clarifies these behaviors so that ACUs and PDs apply a single normative approach. retains command code 0xA3 and introduces the PIVGETDATA request format defined here. An OSDP 3.0 ACU **SHALL** send the new format to a PD that supports it. An OSDP 3.0 PD **SHALL** support the new format and **MAY** also support the OSDP 2.2 format. The ACU **SHALL** confirm Enhanced PIV support (Function Code 12 bit 0x04) before using the new format. The reply uses the OSDP 3.0 multi-part layout. Worked exchanges appear in Appendix B.

The ACU

Recognizing the request format The PD **SHALL** transmit the PIV Object Identifier as three bytes encoded most-significant-byte-first determine the format from the command data length, excluding the command code, OSDP header, security block, and trailer. The OSDP 2.2 format contains exactly 0x05 data bytes: a three-byte object identifier, a one-byte element identifier, and a one-byte offset. The new format contains 0x06 plus Identifier Length plus Tag Length data bytes, giving a range of 0x07 through 0x0C bytes. These lengths do not overlap.

A PD that supports both formats **SHALL** interpret exactly 0x05 data bytes using the OSDP 2.2 definition, including its selection and response rules. A new-format-only PD **SHALL** reject that request with PIVERROR 0x0005 (invalid parameter) on a later poll after ACK. For a new-format request, the PD **SHALL** validate both length fields and the exact total length before accessing the credential. Any other data length or invalid new-format field length produces the same error. The PD **SHALL NOT** reinterpret a malformed new-format request as an OSDP 2.2 request.

Table 3: Get PIV Data (osdp_PIVGETDATA)

Size (bytes)	Name	Meaning	Value
0x01	CMND	Command identifier.	0xA3
0x01	Identifier Length	Number of bytes in Object Identifier. When the identifier contains fewer than three significant bytes, the	0x01-0x03
0x01-0x03	Object Identifier	Complete BER tag identifying the credential data object. Native byte order; no leading padding.	BER tag
0x01	Tag Length	Number of bytes in Element Tag. Zero selects the complete object and omits Element Tag.	0x00-0x03
0x00-0x03	Element Tag	Complete BER tag of a unique direct child of the object's outer container.	BER tag

Size (bytes)	Name	Meaning	Value
0x02	Offset	Starting byte within the selected object or element. Zero identifies its first tag byte. Least-significant byte first.	0x0000–0xFFFF
0x02	Requested Length	Maximum bytes to return, least-significant byte first. Zero requests all remaining bytes.	0x0000–0xFFFF

Request encoding The ACU **SHALL** ~~left-pad the value with or, as required, so that the transmitted field is exactly three bytes.~~ The PD provide valid Object Identifier and Element Tag encodings. The PD **SHALL** pass Object Identifier to the card unmodified and use Element Tag unmodified to select a matching element from the response. Each length field **SHALL** equal the number of tag bytes that follow it. Identifier Length is one to three; Tag Length is zero to three. Tag Length zero omits Element Tag. The PD **SHALL NOT** ~~forward those pad bytes to the credential; it~~ reject invalid field lengths, truncated requests, and trailing request bytes with `osdp_PIVERROR 0x0005` (invalid parameter).

The PD **SHALL** issue the ~~ISO 7816-4 card's GET DATA APDU using only the significant identifier bytes.~~ If VCI secure messaging operation for the requested object on the active credential interface. Each request reads the credential; the PD **SHALL NOT** substitute object data retained from an earlier request. If the credential denies access through that interface, the PD **SHALL** report the failure using `osdp_PIVERROR`. When credential Secure Messaging (SM) is established, the PD **SHALL** use ~~the appropriate secure channel automatically; no explicit selector is present in the command.~~ The field previously described as “PIV element ID” is renamed *PIV Tag*. A tag value of directs the PD to return the ~~that SM context.~~ The credential enforces VCI availability and the requested object's access conditions. The PD **SHALL** complete BER-TLV structure, including the native outer tag (for example card-side response chaining and remove status words and secure-messaging envelopes before selecting response data. A requested OSDP length does not limit the card-side transfer needed to complete that operation.

Selection When Tag Length is zero, the PD **SHALL** select the complete object, including its outer tag, length, and value. The PD **SHALL** preserve the original bytes returned by the credential. PIV responses use `0x53` for ordinary objects and native `0x7E` Discovery, and `0x7F61` BIT Group Template) or the /wrapper used by other PIV objects. ~~Any non-zero value selects a single root-level tag within the object. When the outer tag is or, containers for Discovery and the BIT Group. Other application profiles may define other containers; whole-object selection does not substitute or remove them.~~

For nonzero Tag Length, the PD **SHALL** parse the outer container and select exactly one matching direct child, including that child's tag, length, and value. The enclosing object's tag and length are not included. The PD **SHALL NOT** search recursively, decompress the value, or re-encode the selected element. The PD **SHALL** inspect the complete child sequence before accepting a unique match. No match produces `0x1024` (data object or tag not found); multiple matches produce `0x1027` (ambiguous element selection); malformed container or child encoding produces `0x1028` (malformed credential-object encoding). Element extraction requires definite BER lengths, a single complete outer container, and child boundaries within that container.

To select an inner tag, the PD **SHALL** strip that wrapper before returning the selected tag. For other outer tags search the elements directly inside the object's outer container. In BER-TLV encoding, *primitive* means the tag marks its value as data bytes; *constructed* means the tag marks its value as a sequence of TLV elements. For PIV objects returned under tag 0x53, the PD **SHALL** return the requested tag if present and treat the contents as TLV elements even though the tag marks them as primitive. Child values are opaque regardless of their constructed bit; the PD **SHALL NOT** reject the request solely because the wrapper is not /. Tags recursively validate them as ASN.1 structures. Whole-object selection does not require child parsing. An empty card response is valid whole-object data; a nonzero Tag Length against an empty response **SHALL** be a single byte in the range -; multi-byte DER tags are not supported and **MUST** be retrieved by issuing a tag value of and parsing the returned object at the ACU. The data offset now supports one- or two-byte encodings. PDs produce 0x1024 (data object or tag not found).

Offset and requested length The PD **SHALL** accept either form and determine which was used based on the command length. The offset applies to the selected payload (entire object when the tag is zero, or the chosen tag otherwise). When a credential-side error occurs, PDs apply Offset and Requested Length once, before dividing the data into fragments. Offset zero identifies the first byte of the selected object's or element's tag. The PD **SHALL** start at Offset and return no more than Requested Length bytes. Requested Length zero requests all remaining bytes. If fewer bytes remain, the PD **SHALL** return those bytes. The resulting bytes are the requested data. Offset equal to the selected-data length produces an empty success; Offset beyond that length produces 0x0005 (invalid parameter). The requested data **SHALL NOT** exceed 65,535 bytes. An oversized through-end result produces 0x1029 (transfer length exceeded); a bounded range within a larger object is permitted. A nonzero Offset can skip tag and length bytes. The requested range may begin or end within a tag, length, or value. The PD **SHALL** return the unchanged osdp_PIVDATAR reply (for example with MpFragmentSize = 0) to maintain OSDP 2.2 compatibility. If available, PDs those bytes unchanged. The ACU is responsible for interpreting a partial result.

Figure 1 shows how selection and fragmentation relate.

Acceptance and completion The PD **MAYSHALL** report additional detail via acknowledge an accepted operation with osdp_PDERRORACK. Acceptance does not indicate successful card access or parameter validation. The PD **SHALL** return the result on a subsequent osdp_POLL using code when security prerequisites are not satisfied, when the requested object or tag is absent, or when the credential is not present or is removed; see Subsection 2.2. ACUs: osdp_PIVGETDATAR for success or osdp_PIVERROR for failure. While processing is incomplete, the PD **SHALL** tolerate an optional answer polls with osdp_PDERRORACK following that reply; OSDP 2.2 ACUs may log and ignore the unknown reply code. Unsupported commands and transport faults retain their OSDP osdp_NAK responses.

1 CMND Command identifier. 0xA3-3 PIV Object Identifier Three-byte identifier of the PIV data object per SP-800-73-4, encoded MSB first. Leading zero bytes SHALL be supplied by the ACU when the identifier is shorter than three bytes; the PD SHALL drop those pad bytes before issuing the APDU. 0x00-0xFF¹ 1 PIV Tag Single-byte ASN.1 DER tag of a root-level element. Tag value selects the entire object; non-zero tags select a root-level element and require the PD to strip the outer /wrapper, verify the length, and return only the tagged value. Multi-byte tags are not directly addressable and SHALL be retrieved by requesting tag - 0x00; or 0x01-0xFF 1-2 Data Offset Byte offset within the requested content. Single-byte and two-byte encodings

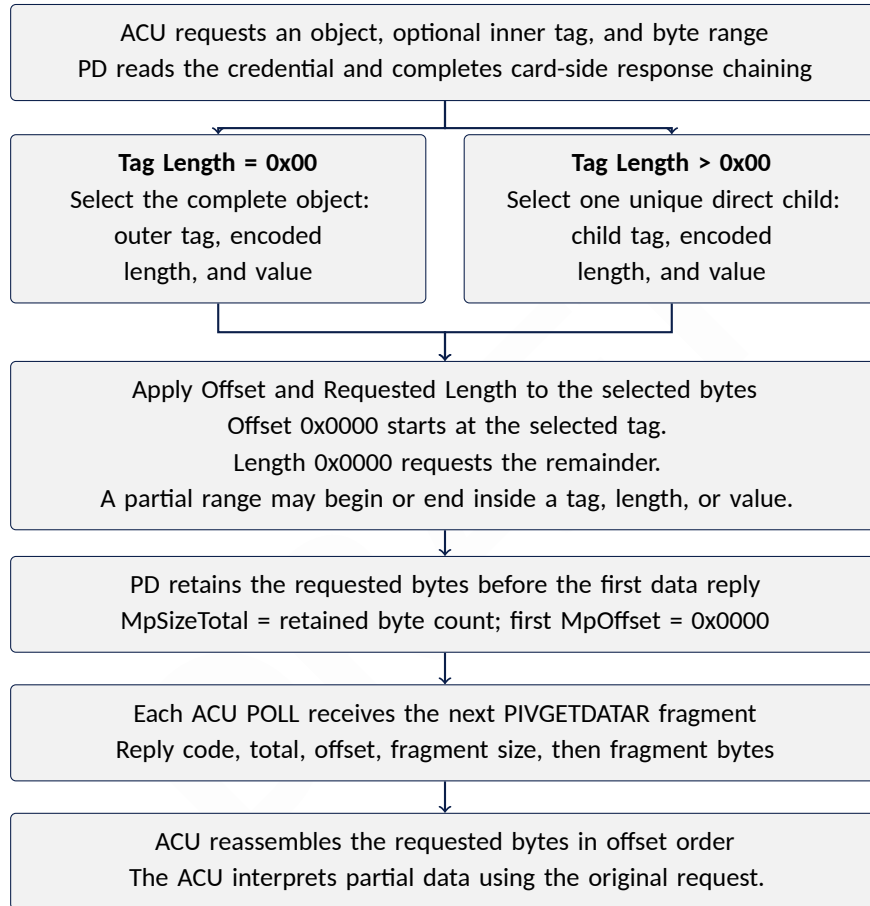


Figure 1: GET DATA selection precedes byte-range selection and OSDP fragmentation. ACK acknowledges acceptance. A failed read or selection ends with PIVERROR on a poll instead of data fragments. Child-selection errors are defined in the text.

~~SHALL be accepted; the PD determines the format from the packet length. 0x00-0xFFFF. Only one operation may be outstanding at that PD during this read. Until the read completes or is aborted, the ACU SHALL send only osdp_POLL or osdp_ABORT to that PD. The PD SHALL reject a competing command with osdp_NAK 0x09 (unable to process command record) without replacing the active operation. Traffic to other PDs is unaffected. On osdp_ABORT, the PD SHALL discard pending data or errors, cancel further delivery, and return osdp_ACK. It SHALL NOT subsequently report completion for that operation.~~

2.1.4 PIV Data Reply (osdp_PIVDATAR)

The reply structure is unchanged from a field-count perspective, but OSDP 3.0 aligns the semantics with the command refinements above:-

~~MpSizeTotal, MpOffset, and MpFragmentSize continue to be encoded LSB first; they now explicitly report the size, fragment base, and fragment length for the content selected by the osdp_PIVDATA request (entire object or chosen tag). The PD SHALL prepare and retain all requested data before sending the first data fragment. The data SHALL come from one credential operation. The PD SHALL NOT combine data from different credential presentations. Once delivery begins, the retained bytes remain unchanged. Card removal before all requested data is retained produces 0x1026 (credential not present or removed); removal afterward does not change the retained data. A card-operation timeout produces 0x0002 (smart-card timeout), insufficient PD resources produce 0x1021 (insufficient PD memory or storage), and unsatisfied credential security conditions produce 0x1023 (credential security status not satisfied). A failure produces one terminal CARDosdp_DATAPIVERRORSHALL convey the exact bytes obtained from the credential after applying any tag-stripping specified by the command. No additional framing or padding is permitted. , not an empty osdp_PIVGETDATAR followed by optional error detail. Error definitions appear in Subsection 2.2.~~

1-

2.1.4 PIV Data Reply (osdp_PIVGETDATAR)

Table 4: PIV Data Reply (osdp_PIVGETDATAR)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Reply identifier.	0x80
<u>0x02</u>	MpSizeTotal	Length in bytes of the complete data object or tagged element; <u>least-significant Total number of requested data bytes across all fragments. Least-significant</u> byte first.	0x0000-0xFFFF
<u>0x02</u>	MpOffset	Offset in bytes Position of this fragment within the complete payload; <u>least-significant's first byte within the requested data. Least-significant</u> byte first.	0x0000-0xFFFF

Size (bytes)	Name	Meaning	Value
0x02	MpFragmentSize	Length in bytes of the data carried in this fragment, least-significant- <u>Number of Card Data bytes in this reply. Least-significant byte first.</u>	0x0000–0xFFFF
0x00-n	Card Data	Requested data bytes exactly as returned by the credential. When a tagged element was selected, this sequence contains the complete selected BER-TLV, including tag, length, and value. The bytes returned in this reply, in their original order.	0x00–0xFF

The PD **SHALL** transfer the requested data using the OSDP 3.0 multi-part structure and usage rules. The ACU retrieves each fragment with `osdp_POLL`. Every reply contains the reply identifier, the three two-byte multi-part fields, and the next portion of the requested data. The multi-part fields use least-significant byte first, in the order shown in the table.

MpSizeTotal **SHALL** specify the number of requested data bytes, not the original object length. Its value **SHALL** remain unchanged throughout the transfer. *MpOffset* identifies the first byte of this fragment within the requested data. The first fragment **SHALL** have *MpOffset* zero, regardless of Offset in the command. Each subsequent fragment **SHALL** start immediately after the preceding fragment. *MpFragmentSize* **SHALL** equal the number of data bytes in that reply.

Fragment boundaries may occur anywhere within the requested data, including within a tag or length field. The PD **SHALL NOT** add or repeat TLV headers for individual fragments. The ACU reconstructs the requested data by joining the fragments in order. Reply identifiers, multi-part fields, and OSDP packet headers and trailers are not part of that data and are not counted in *MpSizeTotal* or *MpFragmentSize*.

For example, a request for 0x0104 bytes starting at Offset 0x0100 produces *MpSizeTotal* 0x0104. With 0x0072 data bytes per reply, the three fragments have *MpOffset* values 0x0000, 0x0072, and 0x00E4, and sizes 0x0072, 0x0072, and 0x0020.

For an empty success, the PD **SHALL** return one `osdp_PIVGETDATAR` with all three multi-part fields zero and no data bytes. The ACU **SHALL** interpret this first reply as successful completion with an empty result. For `PIVGETDATAR`, the early-termination indication applies only when *MpSizeTotal* is nonzero. For a nonempty result, delivery completes when the distinct accepted fragment lengths sum to *MpSizeTotal*. Normal OSDP retransmission rules apply: replaying a reply does not advance the data offset or execute the card operation again. The ACU **SHALL** discard partial data on cancellation or the OSDP 3.0 early-termination indication (zero fragment size and offset at or beyond total). That indication is not successful completion of a nonempty transfer.

2.2 ~~PD~~PIV Error Codes~~for PIV~~

PD error codes are carried in osdp_~~PDERROR~~PIVERROR using two-byte codes (least-significant byte first). The common PD codes remain available~~(for example: 0x0001 SC_ERROR with optional SW1|SW2 (smart-card error), 0x0002 SC_TIMEOUT (smart-card timeout), 0x0004 TLV_MISSING (required TLV missing), 0x0005 INVALID_PARAM (invalid parameter), 0x0006 SECURE_CHANNEL_REQUIRED (secure channel required), 0x0007 MP_INVALID (invalid multi-part transfer), 0x0008 MP_OUT_OF_SEQUENCE, (multi-part fragment out of sequence), and 0x0009 MP_OUT_OF_BOUNDS (multi-part fragment out of bounds))~~. The following PIV-specific codes occupy the 0x1000-0x10FF range:

Table 5: Enhanced PIV Error Codes

Code	Description
0x1020	Function not supported by credential (PD capability gaps SHALL use osdp_NAK function-not-supported).
0x1021	Insufficient PD memory or storage to complete the requested operation.
0x1022	Insufficient credential memory or storage space.
0x1023	Credential security status not satisfied.
0x1024	Data object or tag not found.
0x1025	VCI not established.
0x1027 <u>0x1026</u>	Credential not present or removed.
<u>0x1027</u>	<u>Ambiguous element selection: more than one direct child matches the requested tag.</u>
<u>0x1028</u>	<u>Malformed credential-object encoding encountered during element extraction.</u>
<u>0x1029</u>	<u>Selected response exceeds the supported transfer length.</u>

2.2.1 ~~PD~~Application Error Reply (osdp_PDERROR)

The assignments are tentative. Unlisted codes remain unassigned by this proposal.

~~PDs-~~

2.2.1 ~~PIV~~ Error Reply (osdp_PIVERROR)

~~The PD SHALL use osdp_PDERROR only for credential-side or other global application errors~~ PIVERROR for errors in PIV operations that are not ~~already defined in~~ reported by a command-specific ~~reply. The error code is two bytes, encoded result. Each reply contains one two-byte error code, least-significant byte first. When a card/APDU failure occurs (for example SC_ERROR or a PIV-APDU failure), the PD MAY append the ISO/IEC 7816 status word (SW1|SW2) when it is available to the application layer; otherwise the status word is omitted. A single error is reported per osdp_PDERROR reply. Transport-level faults (framing, CRC, sequence)~~ The originating command defines when the error is returned; PIVGETDATA

errors are returned on a later poll. Transport faults continue to use osdp_NAK. This reply replaces the name osdp_PDERROR; its tentative byte value 0x8A and payload structure are unchanged.

4-

Table 6: PIV Error Reply (osdp_PIVERROR)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Reply identifier.	0x8A (tentative)
<u>0x02</u>	Error Code	Two-byte error code, least-significant byte first.	0x0001–0x10FF
0 or 2 <u>0x00 or 0x02</u>	Status Word	Optional ISO/IEC 7816 status word for card/APDU failures; present only when the PD can surface it to the application layer <u>SW1 followed by SW2 when a card command returned the failure being reported. Omitted for failures detected locally by the PD. See the status-word rules below.</u>	–

When a credential command returns the failure being reported, the PD **SHALL** include that command's status word in SW1, SW2 order. For a valid protected response, the PD **SHALL** use the status word inside the SM response. The PD **SHALL** omit the status word for failures detected locally, including invalid OSDP parameters, missing trust anchors, timeouts, card removal, and failed verification of a protected response. A success status from an earlier credential command **SHALL NOT** be attached to a later failure.

2.3 Updated Function Codes

2.3.1 Function Code 12 – Smart Card Support

Function Code 12 advertises smart-card transport capabilities and profile support at the PD. The compliance byte uses the following bit assignments:

Table 7: Function Code 12 Compliance Flags

Mask	Description
0x01	PD supports transparent reader mode.
0x02	PD supports extended packet mode.
0x04	PD supports the OSDP 3.0 Enhanced PIV profile, including osdp_PIVGETDATA refinements and related commands.
0x08	PD supports OSDP 3.0 autonomous PIV operation (PIV Auto).

Bits not listed are reserved and **SHALL** be transmitted as zero.

2.3.2 Function Code 13 – Reader Interfaces

OSDP 2.2 defined Function Code 13 as a reader count indicator with a compliance byte fixed at 0x00. Enhanced PIV-capable PDs **SHALL** continue to populate the reader count field, but they **MAY** set bits within the compliance byte to advertise the downstream credential technologies they expose. A value of 0x00 remains valid and retains the original OSDP 2.2 meaning of “interface presence unspecified.”

When a PD reports specific interfaces, it **SHALL** encode the compliance byte using the flags in Table 8. The reader count byte continues to report the number of attached credential interfaces.

Table 8: Function Code 13 Compliance Flags

Mask	Description
0x01	PD presents a contact card interface (ISO 7816 or equivalent).
0x02	PD presents a high-frequency contactless credential interface (13.56 MHz NFC).
0x04	PD presents a low-frequency credential interface (approximately 125 kHz prox).
0x08	PD supports <u>credential Secure Messaging and Virtual Contact Interface (VCI)</u> secure messaging for applicable for applicable interfaces and modes.
0x10	PD presents a Bluetooth credential interface.
0x20	PD presents a barcode credential interface.
0x40	PD presents an ultra-high-frequency credential interface (UHF/RAIN RFID).

All other bits are reserved for future use and **SHALL** be transmitted as zero.

2.3.3 Function Code 14 – Biometric Support

Function Code 14 remains backward compatible with OSDP 2.2. A compliance value of 0x00, 0x01, 0x02, or 0x03 retains its original meaning and indicates that the PD does not support the OSDP 3.0 biometric command set. Devices that support the enhanced commands **SHALL** set Bit 7 (0x80). When Bit 7 is asserted, the remaining bits of the compliance byte are interpreted as a capability bitfield. Bits not explicitly defined in Table 9 are reserved and **SHALL** be transmitted as zero.

Table 9: Function Code 14 Compliance Flags

Mask	Description
0x80	PD supports the OSDP 3.0 biometric command set.
0x01	PD performs off-card fingerprint comparison using FIPS 201 BIO ANSI INCITS 378 templates.
0x02	PD performs on-card fingerprint comparison using FIPS 201 OCC processes.
0x04	PD performs off-card iris comparison aligned with FIPS 201 BIO guidance.
0x08	PD performs off-card facial comparison using ANSI INCITS 385-2004 images stored on the credential.

3 New or Enhanced PIV Commands

This section introduces commands defined exclusively for the OSDP 3.0 Enhanced PIV profile.

Unless specified otherwise, each multi-part command or reply defined in this section places *MpSizeTotal*, *MpOffset*, and *MpFragmentSize*, in that order, immediately after *CMND*. Each field is a two-byte unsigned integer encoded least-significant byte first. The bytes following *MpFragmentSize* are a fragment of the logical message data: *MpSizeTotal* is the complete logical-data length, *MpOffset* is the zero-based position of the fragment, and *MpFragmentSize* is the exact number of fragment bytes in the current message. The receiver **SHALL** apply the OSDP multi-part sequencing and bounds rules and reassemble the complete logical data before interpreting command-specific fields. Fixed fields shown after the multi-part fields in a packet table occur at their stated position in the reassembled data and are not repeated in every fragment. The separate *osdp_FILETRANSFER* framing remains unchanged.

3.1 Credential preparation and Secure Messaging

OSDP Secure Channel protects communication between the ACU and PD. Credential Secure Messaging (SM) protects credential commands and replies between the PD and the credential. Credential SM can operate over contact or contactless. The Virtual Contact Interface (VCI) allows a contactless credential to perform operations that otherwise require contact, subject to the credential's access rules. When the credential requires pairing, the PD establishes SM before submitting the eight-digit pairing code. Pairing is the default, recommended configuration in SP 800-73-5 Part 1, Section 5.5. The pairing code and the cardholder's PIN serve separate purposes: pairing enables VCI; PIN verification authorizes PIN-protected operations.

Credential presentation The PD **SHALL** select the configured application on an interface allowed by *osdp_PIVMODE*. For a PIV application, the PD **SHALL** inspect the SELECT response's application property template (tag 0x61). Its cryptographic algorithm template (tag 0xAC) contains algorithm identifiers in tag 0x80. The PD **SHALL** examine all such entries. An identifier of 0x27 or 0x2E advertises credential SM; SP 800-73-5 Part 2, Section 3.1.1, requires this indication when the PIV application supports SM. The PD **SHALL** distinguish a malformed response from a valid response that advertises neither SM algorithm.

Discovery Object The Discovery Object (0x7E) describes the credential's PIN and VCI policy. Its PIN Usage Policy (tag 0x5F2F) identifies the supported PIN options, the preferred PIN when both PIN types are enabled, and whether VCI requires pairing. The PD uses this information to prepare PIN and VCI operations. SP 800-73-5 Part 1, Section 3.3.2, defines the policy bytes.

The PD **SHALL** read the Discovery Object when preparing a credential-side PIN operation or attempting SM setup, unless the PD already holds the policy for the selected application on the current credential. Automatic reads **SHALL** be limited to those workflows. Collecting a PIN for later use requires no credential read. A status query reports the policy already read, or 0xFFFF when unavailable. The ACU can also request the object explicitly with *osdp_PIVGETDATA*.

In the first policy byte, mask 0x08 indicates VCI support. When VCI is supported, mask 0x04 permits VCI without a pairing code; otherwise pairing is required. If the object is absent, VCI is unavailable. The SELECT response remains the source of SM capability information.

Automatic SM preparation After selecting a PIV application, the PD checks the requirements for SM setup. If PIVMODE flag 0x02 is set, the PD supports an SM algorithm advertised by the credential, and at least one trust anchor is loaded, the PD **SHALL** attempt SM setup. This applies to both contact and contactless credentials, in commanded mode and PIV Auto.

Selecting a different application starts this check again. Selecting the same PIV application again preserves the current SM session and PIN-verification status. Reselection alone therefore requires neither new SM setup nor repeated PIN verification. The credential's access rules still apply to each operation. These selection rules follow SP 800-73-5 Part 2, Section 3.1.1.

The PD's trust store contains the anchors loaded through `osdp_PIVVCILOADTA`. The PD checks its own store before starting SM. An empty store prevents setup. If anchors are present, the PD must still find one that can validate this credential's certificate, directly or through an intermediate certificate. Loading an anchor makes it available for the next attempt. To start a new attempt with a credential already present, the ACU reapplies PIVMODE; the mode-change clearing rules apply.

The credential returns its SM card-verifiable certificate (CVC) during key establishment. The PD **SHALL** validate that certificate using a loaded anchor, directly or through an intermediate CVC, and verify the authentication cryptogram as specified in SP 800-73-5 Part 2, Section 4.1. The PD **SHALL** report SM as established only after those checks succeed.

Proceeding without SM When SM is disabled, unsupported, or unavailable because a trust anchor is missing, the PD **SHALL** check whether the operation requested by the ACU or configured in PIV Auto requires SM. If SM is required, the PD **SHALL** terminate the operation without sending the operation's command to the credential and return the defined failure reply. Otherwise, the PD **SHALL** perform the requested operation under the credential's access rules. The PD **SHALL NOT** substitute another key, algorithm, interface, or operation to avoid an SM requirement.

PIV key 0x9A requires PIN verification or a supported biometric comparison within the credential, as specified in SP 800-73-5 Part 1, Section 5.1, Table 5. The PIN-based contactless PIV Auto workflow with key 0x9A therefore requires SM, VCI, and successful PIN verification. The PD **SHALL** check those requirements in that order before sending the authentication command. Missing PIN verification returns Result 0x05, Detail 0x02 (PIN verification required). If SM is unavailable, the PD reports `osdp_PIVSTATUS` Result 0x02 (SM failure) with the applicable detail code. PIV Auto with key 0x9E can continue without SM and reports the credential-authentication result. A commanded PIN verification over contact can also proceed without SM.

The PD may start key establishment before discovering that the required trust anchor is missing. In that case, the PD **SHALL** discard any session keys derived during setup. The PD then handles the requested operation using the rules above.

In commanded mode, the ACU obtains the preparation result with `osdp_CARDSTATUS`. The PD reports the established SM and VCI state in `osdp_CARDSTATUSR`. A missing anchor leaves permitted data reads available. For example, the ACU can use `osdp_PIVGETDATA` to read the Secure Messaging Certificate Signer object (0x5FC122) and inspect the signer certificate and any intermediate CVC. The credential's own SM CVC is returned during key establishment; that CVC is separate from the signer data object. The ACU can request

osdp_CARDSTATUS Status Type 0x02 to obtain the setup outcome and the hash of the CVC received during that attempt, including when setup failed.

Failed security checks A missing or invalid credential CVC, invalid authentication cryptogram, or failure to verify an SM reply **SHALL** end the current operation with a failure result. The PD **SHALL** block further data access and authentication commands until the credential is removed, a different application is successfully selected, or PIVMODE is reapplied. PIVSELECT remains available to select a different application; selecting the same application preserves the blocked state. This prevents continued use of a session that failed its security checks. The rule also applies when automatic SM setup fails before an ACU command arrives. PIVMODE, trust-store management, and SM diagnostic requests remain available. For a standard status request (Status Type 0x01), the PD terminates the request without sending further commands or data to the credential and returns osdp_PIVERROR 0x1023 (credential security conditions not satisfied). This distinguishes a blocked credential from operation without SM.

For blocked operations, the PD reports osdp_PIVSPER 0x08 (security condition not met) for SPE, or osdp_PIVERROR 0x1023 (credential security conditions not satisfied) for commands that use PIVERROR.

If an operation requires SM, VCI, or PIN verification and that requirement is unmet, the PD **SHALL** report failure in the operation's completion reply. PIV Auto uses osdp_PIVSTATUS; SPE uses osdp_PIVSPER.

Figure 2 summarizes credential preparation and the decision to proceed with the requested operation.

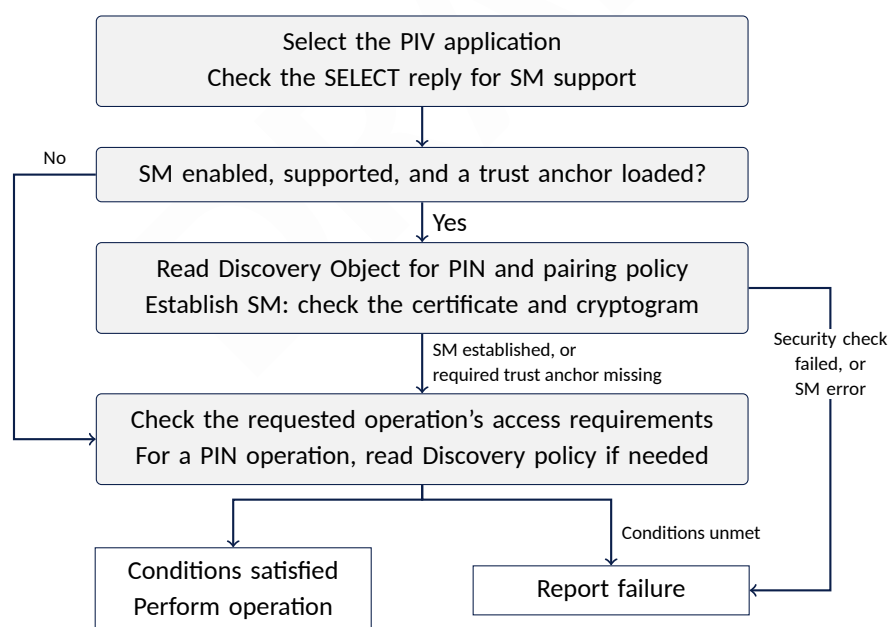


Figure 2: **How the PD prepares a credential.** The PD establishes SM when enabled and supported, using a loaded trust anchor. The credential's access rules determine which operations can proceed. Contactless PIN verification requires VCI.

Contactless operation When pairing is required, the ACU sends `osdp_PIVXMITPAIRING` after SM is established. The PD submits the pairing code to the credential through SM. A successful verification enables VCI. For a credential whose Discovery Object permits VCI without pairing, establishing SM also enables VCI. PIN verification over contactless requires VCI. Biometric comparison within the credential and pairing-code submission can use contactless SM without VCI, subject to the credential's access rules. SM and VCI preserve the credential's operation-specific restrictions, including the contact-interface restriction on ordinary PIV PUT DATA.

Protected credential commands When credential SM is established, the PD **SHALL** use that SM session to protect PIV commands that support SM. The PD **SHALL** verify and unwrap each protected reply before interpreting the credential data or status. These requirements apply to data access, authentication, PIN verification, and biometric operations. The PD **SHALL** follow SP 800-73-5 Part 2, Section 4.2, for encryption, MACs, counters, and credential-command and response chaining. The PD's GET RESPONSE exchanges follow that section's chaining rules.

The PD **SHALL** discard the SM session keys on credential reset, an SM error, or replacement by newly established session keys, as specified in SP 800-73-5 Part 2, Section 4.3. This requirement applies to the outer SW processing status defined in Section 4.2.7. The protected command status in tag `0x99` reports the result of the credential operation separately. When SM ends, the PD **SHALL** clear dependent VCI and PIN-verification status and apply the PIN-cache clearing rules.

A verified protected reply reporting a credential rejection **SHALL** end the requested operation with that failure while preserving SM. The PD **SHALL** update verification status according to the credential's response. An incorrect PIN (`0x63Cx`) clears PIN-verification status and the cached PIN; established pairing and VCI remain available. An incorrect pairing code (`0x6300`) clears pairing verification, making VCI unavailable when pairing is required. These credential rules are defined in SP 800-73-5 Part 2, Sections 3.2.1.1 and 3.2.1.3. The SM diagnostic outcome remains unchanged. A failed certificate, cryptogram, or protected-reply check follows the blocked-state rules above.

After an authenticated PIN rejection, the ACU can query remaining attempts with PIVSPE Mode `0x06`, acquire a corrected PIN, and request verification again. After a pairing rejection, the ACU can submit a corrected pairing code. Neither retry requires a new SM session.

After an SM failure, the ACU **SHALL** reapply PIVMODE to start a new SM attempt before retrying a protected operation. For a commanded retry, the ACU disables PIV Auto in that configuration, waits for SM establishment, supplies any required pairing code, and requests PIN acquisition and verification again. The PD **SHALL NOT** retry VERIFY automatically. Figures 3 and 4 show the ACU and PD exchanges after authenticated rejection. Figure 5 shows recovery after SM failure. The PD **SHALL** keep a completed operation's reply until it is delivered or canceled.

Commanded PIN verification For a PIN entered before credential presentation, the ACU first requests Mode `0x04` (acquire and cache) and waits for completion. After credential presentation, the PD prepares the credential. The ACU checks credential status and supplies any required pairing code before requesting cached-PIN verification. Figure 6 shows a successful contactless workflow with SM enabled, a suitable trust anchor loaded, and PIV Auto disabled. The ACU obtains the pairing code during enrollment or through

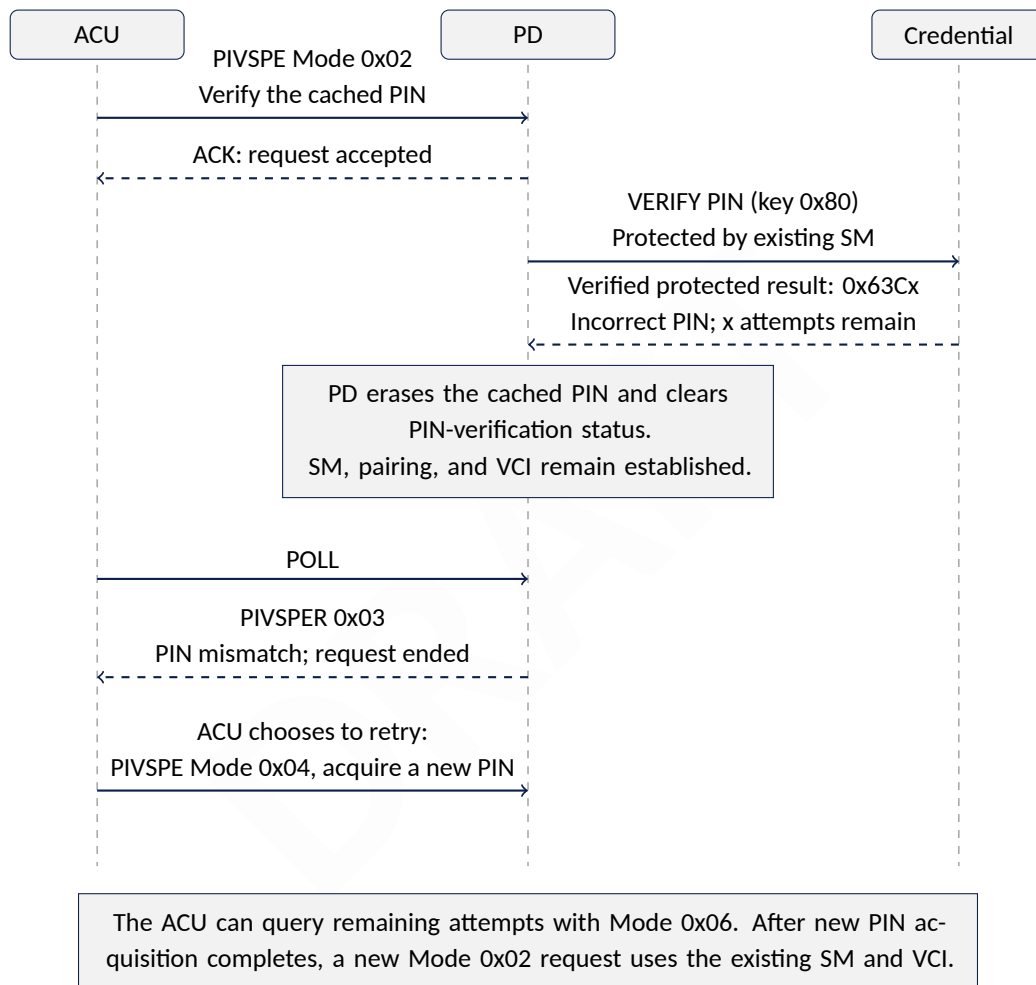


Figure 3: Incorrect PIN over established VCI. The PD verifies the protected reply before reporting the rejection. PIN entry occurs at the PD; the ACU never receives the PIN. Acquisition and its completion follow the normal ACK/POLL exchange.

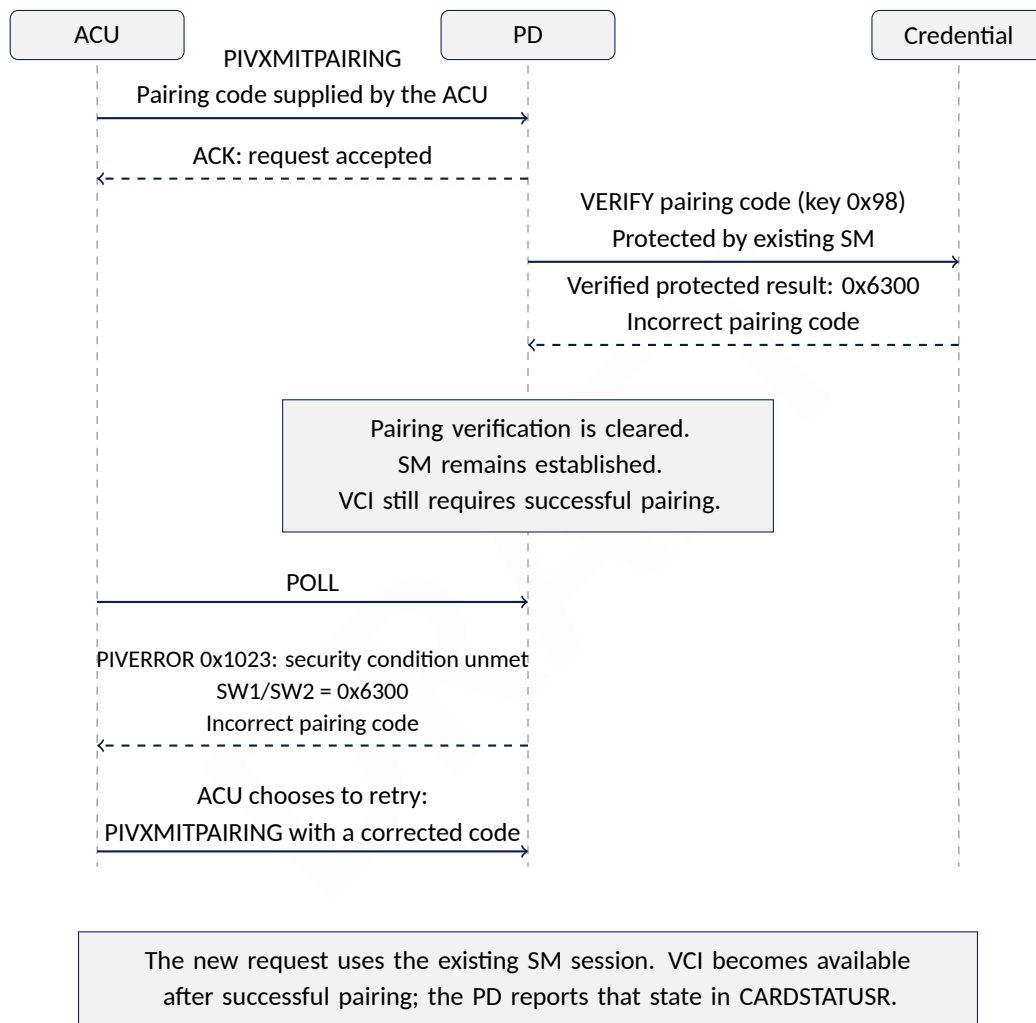


Figure 4: Incorrect pairing code on a contactless credential that requires pairing. The pairing code originates at the ACU. Each retry is a new command; the PD never repeats VERIFY automatically.

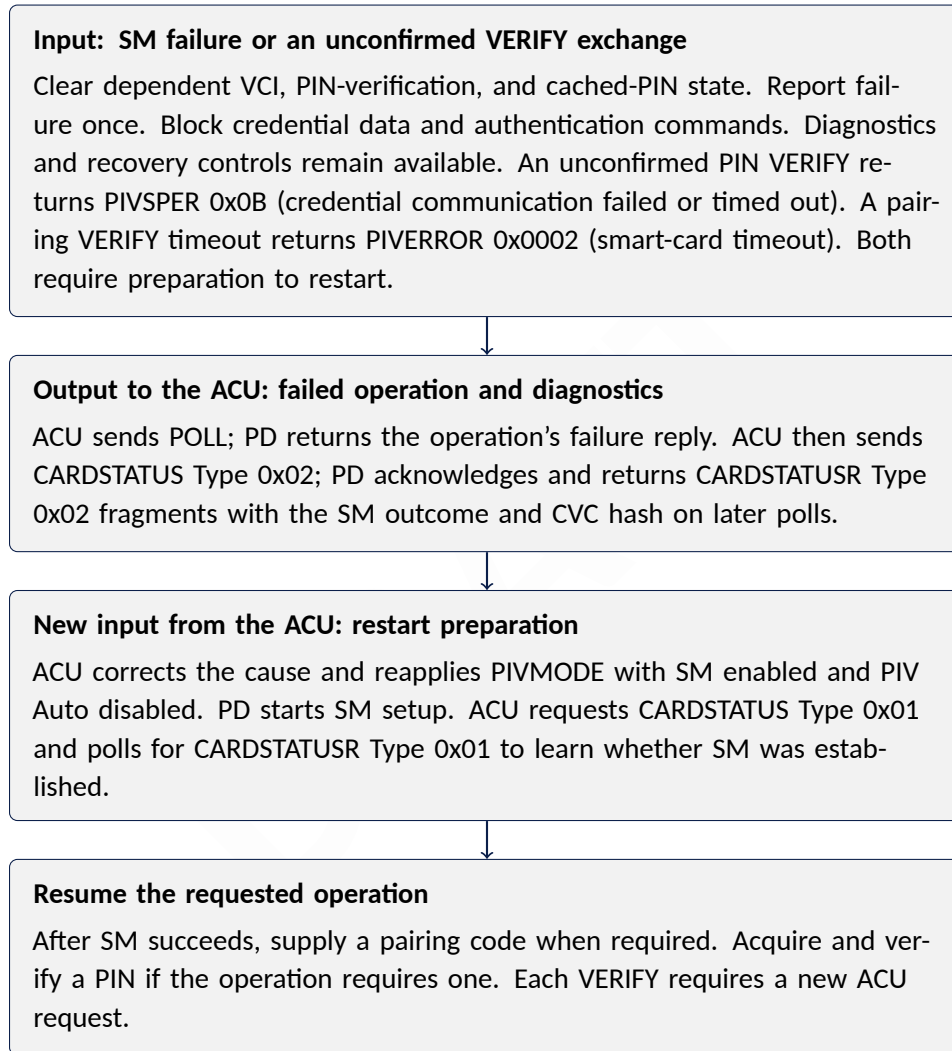


Figure 5: Recovery after SM failure or an unconfirmed VERIFY. The ACU receives the failed operation's result before requesting diagnostics and restarting preparation.

another deployment-defined process. On contact, pairing is omitted and Mode 0x03 uses the selected contact interface, with SM when established.

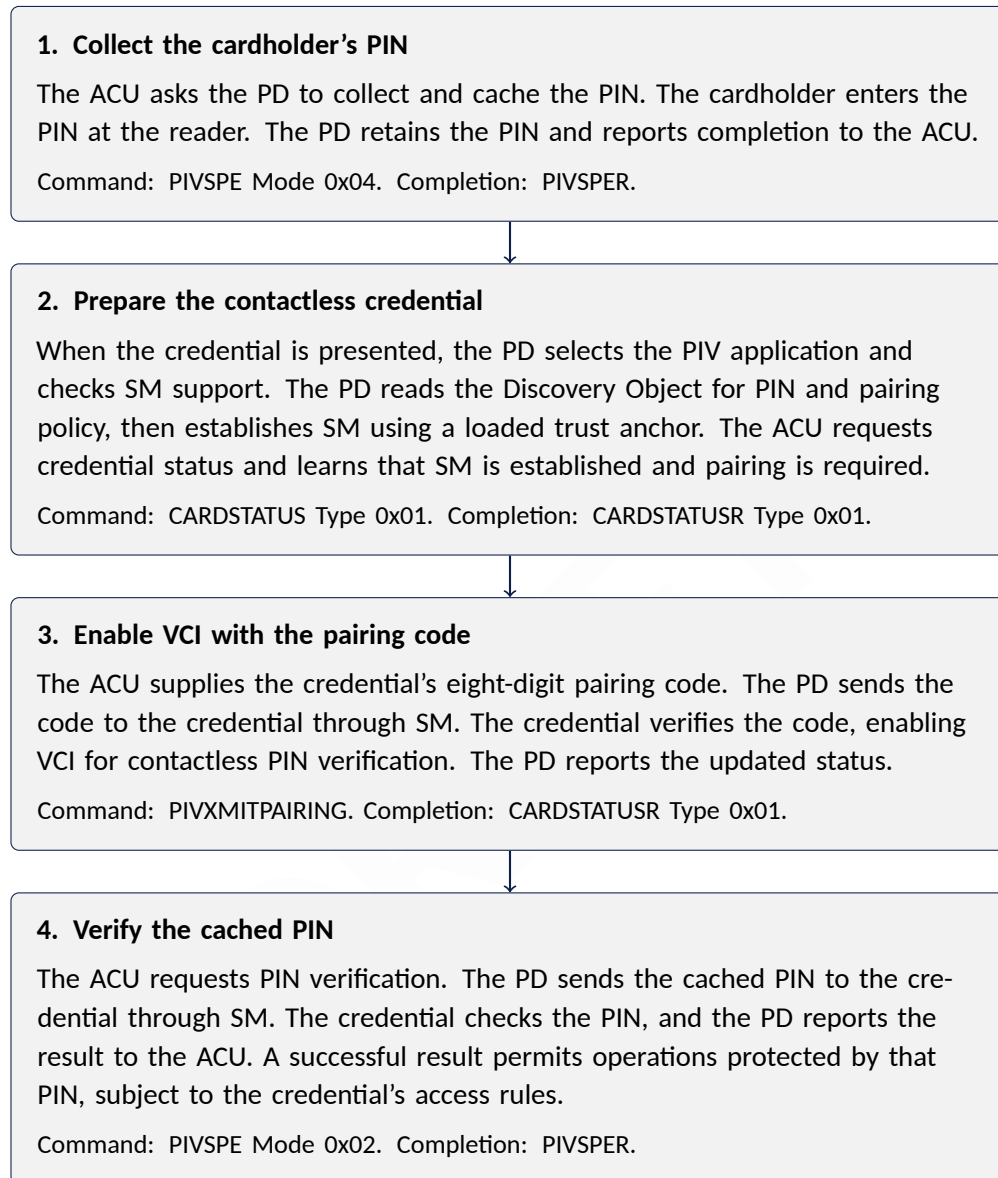


Figure 6: Contactless PIN verification with a pairing code. The ACU directs the reader (PD); the credential verifies the pairing code and PIN. Each OSDP command receives ACK when accepted and a completion reply on a later POLL. Appendix D shows the packet exchanges.

ACU recovery after a PIV Auto failure PIV Auto reports one result for the configured authentication attempt. The ACU can then request SM diagnostics, correct missing trust material, or choose another authentication workflow. Loading an anchor makes that anchor available to the PD. Reapplying PIVMODE starts a new automatic attempt and clears the previous PIN and SM state. The ACU **SHALL** finish trust recovery before pairing and PIN verification that depend on the new SM session.

When SM is established but the configured authentication requires unsupported VCI, the PD **SHALL** terminate the attempt and report `osdp_PIVSTATUS Result 0x05, Detail 0x03` (required VCI unsupported). The PD sends no authentication command for that attempt. Missing SM support is reported as `Result 0x02, Detail 0x05` (required SM unsupported). A trust-anchor update cannot add either capability. The ACU must select a supported workflow or reject the transaction. Appendix D.12 shows trust recovery followed by commanded pairing, PIN verification, and authentication.

3.2 PIV Put Data (`osdp_PIVPUTDATA`)

The `osdp_PIVPUTDATA` command (command code 0xA8) instructs the PD to deliver an ISO 7816-4 PUT DATA operation to the credential. The ACU **SHALL** construct the payload as a complete BER-TLV object, including the desired outer tag (0x7E for the Discovery Object, 0x7F61 for a BIT Group Template, 0x5C/0x53 sequences for other data objects, and so on). The PD **SHALL** forward the payload to the credential using CLA = 0x00, INS = 0xDB, P1 = 0x3F, and P2 = 0xFF. If ~~VCI-secure-messaging-credential~~ Secure Messaging (SM) is established, the PD **SHALL** ~~select the secure channel automatically use the established SM context~~ and adjust the CLA accordingly. Chained APDUs **SHALL** be generated by the PD when the payload exceeds a single-card APDU.

Table 10: PIV Put Data (`osdp_PIVPUTDATA`)

Size (bytes)	Name	Meaning	Value
0x01	CMND	Command identifier.	0xA8 (tentative)
0x02	MpSizeTotal	Total size of the complete PUT DATA payload, least-significant byte first.	0x0000–0xFFFF
0x02	MpOffset	Offset of this fragment within the complete payload, least-significant byte first.	0x0000–0xFFFF
0x02	MpFragmentSize	Number of payload bytes carried in this fragment, least-significant byte first.	0x0000–0xFFFF
0x00-n	Data	BER-TLV payload to be written to the credential. This field already includes the outer object tag selected by the ACU.	0x00–0xFF

Responses [PIV PUT DATA requires the contact interface, including when SM is established. On contactless, the PD SHALL terminate the request without sending PUT DATA to the credential and return `osdp\_PIVERROR 0x1023` \(security conditions not satisfied\).](#) A successful write is acknowledged with `osdp_ACK`. If the PD does not implement the command, ~~it the PD SHALL~~ return `osdp_NAK` function-not-supported. ~~When For~~ a credential-side failure occurs, the PD **SHALL** report ~~the failure—~~`osdp_PIVERROR`, either immediately or on a subsequent `osdp_POLL` ~~using `osdp_PDERROR` and one of the PIV error codes defined in Subsection 2.2.2.~~. The applicable codes are 0x1020 ~~when the function is~~ (function not supported

by ~~the credential~~credential), 0x1021 ~~when the PD lacks buffer space to stage the payload~~(insufficient PD buffer space), 0x1022 ~~when the credential has insufficient storage~~(insufficient credential storage), 0x1023 ~~when security conditions are not met, or when the credential is not present or is removed during the operation. The status word is optional~~(security conditions not satisfied), and 0x1026 ~~(credential absent or removed)~~. The status-word rules in Subsection 2.2.1 apply.

3.3 PIV Application Select (osdp_PIVSELECT)

The osdp_PIVSELECT command (command code 0xB0, tentative) instructs the PD to select the application identified by the supplied Application Identifier (AID) on the currently active credential. The command provides an OSDP-native selector for ISO/IEC 7816-4 SELECT by DF name; it is not a raw APDU tunnel and does not expose APDU header bytes to the ACU. PDs **SHALL** advertise support via Function Code 12 bit 0x04 (Enhanced PIV) before controllers attempt to use this command.

Table 11: PIV Application Select (osdp_PIVSELECT)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Command identifier.	0xB0 (tentative)
<u>0x01</u>	Selection Flags	Selection controls. All bits are reserved for future use and SHALL be transmitted as zero.	0x00
<u>0x01</u>	AID Length	Length of the Application Identifier.	0x05–0x10
5-16 <u>0x05-0x10</u>	AID	Application Identifier to select by ISO/IEC 7816 DF name.	0x00–0xFF

The PD **SHALL** issue SELECT with CLA = 0x00, INS = 0xA4, P1 = 0x04, P2 = 0x00, Lc = AID Length, Data = AID, and Le = 0x00. The PD **SHALL** request ~~normal FCI return data from the credential and SHALL use the active credential~~the normal application property template over the interface selected by the current PIV mode ~~or by the PD's card-present workflow. If VCI secure messaging is established and remains valid for the selected~~. A selection within an established SM context follows the shared transport rules in Subsection 3.1. ~~When the credential selects a different~~ application, the PD **SHALL** ~~select the secure channel automatically and adjust the CLA accordingly; otherwise changing applications SHALL clear VCI-dependent state, cached pairing codes, and any selected application metadata derived from the prior application. clear the previous application's SM session, VCI status, pairing code, cached PIN, PIN-verification status, and application information. This prevents the PD from carrying security state from one application into another. Selecting the PIV application when a credential is first presented preserves a PIN collected before presentation. Selecting the same PIV application again preserves its SM session and PIN-verification status. If the credential rejects SELECT, the PD keeps the current application and its state.~~

The AID Length field **SHALL** be in the ISO/IEC 7816 application identifier range of 5 to 16 bytes. If the payload is malformed, the AID length is outside that range, or any reserved selection flag is nonzero, the PD **SHALL** report osdp_PERRORPIVERROR 0x0005 (invalid parameter). A successful selection is acknowledged with osdp_ACK; after success, subsequent PIV commands operate against the selected application until the credential is removed, a new osdp_PIVSELECT succeeds, or osdp_PIVMODE changes the operating context.

The PD **SHALL** update the selected AID reported by `osdp_CARDSTATUSR`; if the credential returns an FCI template containing tag 0x4F, that value **SHALL** be reported, otherwise the PD **SHALL** report the requested AID.

If the PD does not implement the command, ~~it~~the PD **SHALL** return `osdp_NAK` function-not-supported. Credential-side failures **SHALL** be reported using `osdp_PDERRORPIVERROR`: 0x1020 ~~when the credential does not support application selection~~(application selection unsupported), 0x1023 ~~when security conditions are not satisfied~~(security conditions not satisfied), 0x1024 ~~when the requested AID is not present, or when the credential is not present or is removed during selection. The status word is optional~~(requested AID absent), or 0x1026 (credential absent or removed). The status-word rules in Subsection 2.2.1 apply.

3.4 Card Status (`osdp_CARDSTATUS`)

The `osdp_CARDSTATUS` command (command code 0xA9, tentative) requests the current credential context from the PD. PDs **SHALL** advertise support via Function Code 12 bit 0x04 (Enhanced PIV) before controllers attempt to use this command. The payload selects ~~the status record that should be returned (currently only the standard profile)~~standard credential status or SM diagnostics.

Table 12: Card Status Request (`osdp_CARDSTATUS`)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Command identifier.	0xA9 (tentative)
<u>0x01</u>	Status Type	Requested status record. 0x01 selects the standard PIV card status profile. <u>Values: standard PIV credential status. 0x02: SM diagnostic record.</u> <u>Values 0x00 and 0x03-0x7F are reserved and shall elicit return</u> <u><code>osdp_PIVERROR</code> 0x0005 (invalid parameter).</u> Values 0x80-0xFF are reserved for private use.	0x00-0xFF

PDs **SHALL** return `osdp_NAK` function-not-supported when the command itself is not implemented. If the requested status type is not supported or the payload is malformed, PDs **SHALL** report the condition via `osdp_PDERRORPIVERROR` (for example 0x0005 ~~incorrect parameter~~(invalid parameter)) on a subsequent `osdp_POLL`.

3.5 Card Status Reply (`osdp_CARDSTATUSR`)

~~In response to~~Every `osdp_CARDSTATUSR` reply **SHALL** include Status Type immediately after the reply code. Type 0x01 selects the standard credential status fields below. Type 0x02 selects the diagnostic multi-part layout in Subsection 3.5. The ACU SHALL select the reply layout from this byte and verify that the type matches the pending request. An unknown or mismatched type SHALL be treated as an invalid reply; the

ACU discards the reply and any partially reassembled record. The ACU can cancel the pending operation with ABORT and issue a new request.

The PD accepts osdp_CARDSTATUS, the PD reports a ready-to-proceed credential state after completing its card-present workflow (power, application selection, and required reads). PDs **SHALL NOT** send Type 0x01 with osdp_ACK and returns osdp_CARDSTATUSR unsolicited on a subsequent osdp_POLL. ~~Controllers **SHOULD** after the current credential preparation completes.~~ The PD **SHALL** use a finite, documented preparation timeout. If SM is established and the credential requires a pairing code, the PD reports that state so the ACU can supply the code. The ACU **SHALL** interpret the following combination as pairing required: Detected Credential Type 0x02, Credential Security Status 0x02, and an available PIN Usage Policy whose first byte has mask 0x08 set and mask 0x04 clear. Security Status 0x02 alone indicates SM without VCI; contact SM requires no pairing. While status is pending, the ACU sends only osdp_POLL or osdp_ABORT.

osdp_CARDSTATUSR also completes an accepted osdp_PIVXMITPAIRING command, using Status Type 0x01. ~~Controllers continue to use poll-based osdp_RAW replies as the single aggregated “card-presented and data-acquired” indication; PIN-usage-policy and VCI-connectivity **SHALL NOT** be included in unsolicited poll indications. Detailed readiness (credential type, selected AID, cached data, and readiness flags) **SHALL** be returned only in solicited osdp_CARDSTATUSR. When VCI for ordinary card-data indications. Whenever credential SM is established, the reply reports both the secure-messaging status reply reports the CVC IIN and the loaded anchor IIN used by the PD. These values are identical for direct CVC, including for contact SM. The values match for direct verification and differ when an Intermediate CVC is used. The PD **SHALL** set the VCI-Trust Anchor ID to the identifier assigned in osdp_PIVVCILOADTA; this value **SHALL** correspond to an entry in the SM Trust Anchor ID identifies the corresponding osdp_PIVVCITASTATUSR inventory entry, whose IIN matches the VCI SM Anchor IIN field, so that the ACU can correlate reported VCI state with the anchor-management record.~~

Table 13: Card Status Reply (osdp_CARDSTATUSR)

Size (bytes)	Name	Meaning	Value
0x01	CMND	Reply identifier.	0x85 (tentative)
0x01	Status Type	Standard credential status, including successful pairing completion.	0x01
0x01	Detected Credential Type	Current card interface: 0x00 none, 0x01 ISO 7816 contact, 0x02 ISO 14443 contactless, 0x03 other ISO 7816 credential (for example DESFire, Seos), 0x04 low-frequency credential, 0x05 unknown type, 0x06-0x7F reserved, 0x80-0xFF private use.	0x00-0xFF

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	VCI Credential Security Status	0x00 VCI not established, SM absent ; 0x01 VCI established. Values SM established and VCI available ; 0x02 SM established without VCI. Contact SM uses 0x02. Values 0x03-0x7F reserved, 0x80-0xFF private use.	0x00-0xFF
<u>0x01</u>	VCI SM Trust Anchor ID	The Trust Anchor ID used by the PD during VCI establishment. The PD SHALL set this field to the identifier- Identifier assigned in osdp_PIVVCILOADTA ; this value SHALL correspond to an entry in the inventory whose IIN matches the for the anchor used to validate credential SM. The anchor's inventory IIN matches VCI SM Anchor IIN field. Set to 0x00 when VCI is not established SM is absent .	0x00-0xFF
<u>0x08</u>	VCI SM CVC IIN	Issuer Identification Number from the secure messaging credential SM CVC tag 0x42. Set to all zero when VCI is not established SM is absent.	0x00-0xFF
<u>0x08</u>	VCI SM Anchor IIN	Issuer Identification Number of the loaded VCI trust anchor used by the PD to validate credential SM. Set to all zero when VCI is not established or no anchor was used SM is absent.	0x00-0xFF
<u>0x02</u>	PIN Usage Policy	PIN Usage Policy bytes (tag 0x5F2F) returned verbatim when available. A value of 0xFFFF indicates the policy is not present or has not yet been read.	0x0000-0xFFFF
<u>0x01</u>	Selected AID Length	Length (0-16) of the selected AID. 0x00 indicates no application is currently selected.	0x00-0x10
0-16 <u>0x00-0x10</u>	Selected AID	Application Identifier read from the credential (for example from the FCI template). Present only when the length is non-zero.	0x00-0xFF

~~A PD that cannot provide the requested status. Each accepted Type 0x01 status request SHALL return complete with either osdp_NAKCARDSTATUSR when the command is unsupported. For credential-side failures encountered while preparing status or osdp_PIVERROR. If a failed SM security check has blocked credential operations, the PD MAYSHALL omit osdp_CARDSTATUSR data and, if available, report details via terminate a Status Type 0x01 request without sending further commands or data to the credential. The PD SHALL return osdp_PDERROR on a subsequent osdp_POLL. Successful replies include the entire selected AID (when available) and the fixed two-byte PIVERROR 0x1023 (credential security conditions not satisfied) on the next poll. Otherwise, the standard status reply includes the selected AID when available and any previously read PIN Usage Policy field after the PD has completed its preparation. When PIN Usage Policy bytes from the credential's Discovery Object (bytes from Discovery Object tag 0x5F2F) are available, the PD SHALL report those two bytes verbatim. When no PIV, unchanged. The status request itself requires no Discovery Object read. The PD reports 0xFFFF when the policy is unavailable. Security status reports the result of credential preparation. With no credential present, the PD reports credential type and security status zero, AID length zero, and PIN Usage Policy is present or the policy has not yet been read, the 0xFFFF. Removal during preparation returns osdp_PIVERROR 0x1026 (credential absent or removed). A preparation timeout returns osdp_PIVERROR 0x0002 (smart-card timeout). osdp_ABORT cancels the request with osdp_ACK and clears its pending completion reply.~~

SM diagnostics (Status Type 0x02) The ACU uses this status type to determine why SM setup failed and identify the CVC received by the PD. The PD SHALL accept this request while credential operations are blocked by a failed security check. The PD SHALL report . The VCI fields describe stable readiness (for example whether a secure channel is established) and the IIN evidence the ACU needs for validation ; transient negotiation state return the most recent setup outcome and the CVC hash for the current credential and selected application. This request reads retained PD information; the PD SHALL NOT be reported. If the credential is removed before preparation completes send a command to the credential in response to the diagnostic request.

The PD acknowledges the request with osdp_ACK, then returns osdp_CARDSTATUSR on subsequent polls. Status Type 0x02 uses the OSDP 3.0 multi-part format in Table 14. Every diagnostic fragment SHALL contain Status Type 0x02 immediately after the reply code, followed by MpSizeTotal, MpOffset, and MpFragmentSize. Status Type identifies the layout before the ACU reads the multi-part fields. The repeated Status Type byte is outside the diagnostic record and SHALL NOT be counted in the multi-part lengths or offsets.

Table 14: SM Diagnostic Reply (CARDSTATUSR, Status Type 0x02)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	<u>CMND</u>	<u>Reply identifier.</u>	<u>0x85 (tentative)</u>
<u>0x01</u>	<u>Status Type</u>	<u>SM diagnostics. Repeated in every fragment, outside the diagnostic record.</u>	<u>0x02</u>
<u>0x02</u>	<u>MpSizeTotal</u>	<u>Total diagnostic record length, least-significant byte first.</u>	<u>0x0023</u>
<u>0x02</u>	<u>MpOffset</u>	<u>Offset within the diagnostic record, least-significant byte first.</u>	<u>0x0000-0xFFFF</u>
<u>0x02</u>	<u>MpFragmentSize</u>	<u>Diagnostic bytes in this reply, least-significant byte first.</u>	<u>0x0001-0xFFFF</u>
<u>0x01</u>	<u>Outcome</u>	<u>0x00: setup not attempted; 0x01: SM established; 0x02: SM unavailable; 0x03: security check or SM processing failed, credential operations blocked.</u>	<u>0x00-0x03</u>
<u>0x01</u>	<u>Detail</u>	<u>Reason from Table 15.</u>	<u>0x00-0x07</u>
<u>0x01</u>	<u>CVC Hash Present</u>	<u>0x01 when the PD received a complete CVC; otherwise 0x00.</u>	<u>0x00-0x01</u>
<u>0x20</u>	<u>CVC Hash</u>	<u>SHA-256 of the received CVC, including tag 0x7F21, its encoded length, and value, unchanged. All zero when CVC Hash Present is 0x00.</u>	<u>0x20 bytes</u>

Outcome values 0x04-0xFF and Detail values 0x08-0xFF are reserved for future use.

Table 15: SM Diagnostic Detail Codes

Value	Description
<u>0x00</u>	<u>No failure detail. Used with Outcome 0x00 or 0x01.</u>
<u>0x01</u>	<u>Trust store empty. Outcome 0x02; CVC Hash Present is zero because setup was skipped.</u>
<u>0x02</u>	<u>No matching trust path through the loaded anchors. Outcome 0x02. The ACU can inspect the signer object to determine whether an anchor or intermediate CVC is missing.</u>
<u>0x03</u>	<u>CVC missing, malformed, or invalid. Outcome 0x03. CVC Hash Present is zero when no complete CVC was received.</u>

Value	Description
0x04	Authentication cryptogram invalid. Outcome 0x03.
0x05	No credential SM algorithm supported by both the PD and credential. Outcome 0x02.
0x06	Other SM processing failure. Outcome 0x03.
0x07	SM disabled by PIVMODE. Outcome 0x02.

The diagnostic record consists of Outcome, Detail, CVC Hash Present, and CVC Hash, in that order. The PD **SHALL** assemble the complete record before dividing the record into reply fragments. The multi-part fields describe that complete record. A fragment may end within any field. The ACU **SHALL** reassemble the record before interpreting the fields.

The PD **SHALL** compute and retain the hash of a complete received CVC even when certificate or cryptogram validation fails. The hash identifies the received bytes; the hash provides no evidence that the CVC is trusted. The diagnostic record requires only the outcome, detail, presence flag, and 32-byte hash. The PD has no obligation to retain the CVC or other credential objects for later retrieval. PIVGETDATA reads the requested object from the credential through the active interface. If no complete CVC was received, the PD **MAY** report the condition via `osdp_PDERROR` (for example) instead of `osdp_CARDSTATUSR`. `osdp_CARDSTATUSR` is sent only in response to a matching `osdp_CARDSTATUS` request. **SHALL** set CVC Hash Present to zero, fill CVC Hash with zero bytes, and report the applicable reason.

The PD **SHALL** keep the diagnostic record until credential removal or reset, PD reset, successful selection of a different application, or reapplication of PIVMODE. A later SM failure updates Outcome and Detail while retaining the CVC hash from that session. Loading a trust anchor leaves the previous outcome unchanged. Before a setup attempt, the record has Outcome 0x00, Detail 0x00, CVC Hash Present zero, and an all-zero CVC Hash.

The ACU **SHALL** receive any pending PIV Auto result before requesting diagnostics. While a diagnostic reply is pending, the ACU sends only POLL or ABORT. A competing request receives `osdp_NAK` 0x09 (unable to process command). ABORT cancels the transfer with ACK. Credential removal during transfer terminates the transfer with `osdp_PIVERROR` 0x1026 (credential absent or removed); the ACU discards any partial record.

3.6 PIV Secure PIN Entry (`osdp_PIVSPE`)

The `osdp_PIVSPE` command (command code 0xAA, tentative) controls Secure PIN Entry for the Enhanced PIV profile. A PD **SHALL** advertise both Function Code 12 bit 0x04 (Enhanced PIV) and Function Code 15 Secure PIN Entry support before an ACU uses this command. The OSDP 2.2 `osdp_XWR` Mode-01 SPE command and `osdp_PR01SPER` reply remain unchanged.

The ACU is responsible for conveying a clear indication that PIN entry is in progress. While the PD is acquiring a PIN, it **SHALL** suppress `osdp_KEYPAD` reports so that no PIN digit is sent to the ACU.

Table 16: PIV Secure PIN Entry (osdp_PIVSPE)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Command identifier.	0xAA (tentative)
<u>0x01</u>	SPE Mode	Secure PIN Entry operating mode. See Table 17.	0x00–0xFF
<u>0x01</u>	Start Timeout	Seconds allowed to begin key entry; 0x00 selects a finite PD default <u>the documented PD default (a value from 0x01 through 0xFF seconds)</u> .	0x00–0xFF
<u>0x01</u>	Inter-key Timeout	Seconds allowed between keystrokes; 0x00 selects a finite PD default <u>the documented PD default (a value from 0x01 through 0xFF seconds)</u> .	0x00–0xFF
<u>0x01</u>	Flags	Behaviour modifiers. See Table 18.	0x00–0xFF
<u>0x02</u>	Cache Validity	PIN cache validity in seconds (LSB first). Mode 0x04 requires 0x0001–0xFFFF; other modes transmit 0x0000.	0x0000–0xFFFF

Table 17: SPE Mode Values

Value	Description
0x00	Report the persistent SPE status without initiating PIN entry.
0x01	Acquire a PIN and verify it with the credential currently present.
0x02	Execute VCI PIN verification using a cached PIN. <u>Verify the cached PIN over established VCI.</u>
0x03	Execute normal PIN verification using a cached PIN. <u>Verify the cached PIN over the selected credential interface. Use established credential SM when present.</u>
0x04	Acquire and cache a PIN.
0x05	Abort active PIN entry and clear the cached PIN.
0x06	Report attempts remaining without initiating PIN verification.
0x07–0x7F	Reserved for future use.
0x80–0xFF	Private use.

Table 18: SPE Flag Values

Mask	Description
0x02-0x01	Retain a cached PIN across credential removal until its validity expires or another clearing event occurs. Valid only with Mode 0x04.
0x08-0x02	Auto Execute When Credential Is Presented. Valid only with verification Modes 0x01-0x03.
0x01, 0x04, 0x10-0x80 0x04-0x80	Reserved and flag bits ; SHALL be transmitted as zero.

Modes are enumerated values, ~~not a bit mask~~. Modes 0x02 and 0x03 submit the cached PIN using PIV VERIFY. Mode 0x02 requires established VCI; Mode 0x03 uses the ~~normal~~-credential interface selected by the active PIV mode. Mode 0x01 acquires the PIN and uses the selected interface. The ACU selects Mode 0x02 to require VCI explicitly, or Mode 0x03 to permit verification on either the selected contact interface or established VCI. Both modes have the same behavior on contactless. Mode 0x02 returns 0x09 (VCI not established) on contact, even when contact SM is established; Mode 0x03 permits that workflow. When credential SM is established, the PD SHALL use that session for Modes 0x01-0x03 and 0x06. Contactless PIN verification requires VCI. Contact PIN verification may use the contact interface without SM. Mode 0x04 requires a Cache Validity from 0x0001 through 0xFFFF. All other modes **SHALL** transmit 0x0000 in that field. The PD **SHALL NOT** transmit an entered or cached PIN to the ACU.

A zero Start Timeout or Inter-key Timeout selects ~~a finite, PD-defined default; neither value permits an infinite timer. The remaining timeout the PD's documented default.~~ Each default SHALL be from 1 through 255 seconds. Nonzero values specify seconds and SHALL override the corresponding default. The ACU should supply explicit values when deployment or test requirements specify the timing. Fields that do not apply to a selected mode **SHALL** be transmitted as zero.

Only one osdp_PIVSPE operation may be outstanding. Modes 0x01-0x04 and 0x06 each initiate one operation. The PD returns osdp_ACK when it accepts the request, then returns one osdp_PIVSPER in response to a later osdp_POLL. While such an operation is pending, the ACU **SHALL** send only osdp_POLL, Mode 0x05, or osdp_ABORT. Returning osdp_PIVSPER ends the operation unconditionally, including after a PIN mismatch.

Mode 0x00 returns osdp_PIVSPER directly. It reports only persistent PIN state; it does not report transient active or waiting states. An ACU recovering from an interrupted session first uses Mode 0x05 or osdp_ABORT, then queries Mode 0x00. Mode 0x06 is accepted with osdp_ACK; the PD reads the present credential without performing PIN verification and returns the count in a subsequent osdp_PIVSPER. This query **SHALL NOT** decrement the credential retry counter. Auto Execute does not make the count available without a present credential. The PD SHALL return 0xFF (attempts unavailable) when the credential is absent or the active interface and security context do not permit the query. Contactless queries require VCI.

Mode 0x05 is idempotent. It aborts active or partial PIN entry, clears the cached PIN, and completes with `osdp_ACK`; the PD **SHALL NOT** send a later `osdp_PIVSPER` for the canceled operation. `osdp_ABORT` cancels active or partial PIN entry but preserves a cache that was completed before the active operation.

~~A cached PIN is used, not consumed. It remains available until its validity expires or it is cleared.~~ successful verification SHALL preserve the cached PIN until expiration or a clearing event. A later explicit verification request may reuse that PIN. The PD **SHALL** clear ~~it the cache~~ on Mode 0x05, PIN mismatch, PD reset, loss of the OSDP ~~secure channel or applicable security context~~ Secure Channel, or a change to the PIV mode or ~~selected application.~~ a change from one selected application to another. Initial application selection and initial SM establishment SHALL preserve a PIN cached before credential presentation. Credential removal also clears it unless flag 0x01 was supplied with Mode 0x04. Loss of credential SM while the credential remains present clears the cache. Removal follows the removal flag even when it also ends SM. PIN-verified status is bound to the current credential, selected application, interface, and credential security context. The PD **SHALL** clear that status on credential removal or reset, ~~application selection or reselection~~ a change to the selected application, interface change, PIV mode change, or ~~VCI teardown~~ loss of credential SM or VCI, and **SHALL NOT** infer it only from an earlier successful reply.

~~Flag-~~

PIN cache expiration The PIN cache holds a PIN in the PD for later verification by a credential. The PD sends the PIN to the credential in a VERIFY command. Successful verification allows operations that require the PIN, subject to the credential's access rules.

Cache Validity limits how long the PD stores the PIN for reuse. When the cache expires, the PD erases the stored PIN. A credential that has already accepted the PIN remains PIN verified until the credential's rules clear that state. Mode 0x00 reports both whether the PD has a usable cached PIN and whether the current credential has accepted PIN verification (Table 20).

Starting and expiring the cache. The PD **SHALL** start the cache timer when PIN acquisition completes successfully, using elapsed time unaffected by clock adjustments. Delivery of the completion reply, status queries, and verification leave the deadline unchanged. A new successful acquisition starts a new deadline. At expiration, the PD **SHALL** erase the cached PIN and mark the cache unavailable.

Expiration before VERIFY. Immediately before submitting VERIFY to the credential transport, the PD **SHALL** check the cache deadline. If the deadline has been reached, the PD **SHALL** send no VERIFY and return `osdp_PIVSPER` 0x0A (no cached PIN). The PD **SHALL** make the deadline check and submission decision as one indivisible step. At the deadline, the PIN is expired and the PD **SHALL NOT** submit VERIFY.

Expiration after VERIFY. Once VERIFY has been submitted, the PD **SHALL** finish the exchange even if the cache expires. The PD reports the credential's verification result, 0x0B (credential communication failed or timed out), or 0x04 (credential removed), as applicable. The PD may retain a command copy needed to finish the exchange under the credential transport rules and **SHALL** erase that copy when the exchange ends. Each new VERIFY requires a new ACU request.

VERIFY without a confirmed result. If communication fails after VERIFY submission, the PD cannot determine whether the credential accepted the PIN. The PD **SHALL** clear its reported PIN-verification state and cached PIN, end any SM session, and require credential preparation to be restarted before another data

or authentication operation. Restarting preparation restores a known security context after an exchange whose outcome is unknown. A verified credential reply determines the reported result even if delivery to the ACU is delayed.

Reporting expiration to the ACU. While idle, the PD erases an expired cached PIN without sending a reply. The ACU learns the current state from a status query or the result of a later verification request. If Auto Execute is waiting for credential presentation, expiration **SHALL** end that request with 0x0A on a later poll. An accepted verification request receives ACK followed by one terminal PIVSPER, unless the ACU cancels the request. Expiration after the operation finishes leaves its pending completion reply unchanged.

Figure 7 shows how expiration affects VERIFY. Appendix D.5 provides packet and timing examples.

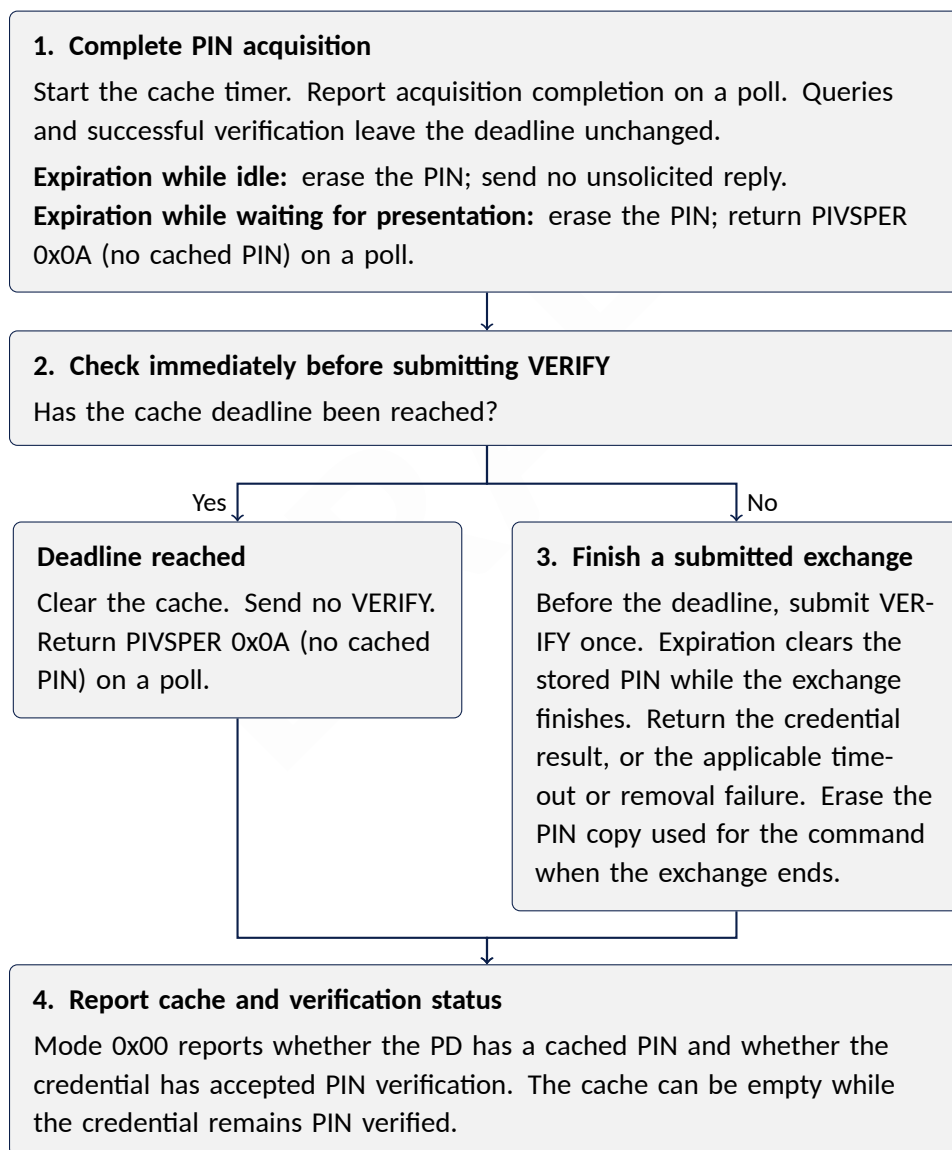


Figure 7: PIN cache lifetime and the VERIFY submission boundary. Idle expiration is silent. Expiration while waiting for presentation ends the pending request with PIVSPER 0x0A on a poll.

Flag 0x02 arms the verification selected by Mode 0x01, 0x02, or 0x03 to execute when a credential is presented. It **SHALL** be zero for Modes 0x00, 0x04, 0x05, and 0x06. A cached auto-execute workflow first acquires the PIN with Mode 0x04, then selects normal or VCI verification with Mode 0x03 or 0x02 and flag 0x02.

~~The PD~~ Without Auto Execute, verification begins after acceptance with the credential and required security context already available. An absent credential returns Result 0x04 (credential absent or removed). When Mode 0x02 is requested or verification uses contactless, missing VCI returns 0x09 (VCI not established). The ACU completes any required pairing before requesting verification.

With Auto Execute, the PD waits for presentation, completes credential preparation (Subsection 3.1), and attempts verification once. If preparation requires a pairing code, the PD completes the SPE operation with 0x09 (VCI not established). If the cached PIN expires while waiting, the PD completes with 0x0A (no cached PIN). A failed verification completes the operation without an automatic retry. The ACU may then perform setup and start a new SPE operation.

The ACU **SHALL** disable PIV Auto before arming SPE Auto Execute. A request to arm SPE while PIV Auto is enabled receives osdp_NAK 0x09 (unable to process command). This ensures that only one operation starts credential preparation and PIN verification when the credential is presented.

The PD **SHALL** reject an unsupported command with osdp_NAK 0x03, an incorrect command length with osdp_NAK 0x02, and an invalid mode, reserved bit, or field combination with osdp_NAK 0x09. Accepted credential-side outcomes are reported through osdp_PIVSPER, not osdp_~~PDERROR~~PIVERROR.

Figure 8 summarizes reply timing and interpretation.

3.7 PIV Secure PIN Entry Reply (osdp_PIVSPER)

The osdp_PIVSPER reply (reply code 0x86, tentative) contains one mode-dependent *Result* byte. ~~Removing the former third byte affects only this Enhanced PIV reply; the OSDP 2.2 osdp_PR01SPER. Tries byte is unchanged.~~

Table 19: PIV Secure PIN Entry Reply (osdp_PIVSPER)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Reply identifier.	0x86 (tentative)
<u>0x01</u>	Result	Mode-dependent SPE status, attempts count, or operation result.	0x00–0xFF

For Mode 0x00, *Result* is the status bitmap in Table 20. For Mode 0x06, it is the attempts value in Table 21. For Modes 0x01–0x04, it is the operation outcome in Table 22.

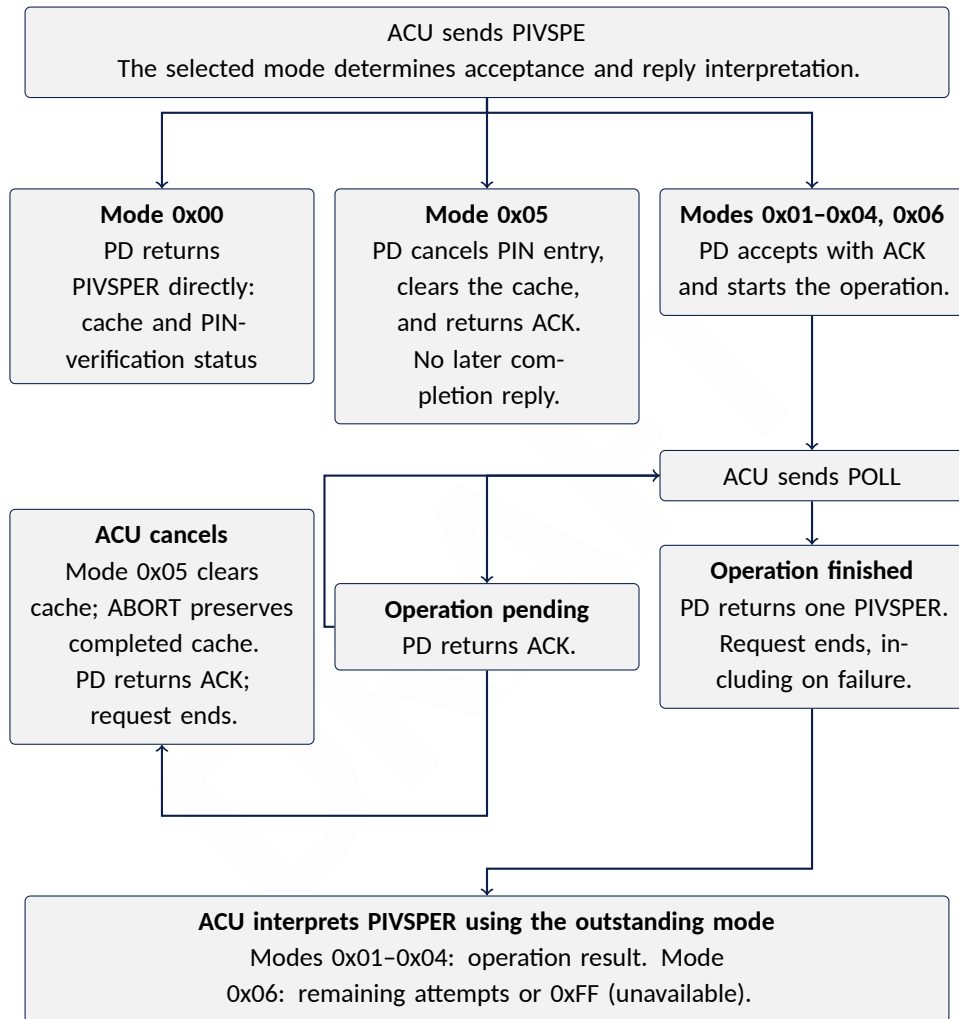


Figure 8: SPE request and reply lifecycle. While an operation is pending, the ACU sends only POLL, Mode 0x05, or ABORT. ABORT cancels the operation while preserving a previously completed cache. An unsupported or malformed request follows the NAK rules.

Table 20: PIVSPE Status Result (Mode 0x00)

Value	Description
0x00	No cached PIN is present and the current credential is not PIN verified.
0x01	A cached PIN is present.
0x02	The current credential and selected application are PIN verified.
0x03	A cached PIN is present and the current credential and selected application are PIN verified.
0x04–0x7F	Reserved for future use.
0x80–0xFF	Private use.

Table 21: PIVSPE Attempts Result (Mode 0x06)

Value	Description
0x00	The credential PIN is blocked.
0x01–0xFE	Number of PIN verification attempts remaining.
0xFF	Attempts remaining are unavailable, including when no credential is present.

Table 22: PIVSPE Result Codes

Value	Description
0x00	Requested operation completed.
0x01	Timeout – PIN entry did not begin within the allotted time.
0x02	Timeout – inter-digit interval exceeded.
0x03	PIN mismatch.
0x04	Credential <u>absent or</u> removed during operation.
0x05	Credential PIN blocked.
0x06	Wrong interface for the requested mode.
0x07	Incorrect parameters.
0x08	Security condition not met.
0x09	VCI not established for the requested mode.
0x0A	No cached PIN is available.
0x0B– 0x7F	<u>Credential communication failed or timed out before a verified completion was received.</u>
<u>0x0C–0x7F</u>	Reserved for future use.
0x80–0xFF	Private use.

3.8 PIV Biometric Match (osdp_PIVBIOMATCH)

The `osdp_PIVBIOMATCH` command (command code 0xAB, tentative) instructs the PD to acquire a live sample and compare it to the specified biometric template from the active application. Depending on the *Match Mode*, the comparison occurs either on the credential (on-card match) or within the PD (off-card match).

Table 23: PIV Biometric Match (osdp_PIVBIOMATCH)

Size (bytes)	Name	Meaning	Value
0x01	CMND	Command identifier.	0xAB (tentative)
0x01	Biometric Type	Selects the template type to compare. See Table 24.	0x00–0xFF
0x01	Match Mode	0x00 off-card match; 0x01 on-card match.	0x00–0x01

Table 24: Biometric Type Values

Value	Description
0x01	Cardholder fingerprint template.
0x02	Cardholder facial image.
0x04	Cardholder iris template.

For contactless on-card comparison, the PD **SHALL** use credential SM. This operation can proceed with SM established and VCI absent; pairing and VCI are required only when the credential's access conditions require them. Other biometric operations use established SM when present.

Controllers set *Match Mode* to select off-card or on-card comparison. Off-card flows require the PD to retrieve the appropriate template and, where applicable (e.g., TWIC contactless), to have a valid privacy key loaded via `osdp_PIVLOADTPK`. On-card flows require the credential to support OCC. If the PD does not implement the command, it **SHALL** return `osdp_NAK` function-not-supported. Credential-side match outcomes (for example PIN verification required or credential removal during operation) **SHALL** be reported in the subsequent `osdp_PIVBIOMATCHR` result code per Table 26. Conditions that prevent the command from being attempted (for example unsupported credential function, missing VCI when required, or invalid command parameters) **MAY** be reported via `osdp_PERRORPIVERROR` on a subsequent `osdp_POLL`.

3.9 PIV Biometric Match Reply (osdp_PIVBIOMATCHR)

After `osdp_PIVBIOMATCH` completes, the PD reports the result as a poll response using reply code 0x87 (tentative).

Table 25: PIV Biometric Match Reply (osdp_PIVBIOMATCHR)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Reply identifier.	0x87 (tentative)
<u>0x01</u>	Result	Outcome of the biometric match. See Table 26.	0x00–0xFF
0 or 32 <u>0x00 or 0x20</u>	Hash	32-byte <u>0x20-byte</u> SHA-256 hash of the biometric template tag 0xBC data field. The PD SHALL include this field only when <i>Result</i> is 0x00 (off-card match success). The PD SHALL omit this field for on-card match success (0x09) and all failure result codes.	0x00–0xFF

When *Result* is 0x00 (off-card match success), the PD **SHALL** include the 32-byte SHA-256 hash of the biometric template tag 0xBC data field in the reply. For on-card match success (*Result* 0x09) and for all failure result codes, the PD **SHALL** omit the hash field; ACUs **SHALL NOT** rely on a hash received alongside any result other than 0x00.

Off-card biometric comparison demonstrates only that the live sample matched template data presented to the PD. Because off-card biometric objects can be copied or duplicated, an off-card biometric match does not by itself prove authenticity of the credential. On-card biometric comparison is a credential-controlled assertion; the biometric object may be attacker controlled and the PD does not receive the template data being matched. To obtain cryptographic proof associated with an on-card biometric success, the ACU **SHALL** perform an additional authentication using an appropriately verified credential key after the successful on-card comparison.

Table 26: PIVBIOMATCH Result Codes

Value	Description
0x00	Biometric match success (off-card match).
0x01	Biometric match timeout (no entry started).
0x02	Biometric match failed (no match).
0x03	PIN verification required.
0x04	Credential removed during operation.
0x05	PIN verification failed.
0x06	PIN verification timeout.
0x07	Reserved for future use.
0x08	Reserved (PD/credential errors are reported via osdp_PIVERROR).
0x09	On-card match success.

Value	Description
0x0A–0x7F	Reserved for future use.
0x80–0xFF	Private use.

3.10 TWIC Privacy Key Load (osdp_PIVLOADTPK)

The osdp_PIVLOADTPK command (command code 0xAC, tentative) instructs the PD to cache a TWIC Privacy Key (TPK) for use with subsequent biometric operations against a TWIC credential. Function Code 14 is the biometric-support compliance byte; before using this command, controllers **SHOULD** verify that Function Code 14 advertises the OSDP 3.0 biometric command set and the relevant biometric modality bits for the intended TWIC operation.

Table 27: TWIC Privacy Key Load (osdp_PIVLOADTPK)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Command identifier.	0xAC (tentative)
<u>0x01</u>	Cache Timeout	Number of seconds to retain the cached TPK. 0x00 clears any cached key immediately; 0x01–0xFF permit caching for up to 255 <u>0xFF</u> seconds.	0x00–0xFF
<u>0x01</u>	Control Flags	Bit 0 set requests automatic clearing when the credential is removed; all other bits SHALL be zero.	0x00–0xFF
32 <u>0x20</u>	TPK	Trusted Processing Key bytes to cache for the next TWIC biometric operation.	0x00–0xFF

Controllers set *Cache Timeout* to define how long (in seconds) the PD may retain the key. A value of 0x00 clears any cached TPK immediately. Values 0x01–0xFF allow caching for up to 255 seconds. Bit 0 of *Control Flags* directs the PD to clear the cached TPK when the credential is removed; all other bits are reserved and **SHALL** be transmitted as zero. If the PD does not support TWIC TPK caching it **SHALL** return osdp_NAK function-not-supported. Invalid payloads (for example reserved bits set) **SHALL** elicit osdp_~~PERROR~~ PIVERROR 0x0005 (invalid parameter).

3.11 PIV VCI Pairing Code Transmit (osdp_PIVXMITPAIRING)

The osdp_PIVXMITPAIRING command (command code 0xAD, tentative) ~~delivers~~ submits an eight-digit pairing code to the ~~PD for use during Virtual Contact Interface (VCI) establishment. The pairing code is transmitted from the ACU to the PD only~~ selected contactless PIV application over established credential SM. The ACU SHALL send it over OSDP Secure Channel 2.0, and is used by the PD to establish VCI secure messaging with the credential. Controllers SHALL NOT issue this command unless a PIV application is currently selected and VCI mode is active. Credential SM, PIVMODE flag 0x02, and credential support for VCI are prerequisites.

Table 28: PIV VCI Pairing Code Transmit (osdp_PIVXMITPAIRING)

Size (bytes)	Name	Meaning	Value
0x01	CMND	Command identifier.	0xAD (tentative)
0x08	Pairing Code	Eight ASCII digits ('0'-'9') used by the PD during VCI establishment for the currently selected PIV application.	0x30-0x39

The pairing code ~~is encoded as~~ contains eight ASCII digits ('0'-'9'). ~~If no PIV application is selected~~ The PD accepts a valid request with osdp_ACK. When pairing is required, the PD ~~SHALL return~~ submits the code using PIV ~~osdp_PDERROVERIFY~~. Otherwise, the PD ~~SHALL~~ cache the pairing code only as an input to VCI establishment for the currently, reference 0x98, protected by the established SM context. Success completes with osdp_CARDSTATUSR on a later osdp_POLL, reporting security status 0x01 (SM and VCI). If VCI is already available, the PD completes with the same status reply without submitting the code again.

An accepted operation completes with osdp_PIVERROR on failure: 0x0005 (invalid parameter) for malformed digits or no selected PIV application and use it when required by the credential's Discovery Object PIN Usage Policy (for example when bit 4 is set and bit 3 is clear in the first byte). The pairing code is not evidence that VCI secure messaging is already established, and osdp_PIVXMITPAIRING is not transported over VCI secure messaging. When VCI secure messaging is already active or the credential does not require pairing to establish VCI, 0x1023 (security conditions not satisfied) for missing SM, disabled SM/VCI preparation, the wrong interface, or a rejected pairing code, 0x1020 (function not supported by credential) when VCI is unsupported, 0x1026 (credential absent or removed), or 0x0002 (smart-card timeout). The PD includes the credential's failure status word as specified in Subsection 2.2.1. If a submitted pairing VERIFY times out without a confirmed result, the PD ~~MAYSHALL~~ ignore the supplied code after acknowledging receipt. ~~PDs end SM, clear dependent VCI, PIN-verification, and cached-PIN state, and require an explicit preparation restart before further credential data or authentication commands. The PD SHALL clear any stored pairing code when VCI is established, when VCI is torn down, return 0x0002 (smart-card timeout) without SW1/SW2 and SHALL NOT retry VERIFY automatically. While pairing is pending, the ACU sends only osdp_POLL or when the active PIV mode changes: osdp_ABORT. Competing commands receive osdp_NAK 0x09 (unable to process command).~~

The PD ~~SHALL~~ clear the transient pairing code on completion, failure, cancellation, or loss of its credential context. osdp_ABORT completes with osdp_ACK and clears the pending completion. For an OSDP retransmission, the PD ~~SHALL~~ replay the previous reply without resubmitting the pairing code.

~~VCI-OSDP security preconditions~~ VCI-trust-anchor Trust-anchor load/status, VCI-pairing-code transmit, VCI-credential SM establishment, and PIV operations that depend on SM or VCI ~~SHALL be accepted only when require~~ OSDP Secure Channel 2.0 is active between the ACU and PD. ~~This with deployment-specific keys.~~ OSDP Secure Channel protects ACU-to-PD management traffic, including osdp_PIVXMITPAIRING; VCI secure messaging is the separate PD-to-credential secure channel established by the PD. A PD ~~SHALL~~ reject VCI-related functionality when operating with factory/default SCBK or other factory keys. Security-precondition failures ~~SHALL~~ be reported using the management commands and their results.

Credential SM protects the PD-to-card exchange. A security-precondition failure returns osdp_PDERROR PIVERROR 0x0006 where the command supports osdp_PDERROR (secure channel required) where supported; otherwise the command's existing failure reporting path reply applies.

3.12 Set PIV Mode (osdp_PIVMODE)

The osdp_PIVMODE command (command code 0xAE, tentative) defines the operating context for PIV, TWIC, and related credential transactions. OSDP 3.0 replaces the OSDP 2.2 “Application + AID” fields with a configuration payload that includes a required Global Interface Mask byte, PIV Auto challenge parameters when autonomous operation is enabled, and a TLV list of configuration descriptors so controllers can specify multiple application profiles, preferred interfaces, and optional Application Identifiers. PDs **SHALL** advertise support via Function Code 12 bit 0x04 (Enhanced PIV).

Table 29: Set PIV Mode (osdp_PIVMODE)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Command identifier.	0xAE (tentative)
<u>0x02</u>	MpSizeTotal	Total size of the reassembled configuration data beginning with PIV Mode Flags, least-significant byte first.	0x0008–0xFFFF
<u>0x02</u>	MpOffset	Offset of this fragment within the reassembled configuration data, least-significant byte first.	0x0000–0xFFFF
<u>0x02</u>	MpFragmentSize	Number of configuration data bytes following the multi-part fields in this fragment, least-significant byte first.	0x0000–0xFFFF
<u>0x01</u>	PIV Mode Flags	First byte of the reassembled configuration data; global behaviour flags (Section 3.12).	0x00–0xFF
<u>0x01</u>	Config Count	Number of configuration entries (1–32) .	0x01–0x20
<u>0x01</u>	Global Interface Mask	Bitmask limiting interfaces available to all configurations (bit 0 contact, bit 1 contactless, bit 3 Bluetooth; bit 2 reserved and set to zero).	0x00–0x0B
<u>0x04</u>	Series Counter	Conditional field present only when PIV Auto is enabled. Least-significant byte first.	0x00000000–0xFFFFFFFF
<u>0x04</u>	Sequence Counter	Conditional field present only when PIV Auto is enabled. Least-significant byte first.	0x00000000–0xFFFFFFFF

Size (bytes)	Name	Meaning	Value
32 <u>0x20</u>	Supplemental Entropy	Conditional fixed-width field present only when PIV Auto is enabled. All zeroes are permitted but not recommended.	32 <u>0x20</u> octets
<u>0x00</u> -n	TLV Data	Sequence of Configuration Entry TLVs (Tag 0x01) that follow the Global Interface Mask, or follow the conditional PIV Auto fields when PIV Auto is enabled.	-

The *PIV Mode Flags* byte enables ~~global behaviours such as~~ autonomous operation and ~~VCI usage~~ credential SM/VCI preparation (Table 30). Bits not listed remain reserved and **SHALL** be transmitted as zero.

Table 30: PIV Mode Flags

Mask	Description
0x01	Enable autonomous PIV (PIV Auto). The PD interrogates credentials, establishes VCI when possible, prepares the credential , derives a KMAC256 challenge from the configured counters and entropy, and reports the signed response <u>authentication result</u> using osdp_PIVSTATUS.
0x02	Permit VCI usage when available. Enable credential Secure Messaging preparation over contact and contactless interfaces. On contactless, also enable VCI when the credential permits VCI without pairing. Applies to autonomous and commanded operation.
0x04	Enforce FICAM strict mode (PIV applications only).

Global and per-entry interface masks use the bit positions in Table 31; the Global Interface Mask bounds the interfaces a PD may use when processing all configuration entries.

Table 31: PIV Mode Interface Masks

Mask	Description
0x01	Permit ISO 7816 contact credential interface use.
0x02	Permit ISO 14443 contactless credential interface use.
0x04	Reserved and SHALL be transmitted as zero.
0x08	Permit Bluetooth credential interface use.
0x10–0x80	Reserved and SHALL be transmitted as zero.

The reassembled configuration data begins at *MpOffset* 0x0000 with *PIV Mode Flags*, followed by the configuration count (1-32) and the required Global Interface Mask byte. *MpSizeTotal* counts all configuration data from *PIV Mode Flags* through the final TLV byte. If and only if PIV Auto is enabled in *PIV Mode Flags*, the

Global Interface Mask is followed by the Series Counter, Sequence Counter, and fixed 32-byte Supplemental Entropy field before the TLV list. When PIV Auto is not enabled, those three conditional fields are absent and the TLV list begins immediately after the Global Interface Mask. Each configuration appears as a TLV:

The configuration count **SHALL** equal the number of complete Configuration Entry TLVs in the reassembled payload. Consequently, *MpSizeTotal* **SHALL** be at least eight bytes when PIV Auto is disabled and at least 48 bytes when it is enabled, and **SHALL** equal the exact encoded length of the required header, any conditional PIV Auto fields, and all configuration TLVs. Truncated fields, trailing bytes, or a count that does not match the TLV sequence **SHALL** be treated as malformed configuration data.

- **Global Interface Mask (required)** — One-byte mask (no tag) that limits the interfaces the PD may use for all configurations (see Table 31). A value of 0x00 indicates no global restriction; reserved bits **SHALL** be zero.
- **Configuration Entry (Tag 0x01)** — Contains the data-model identifier, a per-entry interface mask, the AID length, and the optional AID:
 - *Data Model* (0x01 PIV, 0x02 TWIC, 0x03 CAC, values 0x04-0x7F reserved, 0x80-0xFF private use). CAC entries use the same interface and AID encoding as PIV and TWIC entries.
 - *Interface Mask* — Uses the same bit positions as the global mask (Table 31) and **SHALL NOT** assert interfaces excluded by the Global Interface Mask. A value of 0x00 means “any interface permitted by the global mask.”
 - *AID Length* (0-16) and *AID* bytes when provided.

PDs process configuration entries in order, applying the global mask (when present) before honouring each entry’s mask. Invalid masks, unknown data-model identifiers, or malformed TLVs **SHALL** result in `osdp_PDERRORPIVERROR 0x0005 (invalid parameter)`. Controllers **SHALL** send the command over a secure channel where required; the PD **SHALL** return `osdp_PDERRORPIVERROR 0x0006 (secure channel required)` if the necessary security conditions are not satisfied. Multi-part framing errors result in `osdp_PDERRORPIVERROR 0x0007 (invalid multi-part transfer)`. If the PD cannot cache the requested configuration set, it **SHALL** respond with `osdp_PDERRORPIVERROR 0x1021 (insufficient PD memory or storage)`.

~~When PIV Auto is enabled, PDs interrogate the credential on presentation, establish VCI automatically when possible, derive the PIV Auto challenge, issue the PIV GENERAL AUTHENTICATE command, and report the signed response without additional ACU prompts. If the PD prepares the presented credential as described in Subsection 3.1, derives the challenge, issues PIV GENERAL AUTHENTICATE, and reports completion through osdp_PIVSTATUS. The PD SHALL use the configured key, algorithm, and selected interface for that attempt. If SM is unavailable, the PD continues that same authentication only when the credential's access rules permit the authentication without SM, as for key 0x9E. If the configured authentication requires SM, the PD reports the SM failure. A failed signature or cryptogram check ends the attempt even when the authentication could otherwise proceed without SM.~~

~~PIV Auto can establish VCI when the credential permits VCI without a pairing codeis required. For an authentication that requires SM, the PD signals the state change via a solicited osdp_CARDSTATUSR~~

~~(not via unsolicited poll); the ACU then provides the code using osdp_PIVXMITPAIRING over OSDP Secure Channel 2.0, and the PD uses the code to complete VCI establishment. first checks whether SM is established. If SM is unavailable, the PD reports Result 0x02 (SM failure) with the applicable detail code. The PD then checks any VCI and PIN-verification requirements, in that order. If the selected authentication requires VCI and pairing is required, the PD completes with Result 0x05, Detail 0x01 (pairing code required). The ACU can then begin commanded pairing. Required PIN verification that has not been completed returns Result 0x05, Detail 0x02 (PIN verification required). PIV Auto performs one attempt per presentation. Keeping the same credential presented does not restart the attempt. Removal and a new presentation, or successful reapplication of osdp_PIVMODE, starts a new attempt. Reapplication follows the mode-change clearing rules; loading a trust anchor alone does not restart authentication.~~

PIV Auto configuration **SHALL** be accepted only when OSDP Secure Channel 2.0 is active and the channel is not using a factory/default SCBK or other factory key material. A PD **SHALL** reject PIV Auto configuration or execution with osdp_~~PDERROR~~~~PIVERROR~~ 0x0006 (secure channel required) when these preconditions are not met.

When bit 0x01 of *PIV Mode Flags* enables PIV Auto, the configuration **SHALL** specify a *Series Counter*, a *Sequence Counter*, and a 32-byte *Supplemental Entropy* field. The supplemental entropy field is fixed width and **SHALL** always be present; all zeroes are permitted for compatibility but are not recommended. The ACU **SHALL** ensure that a series counter value does not repeat for the same PD and that no sequence counter value is issued more than once within a series. One conforming implementation strategy is to maintain a monotonic series counter that increments at every ACU boot, similar to the SNMP engine boots counter, and a monotonic sequence counter for challenges issued within that boot series.

The PIV Auto challenge **SHALL** be derived from the active Secure Channel 2.0 session key material using two KMAC256 operations. First, the PD derives a 32-byte PIV Auto KDK with key material formed by concatenating S-ENC, S-MAC1, and S-MAC2; empty input data; output length 32 bytes; and customization string OSDP-PIV-AUTO-KDK-v1. Second, the PD derives the challenge digest using the PIV Auto KDK as the KMAC key, the fixed-width template as input data, output length L, and customization string OSDP-PIV-AUTO-CHALLENGE-v1. The fixed-width template **SHALL** contain the series counter, sequence counter, credential FASC-N, credential UUID, challenged PIV key reference, PIV algorithm identifier, and the 32-byte supplemental entropy field. A successful PIV Auto transaction **SHALL** include the credential UUID; if the UUID cannot be read, the PD **SHALL** report PIV Auto failure instead of deriving a success challenge with an all-zero UUID. The output length L is 32 bytes for SHA-256 profiles and 48 bytes for SHA-384 profiles. PIV Auto challenge derivation **SHALL NOT** use static factory keys.

3.13 PIV Auto Status Reply (osdp_PIVSTATUS)

~~After When~~ a PIV Auto ~~card challenge completes, attempt completes, including a failure during preparation,~~ the PD reports the result as a poll response using reply code 0x89 (tentative). The first osdp_POLL response after completion **SHALL** contain osdp_PIVSTATUS; subsequent fragments, when required, **SHALL** use the same reply and advance *MpOffset* until the complete status payload is delivered. ~~This is an autonomous completion reply; there is~~ There is no corresponding osdp_PIVSTATUS request command.

Table 32: PIV Auto Status Reply (osdp_PIVSTATUS)

Size (bytes)	Name	Meaning	Value
1 <u>0x01</u>	CMND	Reply identifier.	0x89 (tentative)
<u>0x02</u>	MpSizeTotal	Total size of the complete PIV Auto status payload, least-significant byte first.	0x0000-0xFFFF
<u>0x02</u>	MpOffset	Offset of this fragment within the complete status payload, least-significant byte first.	0x0000-0xFFFF
<u>0x02</u>	MpFragmentSize	Length of the status payload bytes in this fragment, least-significant byte first.	0x0000-0xFFFF
<u>0x01</u>	Result	PIV-Auto result for the challenged credential <u>Result of the PIV Auto authentication attempt.</u> See Table 33.	0x00-0xFF
<u>0x01</u>	Detail	Additional result detail. Reserved as Result-specific detail from Table 33. Set to 0x00 when Result is ; may carry SW1 SW2 low byte when Result is where no detail is defined.	0x00-0xFF
<u>0x04</u>	Series Counter	Series counter value used to derive the challenge, least-significant byte first.	0x00000000-0xFFFFFFFF
<u>0x04</u>	Sequence Counter	Sequence counter value used to derive the challenge, least-significant byte first.	0x00000000-0xFFFFFFFF
25 <u>0x19</u>	FASC-N	FASC-N read from the credential.	200-bit FASC-N
16 <u>0x10</u>	UUID	Credential UUID read from the credential. Success replies SHALL carry a nonzero UUID; failure replies may transmit all zeroes when the UUID is unavailable.	16 <u>0x10</u> octets
<u>0x01</u>	Key Reference	PIV key reference challenged by the PD.	For example 0x9E
<u>0x01</u>	Algorithm Identifier	PIV GENERAL AUTHENTICATE algorithm identifier.	For example 0x07
<u>0x02</u>	Signed Response Length	Total length of the signed response, least-significant byte first.	0x0000-0xFFFF

Size (bytes)	Name	Meaning	Value
0x00-n	Signed Response Fragment	Contiguous bytes of the signed response carried in this fragment Complete response bytes returned by the credential. This field is part of the reassembled status payload.	Variable
0x00 or 0x02	Status Word	SW1 followed by SW2 when the credential returned the reported failure. Omitted in the other cases specified below. Part of the reassembled payload.	- ~

Result and detail codes follow Table 33. When *Result* is 0x00, *Detail* is reserved and **SHALL** be 0x00. When *Result* is not 0x00, the PD **SHALL** set *Signed Response Length* to 0x0000 unless a credential response was received and is useful for ACU diagnostics.

The PD **SHALL** append SW1 followed by SW2 when a credential command returned the failure being reported. For a verified protected reply, the PD uses the protected status. The PD **SHALL** omit this field on success, local failure, timeout, credential removal without a failure reply, or failure to verify a protected reply. An earlier command's status **SHALL NOT** be used. These are the status-word rules of Subsection 2.2.1. The status word follows Signed Response in the reassembled payload and may cross a fragment boundary. After accounting for the fixed fields from Result through Signed Response Length in Table 32 (0x37 bytes) and Signed Response Length, the ACU **SHALL** accept exactly zero or two remaining bytes; any other length is malformed. Result 0x06 uses Detail 0x00; the complete status word carries the credential's error. The PD returns one terminal PIVSTATUS, with no follow-up PIVERROR for that attempt.

Table 33: PIV Auto Status Result Codes

Value	Description
0x00	Success. <i>Detail</i> is 0x00, UUID is nonzero, and Signed Response carries the complete returned Dynamic Authentication Template.
0x01	Credential not present or removed before PIV Auto completed.
0x02	VCI establishment or required VCI validation failed. <u>Credential SM unavailable or failed. Detail: 0x01 trust store empty; 0x02 no matching trust path; 0x03 missing or invalid CVC; 0x04 invalid authentication cryptogram; 0x05 required SM unsupported; 0x06 other SM processing failure; 0x07 required SM disabled. Detail assignments are tentative.</u>
0x03	Required credential data object, FASC-N, UUID, key, or certificate evidence is unavailable.
0x04	Requested PIV key reference or algorithm is not supported for PIV Auto.

Value	Description
0x05	Credential security status is not satisfied, including missing PIN or pairing preconditions. <u>Required credential security condition is absent. Detail: 0x01 pairing code required; 0x02 PIN verification required; 0x03 required VCI unsupported; 0x00 other unmet condition. Detail assignments are tentative.</u>
0x06	Credential GENERAL AUTHENTICATE operation failed; Detail may carry SW1 SW2 when available. <u>Detail is 0x00; the Status Word field carries the credential failure status when available under the rules below.</u>
0x07	Signed response verification failed.
0x08	Timeout or PD internal failure before a definitive credential result was available.
0x09–0x7F	Reserved for future standard result codes.
0x80–0xFF	Private use.

For Result 0x02, Details 0x03, 0x04, and 0x06 block further credential operations until the context is restarted, as specified in Subsection 3.1. Details 0x01, 0x02, 0x05, and 0x07 leave operations permitted without SM available.

For failure before challenge generation, the PD SHALL report the configured series and sequence counters and requested key reference and algorithm. It SHALL set unavailable FASC-N and UUID fields to zero and Signed Response Length to zero. These identity fields describe data actually obtained from the credential. The PD SHALL set Detail to zero unless the result defines a detail value.

The reply reports the exact series and sequence counters used for the challenge, the FASC-N and UUID read from the credential, and the signed response returned by the credential. The ACU uses these fields to verify counter uniqueness, bind the response to the credential identity, and validate the signature and certificate path using ACU trust policy. A success result **SHALL** include a nonzero credential UUID. The PD **SHALL** preserve the credential's signed response bytes as returned by the PIV operation, excluding transport status words such as SW1/SW2.

If the complete signed response does not fit in one OSDP response, the PD **SHALL** fragment `osdp_PIVSTATUS` using the OSDP [2.2–3.0](#) multi-part convention. *MpSizeTotal* is the total status payload size, *MpOffset* is the zero-based offset of the fragment, and *MpFragmentSize* is the number of payload bytes carried in that fragment. The logical status payload begins at offset zero with the fixed fields; each fragment carries the next contiguous bytes of that payload. A fragment boundary may occur within the fixed fields or within the signed response.

3.14 VCI Trust Anchor Load (`osdp_PIVVCILOADTA`)

The `osdp_PIVVCILOADTA` command (command code 0xAF, tentative) loads, replaces, or deletes a **Virtual Contact Interface (VCI) credential SM** trust anchor. The existing command name and record format apply to both contact and contactless SM. The ACU **SHALL** validate and authorize each ~~trust~~ anchor under its deployment certificate policy before loading it. ~~Trust anchors must be cached before a PD can verify card certificates during VCI establishment~~The PD uses the loaded public key to validate the credential's SM CVC.

Table 34: VCI Trust Anchor Load (osdp_PIVVCILOADTA)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Command identifier.	0xAF (tentative)
<u>0x02</u>	MpSizeTotal	Total size of the reassembled operation data, including the 12-byte <u>0x0C-byte</u> metadata block and Anchor Data, least-significant byte first.	0x000C–0x024C
<u>0x02</u>	MpOffset	Offset of this fragment within the reassembled operation data beginning with Operation, least-significant byte first.	0x0000–0xFFFF
<u>0x02</u>	MpFragmentSize	Number of operation data bytes following the multi-part fields in this fragment, least-significant byte first.	0x0000–0xFFFF
<u>0x01</u>	Operation	0x00 load/replace anchor; 0x01 delete anchor. Anchor inventory is requested with osdp_PIVVCITASTATUS.	0x00–0x01
<u>0x01</u>	Trust Anchor ID	Controller-assigned identifier for the anchor(1-128).	0x01–0x80
<u>0x08</u>	Issuer Identifier Number	For load/replace, the leftmost eight bytes of the subjectKeyIdentifier from the content-signing X.509 certificate and a match for tag 0x42 in the trust anchor record. For delete, this value SHALL match the stored anchor's IIN.	0x00–0xFF
<u>0x02</u>	Anchor Length	Total size of the variable-length VCI trust anchor record, least-significant byte first. Set to 0x0000 when deleting.	0x0000–0x0240
<u>0x00-n</u>	Anchor Data	Fragment bytes of the OSDP-defined VCI trust anchor record. Omitted for delete operations.	0x00–0xFF

The reassembled operation data begins at *MpOffset* 0x0000 with a 12-byte metadata block containing *Operation*, *Trust Anchor ID*, *Issuer Identifier Number*, and *Anchor Length*, followed immediately by *Anchor Data*. These fields may cross fragment boundaries and are interpreted only after the complete logical data has been reassembled. Controllers set *Operation* to 0x00 to load or replace the anchor identified by *Trust Anchor ID*. Set *Operation* to 0x01 with *Anchor Length* 0x0000 and *MpSizeTotal* 0x000C to delete the anchor. For a

delete operation, the supplied IIN **SHALL** match the IIN of the stored anchor selected by *Trust Anchor ID*; a mismatch **SHALL** be rejected with `osdp_PDERRORPIVERROR 0x0005` ([invalid parameter](#)). Inventory is requested with `osdp_PIVVCITASTATUS`, not with `osdp_PIVVCILOADTA`. The *Issuer Identifier Number* (IIN) is an OSDP-defined anchor selector: it **SHALL** be the leftmost eight bytes of the *subjectKeyIdentifier* from the content-signing X.509 certificate associated with this anchor. For load/replace, this value **SHALL** match the IIN field (tag 0x42) in the VCI trust anchor record.

The *Anchor Length* declares the exact size of the variable-length VCI trust anchor record. For every operation, *MpSizeTotal* **SHALL** equal 0x000C plus *Anchor Length*. For an operation accepted for processing, the sum of all accepted *MpFragmentSize* values **SHALL** equal *MpSizeTotal*, and no fragment **SHALL** extend beyond *MpSizeTotal*. For load operations, the reassembled bytes following the metadata block **SHALL** contain exactly *Anchor Length* bytes. A conforming VCI trust anchor record **SHALL NOT** exceed 576 bytes (0x0240), making the maximum *MpSizeTotal* 0x024C. The PD **SHALL** accept any valid record length from the minimum BER-TLV encoding through 0x0240, subject to supported-key and storage-capacity checks.

PD storage accounting for VCI trust anchors is variable-length. A PD **SHALL NOT** treat every anchor as consuming the maximum 0x0240 bytes unless that is the PD's documented physical allocation model. When reporting capacity, loading, replacing, or deleting anchors, the PD **SHALL** account for stored anchors by their actual *Anchor Length* values and **SHALL** use the same accounting rule for both `osdp_PIVVCITASTATUSR Available Bytes` and `osdp_PIVVCILOADTA` storage acceptance. A replacement operation **SHALL** be evaluated after releasing the storage occupied by the existing anchor with the same *Trust Anchor ID*; if the replacement cannot be completed, the prior anchor **SHALL** remain available. A delete operation **SHALL** release the storage associated with the deleted anchor.

Table 35: VCI Trust Anchor Record Format

Tag	Length	Name	Value
0x7F50	Variable	VCI Trust Anchor Record	OSDP-defined record used to verify a secure messaging CVC or an Intermediate CVC.
0x80	0x01	Record Profile Identifier	0x01.
0x42	0x08	Issuer Identifier Number	Anchor selector. This value SHALL match the IIN in the <code>osdp_PIVVCILOADTA</code> metadata.
0x7F49	1-550- 0x01-0x0226	SubjectPublicKeyInfo	DER-encoded X.509 SubjectPublicKeyInfo for the public key trusted by the ACU. Supported public-key algorithms are RSA public keys up to 4096 bits and EC public keys for P-256 or P-384.

The VCI trust anchor record is intentionally not a NIST SP 800-73 Intermediate CVC and does not carry X.509 path-validation state. The ACU is responsible for selecting, loading, unloading, and validating the VCI anchors it permits a PD to use before the PD relies on them. The PD uses the record only as a local public-key verifier selected by IIN. The maximum record size is derived from the largest supported key case: a 550-byte RSA-4096 SubjectPublicKeyInfo plus the fixed fields above and BER-TLV length overhead. The resulting worst case is 573 bytes; this specification rounds the maximum accepted record size to 576 bytes.

Use during ~~VCI~~-credential SM establishment When the PD receives the secure messaging CVC from the credential, it **SHALL** first extract the Issuer Identification Number from tag 0x42. If a loaded VCI trust anchor has a matching IIN, the PD **SHALL** use that anchor's SubjectPublicKeyInfo to verify the secure messaging CVC signature. If verification succeeds, the PD may continue ~~VCI~~-credential SM establishment using the public key in the secure messaging CVC.

If no loaded VCI trust anchor matches the secure messaging CVC IIN, the PD **SHALL** retrieve the Secure Messaging Certificate Signer data object (~~PIV-data-object~~, tag-5FC122object identifier 0x5FC122, container identifier 0x1017) from the credential and inspect it for one Intermediate CVC (tag 0x7F21). If the object is absent or no Intermediate CVC is present, validation fails. If an Intermediate CVC is present, the PD **SHALL** verify all of the following before continuing:

- The secure messaging CVC IIN (tag 0x42) equals the Subject Identifier (tag 0x5F20) in the Intermediate CVC.
- The Intermediate CVC Role Identifier (tag 0x5F4C) is 0x12.
- A loaded VCI trust anchor exists whose IIN equals the Intermediate CVC IIN (tag 0x42).
- The Intermediate CVC signature verifies using the matching VCI trust anchor public key.
- The secure messaging CVC signature verifies using the public key carried in the Intermediate CVC.

The PD validates only the CVC chain needed for ~~VCI~~-credential SM establishment: either the loaded trust anchor directly verifies the secure messaging CVC, or the loaded trust anchor verifies the Intermediate CVC that then verifies the secure messaging CVC. The PD **SHALL NOT** establish ~~VCI~~-credential SM unless this CVC validation succeeds. The PD **SHALL NOT** perform full X.509 path validation, validity-period evaluation, certificate-policy evaluation, or revocation processing for VCI trust anchors; those checks are completed by the ACU before loading anchors and again as an acceptance policy after ~~VCI~~-credential SM state is reported.

An absent signer object, an absent intermediate CVC, or the absence of a matching loaded anchor leaves the PD without a matching trust path. The PD reports SM Detail 0x02 for that condition. A malformed CVC, incorrect role or linkage, or failed signature check reports Detail 0x03 and blocks credential operations. These distinctions apply to both PIV Auto results and SM diagnostics.

ACU ~~post-VCI-validation~~acceptance of credential SM After receiving osdp_CARDSTATUSR with ~~VCI~~-credential SM established, the ACU **SHALL** validate the content-signing certificate corresponding to the reported ~~VCI~~-SM Anchor IIN before treating the ~~VCI~~-credential SM session as accepted for the deployment. This validation is an additional ACU acceptance check and does not replace the PD's required CVC validation during ~~VCI~~-credential SM establishment. The ACU **SHALL** perform all of the following:

- Obtain the content-signing X.509 certificate. If the ACU does not already possess a cached copy whose subjectKeyIdentifier leftmost-eight-bytes equals the ~~VCI~~-SM Anchor IIN, the ACU **SHALL** use osdp_PIVGETDATA to retrieve the Secure Messaging Certificate Signer data object (~~PIV-data-object~~, tag-5FC122object identifier 0x5FC122, container identifier 0x1017) and extract the certificate from inner tag 0x70.

- Verify that the leftmost eight bytes of the certificate's subjectKeyIdentifier equal the ~~VCI~~SM Anchor IIN reported in osdp_CARDSTATUSR.
- Verify that the certificate's SubjectPublicKeyInfo matches the SubjectPublicKeyInfo stored in the loaded VCI trust anchor record for that IIN.
- Verify that the certificate includes the PIV content-signing extended key usage OID (2.16.840.1.101.3.6.7) or the PIV-I equivalent (2.16.840.1.101.3.8.7).
- Verify that the current time falls within the certificate's validity period ($\text{notBefore} \leq \text{now} \leq \text{notAfter}$). If the certificate has expired or is not yet valid, the ACU **SHALL** reject the ~~VCI~~credentialSM session and **SHALL** unload the corresponding trust anchor.
- Perform X.509 path validation and revocation checking in accordance with the deployment's certificate validation policy.

Table 36: VCI Trust Anchor Examples from NIST Vectors

Vector	Content Signer	Anchor IIN	SPKI	Record
card_2, card_5	Test PIV Content Signer 4 (EC P-384, direct)	B6103792270FBD08	120 <u>0x78</u> bytes	140 <u>0x8C</u> bytes
card_3	Test PIV Content Signer 1 (RSA-2048, intermediate)	9BC5E7735F14EC7F	294 <u>0x126</u> bytes	317 <u>0x13D</u> bytes
card_4	Test PIV Content Signer 3 (EC P-256, direct)	1905976BDDCA823D	91 <u>0x5B</u> bytes	110 <u>0x6E</u> bytes
card_16	Test PIV-I Content Signer 1 (RSA-2048, intermediate)	4CC9C7B3A4F1C83E	294 <u>0x126</u> bytes	317 <u>0x13D</u> bytes

In the card_16 intermediate example, the secure messaging CVC IIN is 6F8FF48F42B3E22E; the Intermediate CVC Subject Identifier (tag 0x5F20) is also 6F8FF48F42B3E22E; and the Intermediate CVC IIN (tag 0x42) is 4CC9C7B3A4F1C83E, matching the loaded RSA trust anchor.

PDs **SHALL** reassemble the complete operation data before processing it. For a load or replacement, the PD **SHALL** verify that the reassembled *Anchor Data* portion is exactly *Anchor Length* bytes and is a single 0x7F50 record before committing it to storage. A successful operation returns osdp_ACK. Failures return:

- osdp_NAK function-not-supported when the PD does not support trust-anchor caching.
- osdp_~~PERROR~~PIVERROR 0x0005 (invalid parameter) for malformed metadata (unknown operation, reserved bits, invalid lengths, MpSizeTotal not equal to 0x000C plus *Anchor Length*, or a reassembled Anchor Data length not equal to *Anchor Length*).
- osdp_~~PERROR~~PIVERROR 0x0007 (invalid multi-part transfer) when multi-part sequencing fails.

- osdp_~~PDERROR~~PIVERROR 0x1021 (insufficient PD memory or storage) when the PD lacks storage space for the anchor.

PDs are not required to maintain a real-time clock. PDs may retain either the supplied VCI trust anchor record or an equivalent internal representation so long as the public key and supplied IIN remain available for the verification flow above.

Trust anchors may be stored in volatile or non-volatile memory at the PD's discretion. Controllers should not assume retention beyond the device's documented behaviour.

Replacing or deleting the anchor used by an active SM session **SHALL** invalidate that session, its VCI state, and PIN-verified status. The PD **SHALL** clear security-dependent cached material and trust evidence for that session. A failed replacement preserves the existing anchor and session. Changes to unrelated anchors leave the active session unchanged. Trust-store updates take effect in the next explicitly started preparation attempt.

3.15 VCI Trust Anchor Status (osdp_PIVVCITASTATUS)

The osdp_PIVVCITASTATUS command (command code 0xA6, tentative) enumerates cached trust anchors and reports remaining storage capacity. The request carries no payload. The reply uses the standard multi-part fields (MpSizeTotal, MpOffset, MpFragmentSize) so PDs can stream the inventory across one or more fragments using reply code 0x88 (tentative).

Table 37: VCI Trust Anchor Status Request (osdp_PIVVCITASTATUS)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Command identifier.	0xA6 (tentative)

Table 38: VCI Trust Anchor Status Reply (osdp_PIVVCITASTATUSR)

Size (bytes)	Name	Meaning	Value
<u>0x01</u>	CMND	Reply identifier.	0x88 (tentative)
<u>0x02</u>	MpSizeTotal	Length of the complete status payload, least-significant byte first.	0x0000-0xFFFF
<u>0x02</u>	MpOffset	Offset of this fragment within the status payload, least-significant byte first.	0x0000-0xFFFF
<u>0x02</u>	MpFragmentSize	Length of the data block in this fragment, least-significant byte first.	0x0000-0xFFFF
<u>0x01</u>	Total Anchor Count	Number of cached trust anchors.	0x00-0xFF

Size (bytes)	Name	Meaning	Value
<u>0x02</u>	Available Bytes	Remaining storage capacity for additional variable-length trust anchor records, least-significant byte first.	0x0000–0xFFFF
<u>0x00-n</u>	Anchor Records	Zero or more 12-byte <u>0x0C-byte</u> inventory records. Each record contains: TrustAnchorID (<u>0x01</u> byte for load/delete management), anchor selector IIN (<u>0x08</u> bytes from the stored trust anchor record, not necessarily the secure CVC IIN), AnchorLength (<u>0x02</u> bytes LSB first, maximum 0x0240), and Attributes (<u>0x01</u> byte, reserved). Records that span fragments continue in subsequent replies.	–

The PD first reports the total number of anchors and the remaining free bytes. *Available Bytes* is the amount of trust-anchor storage that remains for additional variable-length VCI trust anchor record data after accounting for currently stored anchors by their actual *Anchor Length* values. Each record thereafter lists a trust anchor by management ID, anchor selector IIN, anchor length, and reserved attributes (transmitted as zero). The IIN in this inventory is the IIN from the stored VCI trust anchor record and is not necessarily the secure messaging CVC IIN reported in `osdp_CARDSTATUSR`. Error responses follow the standard pattern: `osdp_NAK` function-not-supported when the command is not implemented, and `osdp_PDError` 0x0007 (invalid multi-part transfer) for multi-part errors.

Table 39: Trust Anchor Record Structure

Field	Size	Meaning	Value Range
Trust Anchor ID	<u>0x01</u> byte	Identifier assigned by the ACU; reused to replace or delete an anchor.	0x00–0xFF <u>0x01–0x80</u>
Issuer Identifier Number (IIN)	<u>0x08</u> bytes	Anchor selector supplied in the VCI trust anchor record.	0x00–0xFF
Anchor Length	<u>0x02</u> bytes (LSB first)	Total size of the stored variable-length VCI trust anchor record supplied for this anchor.	0x0000–0x0240
Attributes	<u>0x01</u> byte	Reserved for future use; transmitted as zero.	0x00

A References

A.1 Normative References

- Security Industry Association, *Open Supervised Device Protocol (OSDP) v2.2*, 2020.
- National Institute of Standards and Technology, [NIST Special Publication 800-73-5, Interfaces for Personal Identity Verification \(PIV\) — Part 1: PIV Card Application Namespace, Data Model, and Representation, 2024.](#)
- [National Institute of Standards and Technology](#), *NIST Special Publication 800-73-4, Interfaces for Personal Identity Verification (PIV) — Part 2: PIV Card Application Card Command Interface*, February 2015.
- National Institute of Standards and Technology, *NIST Special Publication 800-73-5, Interfaces for Personal Identity Verification (PIV) — Part 2: PIV Card Application Card Command Interface*, 2024.
- National Institute of Standards and Technology, *NIST Special Publication 800-78-5, Cryptographic Algorithms and Key Sizes for Personal Identity Verification*, 2024.
- National Institute of Standards and Technology, *Federal Information Processing Standards Publication 201-3, Personal Identity Verification (PIV) of Federal Employees and Contractors*, January 2022.
- National Institute of Standards and Technology, *NIST Special Publication 800-185, SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*, December 2016.
- International Organization for Standardization and International Electrotechnical Commission, *ISO/IEC 7816-4, Identification cards — Integrated circuit cards — Part 4: Organization, security and commands for interchange*.
- Internet Engineering Task Force, *RFC 5280, Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, May 2008.
- Internet Engineering Task Force, *RFC 2119, Key words for use in RFCs to Indicate Requirement Levels*, March 1997; and *RFC 8174, Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, May 2017.

A.2 Informative References

- National Institute of Standards and Technology, *NIST Internal Report 8347, Personal Identity Verification Cards, Version 2*, 2021.
- U.S. General Services Administration, *Federal Identity, Credential, and Access Management (FICAM) Architecture*, Revision 3, 2023.
- Open Physical, *High Speed Autonomous PIV: Concept Paper*, 2024.

B ~~Sample osdp_PIVDATA Requests~~ PIVGETDATA Worked Examples (Informative)

~~This appendix illustrates representative requests that conforming ACUs may issue using the clarified osdp_PIVDATA command. Object identifiers align with NIST SP 800-73-4 Part 2, Table 6. Each example shows the bytes that appear in the OSDP payload after the~~ These examples show the enhanced request defined in Section 2.1.3, the OSDP 3.0 multi-part response defined in Section 2.1.4, and the error reply defined in Section 2.2.1.

The ACU and PD are simulated. Credential data comes from the identified existing card captures; constructed error cases are identified separately. These examples omit OSDP Secure Channel for clarity. CRC detects transmission errors. The exchanges provide neither authentication nor confidentiality. Some source captures used credential-side secure messaging; the examples use the data recovered after that processing.

The files under `worked-examples/pivdata/` contain the complete frame exchanges, expected results, source hashes, and validation instructions. Each example ID also appears in the data file, so the printed example can be matched to its complete exchange. Long responses are shortened in this appendix only.

B.1 Reading the exchanges

Field values use hexadecimal with a ~~CMNDfield~~; the 0x prefix. Packet dumps use space-separated hexadecimal byte pairs under *Packet bytes (hex)*. Two-byte field values are shown as numbers in the request table and in least-significant-byte-first order in the packet bytes.

An accepted PIVGETDATA command receives ACK. The ACU then sends POLL until the PD returns a data fragment or PIVERROR. The first data fragment has offset zero, even when Offset in the command is nonzero. Further polls retrieve the rest of the data in order, until the ACU has received the advertised total. Polls during processing receive ACK. The examples use a maximum frame size of 0x0080 bytes, including header and CRC; this leaves 0x0072 data bytes per PIVGETDATAR fragment when no security block is present.

A whole-object read includes the object's outer tag, length, and value. An inner-tag read includes the selected element's tag, length, and value, and excludes the enclosing object's tag and length. Offset and Requested Length then determine which bytes to return. Offset zero starts at the selected tag; skipping its tag and length returns only value bytes.

The PD divides the requested data across replies. Each TLV header occurs once in the reassembled data. Each reply has its own OSDP framing, reply identifier, and multi-part fields; the ACU joins only the Card Data bytes. Certificate and biometric values retain their original encoding. Selecting the FASC-N and UUID elements individually allows the ACU to read them regardless of their positions within the CHUID.

All exchanges use PD address 0x01 and CRC-16. Each request table gives the field values; the packet bytes below it show how those values are encoded.

Complete machine-readable exchanges include every frame. The fixture manifest records the inputs, expected results, source locations, and hashes. Hex includes the OSDP header and CRC, without an optional leading mark byte. Long replies show their first 0x18 and last 0x10 bytes; ellipsis text is not transmitted.

B.2 Source fixtures

S1: NIST SD33 card02, contact card capture.

S2: NIST SD33 card01, contact card capture.

The 19 examples below are a representative subset of the 37 checked exchanges in the linked JSON file.

B.3 Complete Discovery Object

Read the complete Discovery Object. Tag Length 0x00 keeps its outer tag 0x7E and its length, followed by all its contents.

Example ID: discovery-whole. Source: S1.

Table 40: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0014 returned bytes in 0x01 data reply.

Table 41: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 0F 00 05 A3 01 7E 00 00 00 00</u> <u>00 4A 01</u>	<u>Read request.</u>
<u>PD</u> <u>strips-leading</u> <u>zeroes before</u> <u>constructing the</u> <u>ISO-7816-4 APDU.</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 22 00 06 80 14 00 00 00 14 00</u> <u>7E 12 4F 0B A0 00 00 03 08 00 00 10</u> <u>00 01 00 5F 2F 02 68 20 DB F9</u>	<u>Total 0x0014</u> <u>Offset 0x0000</u> <u>Fragment 0x0014</u>

Object identifier, tag, offset. The PD transmits GET-DATA for tag and returns the complete CHUID-BER-TLV so the

B.4 Discovery PIN Usage Policy (0x5F2F)

Read the PIN Usage Policy from the Discovery Object. The reply contains tag 0x5F2F, length 0x02, and the two policy bytes.

Example ID: discovery-pin-policy. Source: S1.

Table 42: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x02</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x5F2F</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0005 returned bytes in 0x01 data reply.

Table 43: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>can-validate-the-</u> <u>PIVGETDATA</u>	<u>53 01 11 00 05 A3 01 7E 02 5F 2F 00</u> <u>00 00 00 03 F0</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 13 00 06 80 05 00 00 00 05 00</u> <u>5F 2F 02 68 20 31 67</u>	<u>Total 0x0005</u> <u>Offset 0x0000</u> <u>Fragment 0x0005</u>

B.5 Read the FASC-N element (0x30)

~~Read the FASC-N ,GUID, and expiration fields. Object identifier , tag , offset . Use this request during device discovery to read feature flags (contact /contactless support, optional biometrics) prior to issuing PIV Mode updates. Object identifier , tag , offset . The PD forwards tag with its nested data so the element from the CHUID without depending on its position. The returned element begins with tag 0x30 and length 0x19, followed by the FASC-N.~~

Example ID: chuid-fascn. Source: S1.

Table 44: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x30</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x001B returned bytes in 0x01 data reply.

Table 45: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU can parse the application identifier (tag) and PIN Usage Policy (tag). To retrieve only the AID, repeat the command with tag. Object identifier , tag , offset . Large responses for fingerprint or iris templates require multiple osdp_PIVDATAR fragments; the PIVGETDATA	<u>53 01 12 00 05 A3 03 5F C1 02 01 30</u> <u>00 00 00 00 15 89</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 29 00 06 80 1B 00 00 00 1B 00</u> <u>30 19 D6 50 18 58 55 E5 6D C8 12 79</u> <u>45 A1 6C DB 90 6E 78 80 C0 82 86 50</u> <u>18 43 F5 E1 F1</u>	<u>Total 0x001B</u> <u>Offset 0x0000</u> <u>Fragment 0x001B</u>

B.6 Read the FASC-N value

Read just the FASC-N bytes. Offset 0x0002 skips the selected element's 0x30 0x19 header; Requested Length 0x0019 limits the result to the FASC-N value.

Example ID: chuid-fascn-value. Source: S1.

Table 46: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x30</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0002</u>	<u>Bytes to skip from the selected tag.</u>
<u>Requested Length</u>	<u>0x0019</u>	<u>Maximum bytes to return.</u>

Result: 0x0019 returned bytes in 0x01 data reply.

Table 47: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 02 01 30</u> <u>02 00 19 00 96 DD</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 27 00 06 80 19 00 00 00 19 00</u> <u>D6 50 18 58 55 E5 6D C8 12 79 45 A1</u> <u>6C DB 90 6E 78 80 C0 82 86 50 18 43</u> <u>F5 6C 73</u>	<u>Total 0x0019</u> <u>Offset 0x0000</u> <u>Fragment 0x0019</u>

B.7 Read the UUID element (0x34)

Read the UUID (also called the GUID) element from the CHUID. The returned element begins with tag 0x34 and length 0x10, followed by the UUID.

Example ID: `chuid-guid`. Source: S1.

Table 48: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x34</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0012 returned bytes in 0x01 data reply.

Table 49: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 02 01 34</u> <u>00 00 00 00 13 00</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 20 00 06 80 12 00 00 00 12 00</u> <u>34 10 45 85 34 70 A4 03 48 44 AE 15</u> <u>C6 49 59 59 FA D5 FA C7</u>	<u>Total 0x0012</u> <u>Offset 0x0000</u> <u>Fragment 0x0012</u>

B.8 Read the UUID value

Read just the UUID bytes. Offset 0x0002 skips the selected element's 0x34 0x10 header; Requested Length 0x0010 limits the result to the UUID value.

Example ID: chuid-guid-value. Source: S1.

Table 50: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x34</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0002</u>	<u>Bytes to skip from the selected tag.</u>
<u>Requested Length</u>	<u>0x0010</u>	<u>Maximum bytes to return.</u>

Result: 0x0010 returned bytes in 0x01 data reply.

Table 51: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 02 01 34</u> <u>02 00 10 00 08 EE</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 1E 00 06 80 10 00 00 00 10 00</u> <u>45 85 34 70 A4 03 48 44 AE 15 C6 49</u> <u>59 59 FA D5 75 CB</u>	<u>Total 0x0010</u> <u>Offset 0x0000</u> <u>Fragment 0x0010</u>

B.9 Bounded CHUID prefix

Read the first 0x0010 bytes of the CHUID, starting with its outer tag and length. In this capture, the range ends partway through the FASC-N element.

Example ID: chuid-prefix. Source: S1.

Table 52: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0010</u>	<u>Maximum bytes to return.</u>

Result: 0x0010 returned bytes in 0x01 data reply.

Table 53: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 11 00 05 A3 03 5F C1 02 00 00</u> <u>00 10 00 A1 F4</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 1E 00 06 80 10 00 00 00 10 00</u> <u>53 82 07 C8 EE 02 07 C4 30 19 D6 50</u> <u>18 58 55 E5 5A 2E</u>	<u>Total 0x0010</u> <u>Offset 0x0000</u> <u>Fragment 0x0010</u>

B.10 Complete certificate object

Read the complete certificate object, including its outer 0x53 header and all elements inside it. The result is larger than one reply, so the ACU polls for the remaining fragments.

Example ID: certificate-whole. Source: S1.

Table 54: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>MpSizeTotal/0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x066F returned bytes in 0x0F data replies.

Table 55: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 11 00 05 A3 03 5F C1 05 00 00</u> <u>00 00 00 93 3F</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 80 00 06 80 6F 06 00 00 72 00</u> <u>53 82 06 6B 70 82 06 62 30 82 06 5E</u> <u>... 0x58 bytes omitted ...</u> <u>65 73 74 20 43 41 31 33 30 31 06 03</u> <u>55 04 4F 7E</u>	<u>Total 0x066F</u> <u>Offset 0x0000</u> <u>Fragment 0x0072</u>
<u>POLL/reply</u>	<u>Omitted</u>	<u>MpOffset0x0D exchanges;</u> <u>complete frames are in the</u> <u>JSON file.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll 0x0F.</u>

Message	Packet bytes (hex)	Notes
PD PIVGETDATAR	53 81 41 00 05 80 6F 06 3C 06 33 00 49 D9 67 65 56 F7 94 17 43 D7 84 02 9F 61 BF D6 D3 99 B6 DC D1 DE 0D 87 F5 F8 1A A9 CD 4D B8 BB D0 E6 6E 4B F5 75 87 9D AB F1 EF 91 D2 1A 71 01 00 FE 00 F7 26	Total 0x066F Offset 0x063C Fragment 0x0033

B.11 Read the certificate element (0x70)

Read element 0x70, including its tag, encoded length, and certificate bytes. This omits the enclosing 0x53 header and the other elements in the object. The 0x70 header appears once in the returned data.

Example ID: certificate-element. Source: S1.

Table 56: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x70</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0666 returned bytes in 0x0F data replies.

Table 57: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU PIVGETDATA	53 01 12 00 05 A3 03 5F C1 05 01 70 00 00 00 00 39 81	Read request.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll 0x01.
PD PIVGETDATAR	53 81 80 00 06 80 66 06 00 00 72 00 70 82 06 62 30 82 06 5E 30 82 04 46 ... 0x58 bytes omitted ... 43 41 31 33 30 31 06 03 55 04 03 13 2A 54 2D 25	Total 0x0666 Offset 0x0000 Fragment 0x0072

Message	Packet bytes (hex)	Notes
<u>POLL/</u> MpFragmentSize fields support reconstruction. Object identifier, tag , offset. The reply	<u>Omitted</u>	<u>0x0D exchanges; complete frames are in the JSON file.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll 0x0F.</u>
PD returns the complete PIV Application Identifier BER-TLV-nested inside the Discovery Object, including tag, length, and value <u>PIVGETDATAR</u>	<u>53 81 38 00 05 80 66 06 3C 06 2A 00</u> <u>56 F7 94 17 43 D7 84 02 9F 61 BF D6</u> <u>D3 99 B6 DC D1 DE 0D 87 F5 F8 1A A9</u> <u>CD 4D B8 BB D0 E6 6E 4B F5 75 87 9D</u> <u>AB F1 EF 91 D2 1A 8B 7C</u>	<u>Total 0x0666</u> <u>Offset 0x063C</u> <u>Fragment 0x002A</u>

B.12 Read a compressed certificate element

Read element 0x70 from a card that stores a compressed certificate. The PD returns the original tag, length, and compressed bytes. The ACU can read the object's certificate-information element to determine how to interpret the value.

Example ID: certificate-compressed-element. Source: S2. ~~Tags that occupy more than one byte (for example) are outside the scope of this selector and must be obtained by requesting tag.~~

Table 58: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x70</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x05EA returned bytes in 0x0E data replies.

Table 59: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 05 01 70</u> <u>00 00 00 00 39 81</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 80 00 06 80 EA 05 00 00 72 00</u> <u>70 82 05 E6 1F 8B 08 00 00 00 00 00</u> <u>... 0x58 bytes omitted ...</u> <u>CE F4 47 FF F4 CF 5D BE EF 9C 7B CF</u> <u>39 F7 79 A2</u>	<u>Total 0x05EA</u> <u>Offset 0x0000</u> <u>Fragment 0x0072</u>
<u>POLL/reply</u>	<u>Omitted</u>	<u>0x0C exchanges; complete frames are in the JSON file.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>Poll 0x0E.</u>

Message	Packet bytes (hex)	Notes
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 2E 00 07 80 EA 05 CA 05 20 00</u> <u>7F C4 30 BC D9 5C B8 05 0A 09 18 39</u> <u>F8 6D A5 80 16 10 4D DC 8D 38 FC 05</u> <u>BF 05 DC 1B AF 07 00 00 96 99</u>	<u>Total 0x05EA</u> <u>Offset 0x05CA</u> <u>Fragment 0x0020</u>

B.13 Read a range within the certificate element

Read 0x0104 bytes starting at offset 0x0100 of the certificate element, counting from tag 0x70. The reply offsets are 0x0000, 0x0072, and 0x00E4; each identifies a position within the 0x0104 returned bytes.

Example ID: certificate-offset256. Source: S1.

Table 60: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x70</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0100</u>	<u>Bytes to skip from the selected tag.</u>
<u>Requested Length</u>	<u>0x0104</u>	<u>Maximum bytes to return.</u>

Result: 0x0104 returned bytes in 0x03 data replies.

Table 61: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 05 01 70</u> <u>00 01 04 01 EC 6A</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 80 00 06 80 04 01 00 00 72 00</u> <u>14 30 12 06 03 55 04 0B 13 0B 54 65</u> <u>... 0x58 bytes omitted ...</u> <u>00 03 82 01 0F 00 30 82 01 0A 02 82</u> <u>01 01 CE C4</u>	<u>Total 0x0104</u> <u>Offset 0x0000</u> <u>Fragment 0x0072</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>Poll 0x02.</u>

Message	Packet bytes (hex)	Notes
PD PIVGETDATAR	53 81 80 00 07 80 04 01 72 00 72 00 00 C3 C3 53 BF 0F 03 48 37 48 A4 BA ... 0x58 bytes omitted ... 4A A1 F7 74 E7 A2 DE BC F3 87 AA 94 76 21 24 7E	Total 0x0104 Offset 0x0072 Fragment 0x0072
ACU POLL	53 01 08 00 05 60 8B 33	Poll 0x03.
PD PIVGETDATAR	53 81 2E 00 05 80 04 01 E4 00 20 00 FA 54 AA 2C DF 26 69 45 7E 6D CD 3A 48 EE 5E 4B 31 FD A9 4A D8 73 01 A7 5C C8 50 D2 0A 60 D1 CC 14 CD	Total 0x0104 Offset 0x00E4 Fragment 0x0020

B.14 Complete Key History object

Read the complete Key History object. Its outer 0x53 header and all elements inside it are returned unchanged.

Example ID: key-history. Source: S1.

Table 62: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC10C</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x000A returned bytes in 0x01 data reply.

Table 63: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU PIVGETDATA	53 01 11 00 05 A3 03 5F C1 0C 00 00 00 00 00 71 77	Read request.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll 0x01.
PD PIVGETDATAR	53 81 18 00 06 80 0A 00 00 00 0A 00 53 08 C1 01 04 C2 01 00 FE 00 F3 05	Total 0x000A Offset 0x0000 Fragment 0x000A

B.15 BIT Group with no biometric templates

Read a Biometric Information Template (BIT) Group with no biometric templates. The reply keeps the outer 0x7F61 tag and the element that reports a count of zero.

Example ID: bit-empty. Source: constructed example.

Table 64: Request fields

Field	Value	Notes
Command	0xA3	osdp_PIVGETDATA
Identifier Length	0x02	Bytes in Object Identifier.
Object Identifier	0x7F61	Object to read.
Tag Length	0x00	Select the whole object.
Element Tag	Omitted	Tag Length is zero.
Offset	0x0000	Start at the selected tag.
Requested Length	0x0000	All remaining bytes.

Result: 0x0006 returned bytes in 0x01 data reply.

Table 65: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU PIVGETDATA	53 01 10 00 05 A3 02 7F 61 00 00 00 00 00 3E 14	Read request.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll 0x01.
PD PIVGETDATAR	53 81 14 00 06 80 06 00 00 00 06 00 7F 61 03 02 01 00 35 7C	Total 0x0006 Offset 0x0000 Fragment 0x0006

B.16 Empty 0x53 container

Read an object with no contents. The reply still contains two bytes, 0x53 0x00, because whole-object selection includes the outer tag and length.

Example ID: empty-5300. Source: constructed example.

Table 66: Request fields

Field	Value	Notes
Command	0xA3	osdp_PIVGETDATA
Identifier Length	0x03	Bytes in Object Identifier.
Object Identifier	0x5FC105	Object to read.
Tag Length	0x00	Select the whole object.
Element Tag	Omitted	Tag Length is zero.
Offset	0x0000	Start at the selected tag.
Requested Length	0x0000	All remaining bytes.

Result: 0x0002 returned bytes in 0x01 data reply.

Table 67: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU PIVGETDATA	53 01 11 00 05 A3 03 5F C1 05 00 00 00 00 00 93 3F	Read request.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll 0x01.
PD PIVGETDATAR	53 81 10 00 06 80 02 00 00 00 02 00 53 00 5D 38	Total 0x0002 Offset 0x0000 Fragment 0x0002

B.17 Zero-byte successful response

Handle a successful card response with zero data bytes. The PD sends one PIVGETDATAR with all three multi-part fields zero.

Example ID: empty-success. Source: constructed example.

Table 68: Request fields

Field	Value	Notes
Command	0xA3	osdp_PIVGETDATA
Identifier Length	0x01	Bytes in Object Identifier.
Object Identifier	0x7E	Object to read.
Tag Length	0x00	Select the whole object.
Element Tag	Omitted	Tag Length is zero.
Offset	0x0000	Start at the selected tag.
Requested Length	0x0000	All remaining bytes.

Result: 0x0000 returned bytes in 0x01 data reply.

Table 69: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 0F 00 05 A3 01 7E 00 00 00 00</u> <u>00 4A 01</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 0E 00 06 80 00 00 00 00 00 00</u> <u>C0 05</u>	<u>Total 0x0000</u> <u>Offset 0x0000</u> <u>Fragment 0x0000</u>

B.18 Existing child with an empty value

Read an element whose value is empty. The reply contains its tag and zero length, 0x30 0x00, for a total of two bytes.

Example ID: zero-length-child. Source: constructed example.

Table 70: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x30</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0002 returned bytes in 0x01 data reply.

Table 71: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 10 00 05 A3 01 7E 01 30 00 00</u> <u>00 00 DC 37</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 10 00 06 80 02 00 00 00 02 00</u> <u>30 00 24 66</u>	<u>Total 0x0002</u> <u>Offset 0x0000</u> <u>Fragment 0x0002</u>

B.19 Missing direct child

Request an element that is absent from the object. The PD accepts the command, then reports PIVERROR on the next poll.

Example ID: missing-child. Source: constructed example.

Table 72: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x34</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: osdp_PIVERROR 0x1024 (data object or tag not found); see Section 2.2.1.

Table 73: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU PIVGETDATA	53 01 10 00 05 A3 01 7E 01 34 00 00 00 00 DA BE	Read request.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll 0x01.
PD PIVERROR	53 81 0A 00 06 8A 24 10 A3 74	0x1024 (data object or tag not found)

B.20 Element request against an empty response

Request an element when the card returns no data. The PD reports that the tag was not found. A whole-object request for the same response would succeed with zero bytes.

Example ID: empty-missing-child. Source: constructed example.

Table 74: Request fields

Field	Value	Notes
Command	0xA3	osdp_PIVGETDATA
Identifier Length	0x01	Bytes in Object Identifier.
Object Identifier	0x7E	Object to read.
Tag Length	0x01	Bytes in Element Tag.
Element Tag	0x34	Element to select.
Offset	0x0000	Start at the selected tag.
Requested Length	0x0000	All remaining bytes.

Result: osdp_PIVERROR 0x1024 (data object or tag not found); see Section 2.2.1.

Table 75: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 10 00 05 A3 01 7E 01 34 00 00</u> <u>00 00 DA BE</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVERROR</u>	<u>53 81 0A 00 06 8A 24 10 A3 74</u>	<u>0x1024 (data object or tag not found)</u>

B.21 Ambiguous direct-child selection

Request a tag that occurs twice directly inside the outer container. Element selection requires a unique match, so the PD reports an error.

Example ID: duplicate-child. Source: constructed example.

Table 76: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x30</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: osdp_PIVERROR 0x1027 (ambiguous element selection); see Section 2.2.1.

Table 77: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 10 00 05 A3 01 7E 01 30 00 00</u> <u>00 00 DC 37</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVERROR</u>	<u>53 81 0A 00 06 8A 27 10 F0 21</u>	<u>0x1027 (ambiguous element selection)</u>

C High-Speed Autonomous PIV Data Templates (Informative)

This appendix records the PIV Auto challenge, ISO/IEC 7816 GENERAL AUTHENTICATE templates, and fixture sources used by the demonstration vectors. The APDU examples are validated by the repository tests against the same helper code used by the simulator and replay tools.

C.1 Vector Sources

Table 78: PIV Auto Vector Sources

Source	Use	Notes
NIST SD 33 / IR 8347	Live-card APDU fixtures	Cards 2, 3, 4, 5, and 16 provide VCI contactless enrolment, OPACITY, and key-challenge transcripts.
NIST SM/VCI captures	CVC and trust anchors	Provides validator-ready 7F21, 5FC122, and 7F50 objects.
GSA FICAM card-builder	Local keys and certificates	Used only as reusable local test material for source-backed simulation.
Generated YubiKey PIV	Missing algorithm profiles	Reusable RSA-1024, RSA-3072, and P-384 test keys are committed as public fixtures and loaded through ykmanpiv; RSA-1024 is informative legacy test material only.

C.2 Supported Profiles

Table 79: PIV Auto Supported Profiles

Profile	Alg	Key Ref	PIV Auto	Coverage
RSA-1024	06	9A, 9E	No	Informative legacy/non-current test-only material.
RSA-2048	07	9A, 9E	Yes	NIST SD 33 cards 2, 3, 16; generated-card; GSA-backed simulation.
RSA-3072	05	9A, 9E	Yes	Committed YubiKey material; requires YubiKey firmware with RSA-3072 support.
ECC P-256	11	9A, 9E	Yes	NIST SD 33 cards 4 and 5; generated-card coverage.
ECC P-384	14	9A, 9E	Yes	Committed YubiKey material. NIST card 5 also provides 9C/9D APDU examples.

C.3 Fixed-Width KMAC Challenge Template

Table 80: PIV Auto KMAC Challenge Template

Field	Size	Encoding	Notes
Template label	14 0x0E	ASCII plus version byte	OSDP-PIV-AUTO followed by 01.
Series Counter	0x04	LSB first	ACU-managed boot or epoch counter, for example 07000000.
Sequence Counter	0x04	LSB first	ACU-managed per-series counter, for example 2A000000.
FASC-N	25 0x19	Raw bytes	Credential FASC-N from CHUID.
UUID	16 0x10	Raw bytes	Credential UUID. Successful PIV Auto examples use the credential UUID; unavailable UUID is a failure condition.
PIV Key Reference	0x01	Raw byte	Challenged key reference, normally 9A or 9E.
PIV Algorithm	0x01	Raw byte	Algorithm identifier: 05, 06, 07, 11, or 14.
Supplemental Entropy	32 0x20	Raw bytes	Fixed-width ACU entropy field; all zeroes are valid for test vectors only.

The challenge digest is produced in two steps. First, derive PIV-AUTO-KDK=KMAC256(key=S-ENC|S-MAC1|S-MAC2,data=empty,mac_len=32,custom="OSDP-PIV-AUTO-KDK-v1"). Second, derive the challenge as KMAC256(key=PIV-AUTO-KDK,data=fixed-widthtemplate,mac_len=L,custom="OSDP-PIV-AUTO-CHALLENGE-v1"). L is 32 bytes for SHA-256 profiles and 48 bytes for SHA-384 profiles. The supplemental entropy field is always present and fixed at 32 bytes; all zeroes are valid for test vectors but not recommended for deployments.

C.4 Dynamic Authentication Templates

C.4.1 RSA Key Challenge Input

Table 81: RSA Dynamic Authentication Template

Tag / Bytes	Length	Name	Value
7C	Variable	Dynamic Authentication Template	Outer template for the PIV GENERAL AUTHENTICATE command data.
82	00	Response request	Empty value requesting the card response in tag 82.
81	Modulus length	Challenge block	EMSA-PKCS1-v1_5 encoded digest block with the same length as the RSA modulus.
0001FF...FF00	Variable	Padding prefix	EMSA-PKCS1-v1_5 block prefix before the digest information.
30...	Variable	DigestInfo	ASN.1 DigestInfo for SHA-256 or SHA-384 followed by the KMAC-derived digest; SHA-256 uses prefix 3031300D060960864801650304020105000420 and SHA-384 uses prefix 3041300D060960864801650304020205000430.

Table 82: RSA Dynamic Authentication Examples

Profile	Alg	Key Ref	Tag 81	Response 82	Command APDU
RSA-1024	06	9A/9E	128 0x80 bytes	128 0x80 bytes	Informative legacy/non-current test-only APDU.
RSA-2048	07	9A/9E	256 0x100 bytes	256 0x100 bytes	274-byte 0x112-byte extended APDU.
RSA-3072	05	9A/9E	384 0x180 bytes	384 0x180 bytes	402-byte 0x192-byte extended APDU.

C.4.2 ECC Signature Input

Table 83: ECC Dynamic Authentication Template

Tag / Bytes	Length	Name	Value
7C	Variable	Dynamic Authentication Template	Outer template for the PIV GENERAL AUTHENTICATE command data.
82	00	Response request	Empty value requesting the card response in tag 82.
81	Digest length	Challenge digest	KMAC-derived digest supplied directly to the ECC signing operation.
Digest bytes	32 or 48 <u>0x20</u> or <u>0x30</u> bytes	Digest value	SHA-256-width digest for P-256; SHA-384-width digest for P-384.

Table 84: ECC Dynamic Authentication Examples

Profile	Alg	Key Ref	Tag 81	Response 82	Command APDU
ECC P-256	11	9A/9E	32 <u>0x20</u> bytes	DER ECDSA signature, about 70-72 <u>0x46-0x48</u> bytes	44-byte <u>0x2C</u> -byte short APDU.
ECC P-384	14	9A/9E	48 <u>0x30</u> bytes	DER ECDSA signature, about 102-104 <u>0x66-0x68</u> bytes	60-byte <u>0x3C</u> -byte short APDU.

C.4.3 Worked Dynamic Authentication Examples

Worked examples are generated from
test-vectors/piv-auto-demo/generated/piv-auto-simulation-report.json.

Table 85: PIV Auto Worked Example: PIV Authentication RSA-1024

Field	Value
Profile	9a-rsa1024
Source	Committed YubiKey loader material in test-vectors/piv-auto-yubikey-material/9a-rsa1024. Informative legacy/non-current test-only coverage.
Algorithm ID	0x06
Key Reference	0x9A
KMAC Challenge Digest	5B2FC08860E3999EEF8F6CF5D2B0C9AE05CBB28A5B423BFE4F646F6F47FCA4F2

Field	Value
GENERAL AUTHENTICATE APDU	<pre> 0087059E00018A7C8201868200818201800001FFFFFFFFFFFFFFFFFFFFFFFF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 003041300D0609608648016503040202050004 304C98712B25DB7F477096FA3D0AC93FA2CA6BBE89CB291AC26EDF0D8312CA61 DOCA238F1656247B2DF38E8B72D9AF7F5E00 </pre>
Dynamic Authentication Response	<pre> 7C8201848282018074FA8827F5C2CFDAA5718679727015CD5A928828997F2F14 23D033DE8AADEB3E86431170A714F5155B830F65F3F854120B16BC50F701C0EE A0278DB7EC2D7D6D587BEDB32F5E34EF1421E2C42543FF7DEEF7C5A41A01275B 0BC126AF1208BBBD1E2C65149BC74C9E2CD00D822081B22E5CCB5090D2BD3970 CA309D2124D79E9A28978D5DDF26E3FA8E37C360521C5BBE1AB27FACC24FF4C5 9AB46B7D571777B22E3E4981593D9F689DA3D33F3DE7C385C4315A7F154BE461 968EA05A3315C4424717DD6F3CBF7FB94A135AE33573E58A18185F7E25771E6C 4AB2B89D50007C7A60E6D65CF3BE58F8901B25B867D0FF97981BCBEE2BFCF5E5 7CEB5A81F40EEB44E52A0BD7F538F932AB4F1732828F2E95C4F203967ADF5983 A285A127E6C0D7C2048261290B6B58D8110056DFE65547A7C1BDF78C6FEA62A8 E07EC932AC6B58AD13C9423AAD5815C5251BCECEBBDB181F0B1C362163373463 5FD48CD1AD16798C9B67CDF2BAB290DE0EA0BC5BA54E41736BA4885AF72D729A 5168DC09DDD4B6629000 </pre>
Response Tag Length	<pre> 384-0x0180 bytes </pre>

Table 91: PIV Auto Worked Example: PIV Authentication ECC P-256

Field	Value
Profile	9a-ecp256
Source	Committed P-256 signing key in test-vectors/piv-auto-demo/source-p12, originally imported from GSA FICAM card-builder. NIST SD 33 / IR 8347 and generated-card coverage.
Algorithm ID	0x11
Key Reference	0x9A
KMAC Challenge Digest	A3C81D3205FFB28A2AAAB0EDC21E8B3BEF5BD5F685FF5C686502CA5609B45851
Tag 81 Challenge Input	A3C81D3205FFB28A2AAAB0EDC21E8B3BEF5BD5F685FF5C686502CA5609B45851
GENERAL AUTHENTICATE APDU	0087119A267C2482008120A3C81D3205FFB28A2AAAB0EDC21E8B3BEF5BD5F685FF5C686502CA5609B4585100

Field	Value
Dynamic Authentication Response	7C4A824830460221009D25A44E56572983819019DF26A2720E462A11D2CB3CD86A9142DEF4BCC1271B022100D084982BB9D3E2D3E6B0965956C71DBA0FE44FE328C0A2BD72951DFB4D9296FE9000
Response Tag <u>0x82</u> Length	72 - <u>0x0048</u> bytes

Table 92: PIV Auto Worked Example: Card Authentication ECC P-256

Field	Value
Profile	9e-ecp256
Source	Committed P-256 signing key in test-vectors/piv-auto-demo/source-p12, originally imported from GSA FICAM card-builder. NIST SD 33 / IR 8347 and generated-card coverage.
Algorithm ID	0x11
Key Reference	0x9E
KMAC Challenge Digest	965AF088B02DF65E7C299ABF53A5AB2DCBB3C789458DE8D0C819B19A9266658D
Tag 81 Challenge Input	965AF088B02DF65E7C299ABF53A5AB2DCBB3C789458DE8D0C819B19A9266658D
GENERAL AUTHENTICATE APDU	0087119E267C2482008120965AF088B02DF65E7C299ABF53A5AB2DCBB3C789458DE8D0C819B19A9266658D00
Dynamic Authentication Response	7C4A824830460221008B204FA48FAA27A57D6E90F985199F284D2A8E0D0F2E051BBFBCACBA824619B9022100C6E04880D322DE7D10823EC78C8EA4CCB69FE394F694F1D12C797FA7BCAD76959000
Response Tag <u>0x82</u> Length	72 - <u>0x0048</u> bytes

Table 93: PIV Auto Worked Example: PIV Authentication ECC P-384

Field	Value
Profile	9a-ecp384
Source	Committed YubiKey loader material in test-vectors/piv-auto-yubikey-material/9a-ecp384. Generated-card coverage.
Algorithm ID	0x14
Key Reference	0x9A
KMAC Challenge Digest	C813683447986D513FC17CED5DB4D335BFBEAFB1AF9E5200C6B7B1C1E3535D700C2E05F9011717E153998832B7588D28
Tag 81 Challenge Input	C813683447986D513FC17CED5DB4D335BFBEAFB1AF9E5200C6B7B1C1E3535D700C2E05F9011717E153998832B7588D28

Field	Value
GENERAL AUTHENTICATE APDU	0087149A367C3482008130C813683447986D513FC17CED5DB4D335BFBEAFB1AF9E5200C6B7B1C1E3535D700C2E05F9011717E153998832B7588D2800
Dynamic Authentication Response	7C6A82683066023100D88ABABB9893D88341CFA2F1AB6E3D35E4AF02F4F8D085B5A3DE63574DCF694741D869066B8E6297429120614DCC9CE40231009A3F203BB5EA98839008B0D25F98C98DE8E87622F61279B2F2B29343912999A36311102B310F9985844BCC248350377B9000
Response Tag <u>0x82</u> Length	104 <u>0x0068</u> bytes

Table 94: PIV Auto Worked Example: Card Authentication ECC P-384

Field	Value
Profile	9e-ecp384
Source	Committed YubiKey loader material in test-vectors/piv-auto-yubikey-material/9e-ecp384. Generated-card coverage.
Algorithm ID	0x14
Key Reference	0x9E
KMAC Challenge Digest	F3480C6C1DAD008D3E14D0C815D381F01420E9D3402A175BED097C6949E4BA447FA0A11745CE4D054A95B10A9D5CC689
Tag 81 Challenge Input	F3480C6C1DAD008D3E14D0C815D381F01420E9D3402A175BED097C6949E4BA447FA0A11745CE4D054A95B10A9D5CC689
GENERAL AUTHENTICATE APDU	0087149E367C3482008130F3480C6C1DAD008D3E14D0C815D381F01420E9D3402A175BED097C6949E4BA447FA0A11745CE4D054A95B10A9D5CC68900
Dynamic Authentication Response	7C6982673065023034C6B2659FBE26A78D2615F8C0EBF2E9674292D9DE3CE8A31D9DA297B5AE236AC886E9E9C288D42474564BF0C46AE40C023100CCBD59A047D73F9FD58F60218FBCB4F99B18EBD7EB70EFE5CF64D5EB1A65F1676A867E1B1E5EB4CF9F34B44BE905D29C9000
Response Tag <u>0x82</u> Length	103 <u>0x0067</u> bytes

C.5 APDUs

Table 95: PIV Auto APDU Reference: Card Preparation

Operation	Command APDU	Success Response	Purpose
Select PIV Application	00A404000BA000000308000010000100	9000	Selects the PIV application before reading objects or authenticating.
Get Discovery Object	00CB3FFF035C017E	7E...9000	Reads the Discovery Object so the PD can determine application and PIN/VCI policy.
Verify PIN (optional)	0020008008313233343536FFFF	9000	Verifies example PIN 123456 padded with FF; failure responses include 63CX and 6983.

Table 96: PIV Auto APDU Reference: GENERAL AUTHENTICATE Command

Field	Encoding	Source	Notes
CLA	00	ISO/IEC 7816-4	Use when a credential secure channel is active. Logical command before credential SM wrapping. Apply SP 800-73-5 Part 2, Section 4.2, when SM is established.
INS	87	ISO/IEC 7816-4	GENERAL AUTHENTICATE instruction.
P1	Algorithm ID	PIV profile	05, 07, 11, or 14 for supported PIV Auto profiles; 06 appears only in informative RSA-1024 legacy examples.
P2	Key Reference	PIV profile	9A or 9E for PIV Auto.
Lc	Short or extended	Command length	Short APDUs use one length byte; extended APDUs use 00 followed by two length bytes.
Command data	7C...820081.....	Dynamic Authentication Template	Tag 82 is empty to request a response; tag 81 carries the exact challenge input.
Le	00	Response request	Requests the maximum response supported by the card interface.

Table 97: PIV Auto APDU Reference: GENERAL AUTHENTICATE Response

Case	Response APDU	PD Handling	OSDP Payload
Complete response	7C...82...9000	Strip only the ISO status word.	Carry the complete 7C Dynamic Authentication Template in osdp_CRAUTHR or osdp_PIVSTATUS.
Additional response bytes	61XX	Issue GET RESPONSE with Le XX.	Concatenate returned data before reporting the final 7C template.
GET RESPONSE command	00C00000XX	Retrieve remaining card response bytes.	Not exposed directly to the ACU.
Credential error	69.. or 6A..	Preserve command failure detail.	Report via osdp_PDERROR PIVERROR when available; do not treat as a valid signed response.

Table 98: PIV Auto APDU Reference: OPACITY / VCI Context

Operation	Command Shape	Key Ref	Relationship to PIV Auto
OPACITY establish	0087[27/2E]04[Lc]7C...81.. ..8200	04	Establishes VCI-secure-messaging and-returns-cryptogram-/CVC evidence.-credential SM after CVC and cryptogram verification. <u>Contactless VCI also depends on Discovery policy and pairing when required.</u>
PIV Auto challenge	0087[alg][9A/9E][Lc]7C... 820081.....00	9A/9E	Performs the autonomous key challenge after card preparation and any required VCI setup.
Response handling	7C...82...9000	9A/9E	The PD reports the returned signed response bytes exactly, excluding only SW1/SW2.

D Credential SM and VCI Worked Examples (Informative)

These examples follow the preparation and PIN-verification flows in Subsection 3.1. A simulated ACU sends complete OSDP packets to a simulated PD. The PD replays existing card captures and reports the resulting credential state. These examples omit OSDP Secure Channel for clarity; the test harness assumes that the OSDP Secure Channel authorization requirements are satisfied.

The contact and contactless card01 captures supply the card commands and replies. The verifier checks the CVC signature against an existing trust anchor, recomputes SM key establishment, and checks protected command and response MACs and encryption. The replay retains intervening captured exchanges so the session counters and MAC chains remain intact. Those intervening exchanges preserve the original card session. The card captures include earlier provisioning and data-read commands; those commands impose no additional requirements on automatic card preparation. The examples replay recorded sessions; no connected card is used.

Table 99: Replay coverage

Case	Evidence	Purpose
Contact SM	Captured SM session and CVC validation	Report SM with VCI absent.
Incorrect PIN	Modeled authenticated rejection	Clear the failed PIN while preserving SM and VCI.
Commanded pairing and PIN	Captured protected VERIFY exchanges	Complete pairing before verifying the cached PIN.
PIV Auto failure	Modeled results and full status words	Report the cause across multipart replies.
PIN cache expiration	Modeled timing, with captured VERIFY for the submitted-command case	Separate cache availability from credential PIN verification.

Complete packets, APDUs, and source references are in the SM/VCI example file. The accompanying README gives the replay and independent-verification commands. Packet dumps use hexadecimal byte pairs, including the OSDP header and CRC. Each example starts at sequence 0x01 on an established link to PD address 0x01. Standard status-query and pairing replies use the same osdp_CARDSTATUSR layout with Status Type 0x01. Diagnostic replies carry Status Type 0x02 before the multi-part fields in every fragment. Timing examples identify the modeled elapsed time at each exchange; captured APDUs retain their original order.

D.1 Credential SM over contact

The PD prepares a contact credential and reports SM established without VCI. The reply identifies the CVC issuer and trust anchor used for that SM session.

Example ID: contact-sm. Credential exchanges come from the contact card01 capture. The complete replay retains its original APDU order.

Table 100: Reported security state

Field	Value	Notes
<u>Status Type</u>	0x01	<u>Standard credential status.</u>
<u>Credential Security Status</u>	0x02	<u>SM established without VCI.</u>
<u>SM Trust Anchor ID</u>	0x01	<u>Loaded anchor used to validate SM.</u>
<u>SM CVC IIN</u>	0xB6103792270FBD08	<u>Issuer named by the credential's SM CVC.</u>
<u>SM Anchor IIN</u>	0xB6103792270FBD08	<u>Same issuer: direct trust path.</u>

Table 101: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>CARDSTATUS</u>	<u>53 01 09 00 05 A9 01 8C 9B</u>	<u>Read current state after automatic credential preparation.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll for completion.</u>
<u>PD</u> <u>CARDSTATUSR</u>	<u>53 81 2A 00 06 85 01 01 02 01 B6 10</u> <u>37 92 27 0F BD 08 B6 10 37 92 27 0F</u> <u>BD 08 48 00 0B A0 00 00 03 08 00 00</u> <u>10 00 01 00 98 79</u>	<u>Status 0x02: SM established without VCI.</u>

D.2 PIN before credential presentation

The PD collects the PIN locally and reports completion before the credential is presented. Initial selection and SM establishment preserve the cache. The ACU then completes pairing and requests PIN verification over VCI.

Example ID: cache-before-card. Credential exchanges come from the contactless card01 capture. The complete replay retains its original APDU order.

Table 102: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 04 00 00 00 3C 00</u> <u>FE 9F</u>	<u>Collect a PIN locally; cache for</u> <u>0x003C seconds.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 06 86 00 05 10</u>	<u>Result 0x00: requested PIN</u> <u>operation completed.</u>
<u>ACU</u> <u>CARDSTATUS</u>	<u>53 01 09 00 07 A9 01 EC F5</u>	<u>Read current state after</u> <u>automatic credential</u> <u>preparation.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 07 40 5B 53</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll for completion.</u>
<u>PD</u> <u>CARDSTATUSR</u>	<u>53 81 2A 00 05 85 01 02 02 01 B6 10</u> <u>37 92 27 0F BD 08 B6 10 37 92 27 0F</u> <u>BD 08 48 00 0B A0 00 00 03 08 00 00</u> <u>10 00 01 00 1E 84</u>	<u>Status 0x02: SM established</u> <u>without VCI.</u>
<u>ACU</u> <u>PIVXMITPAIRING</u>	<u>53 01 10 00 06 AD 30 30 30 30 30 30</u> <u>30 32 EB 8B</u>	<u>Submit the captured pairing</u> <u>code.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 06 40 6A 60</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>Poll for completion.</u>
<u>PD</u> <u>CARDSTATUSR</u>	<u>53 81 2A 00 07 85 01 02 01 01 B6 10</u> <u>37 92 27 0F BD 08 B6 10 37 92 27 0F</u> <u>BD 08 48 00 0B A0 00 00 03 08 00 00</u> <u>10 00 01 00 56 3D</u>	<u>Status 0x01: SM and VCI</u> <u>established.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 02 00 00 00 00 00</u> <u>E7 52</u>	<u>Verify the cached PIN over</u> <u>established VCI.</u>

Message	Packet bytes (hex)	Notes
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 06 86 00 05 10</u>	<u>Result 0x00: requested PIN operation completed.</u>

D.3 PIV Auto needs commanded pairing

In this modeled case, the configured PIV Auto authentication requires VCI. SM is established, but pairing remains necessary. PIV Auto reports the unmet condition and ends the attempt. A later poll receives ACK; the ACU can begin commanded pairing.

Example ID: auto-pairing-required.

Table 103: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll for completion.</u>
<u>PD</u> <u>PIVSTATUS</u>	<u>53 81 45 00 05 89 37 00 00 00 37 00</u> <u>05 01 01 00 00 00 02 00 00 00 00 00</u> <u>00 00 00 00 00 00 00 00 00 00 00 00</u> <u>00 00 00 00 00 00 00 00 00 00 00 00</u> <u>00 00 00 00 00 00 00 00 00 00 00 00</u> <u>00 00 00 9A 07 00 00 FF 09</u>	<u>Result 0x05, Detail 0x01: pairing code required.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll for completion.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 06 40 6A 60</u>	<u>No completion pending.</u>

D.4 SM failure diagnostics

In this modeled case, PIV Auto reports Result 0x02, Detail 0x04: invalid authentication cryptogram. The ACU then requests the SM diagnostic record. The PD returns the failure reason and the hash of the received CVC, with no further credential commands. Three fragments carry the 35-byte record. The CVC bytes used for the hash come from the existing capture.

Example ID: diagnostic-cvc-hash.

Table 104: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>CARDSTATUS</u>	<u>53 01 09 00 06 A9 02 BF F2</u>	<u>Request retained SM</u> <u>diagnostics; no credential</u> <u>command is sent.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 06 40 6A 60</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>Poll for completion.</u>
<u>PD</u> <u>CARDSTATUSR</u>	<u>53 81 1F 00 07 85 02 23 00 00 00 10</u> <u>00 03 04 01 A5 50 00 73 BA 17 70 37</u> <u>0A 1D 58 B1 5B 2F F2</u>	<u>Diagnostic: total 0x0023, offset</u> <u>0x0000, fragment 0x0010.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll for completion.</u>
<u>PD</u> <u>CARDSTATUSR</u>	<u>53 81 1F 00 05 85 02 23 00 10 00 10</u> <u>00 97 5D D1 50 7F 39 22 BB 44 20 69</u> <u>9B 2D FB 42 83 52 B1</u>	<u>Diagnostic: total 0x0023, offset</u> <u>0x0010, fragment 0x0010.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll for completion.</u>
<u>PD</u> <u>CARDSTATUSR</u>	<u>53 81 12 00 06 85 02 23 00 20 00 03</u> <u>00 C7 6C E9 B4 96</u>	<u>Diagnostic: total 0x0023, offset</u> <u>0x0020, fragment 0x0003.</u>

D.5 PIN cache expires while idle

The PD acquires a PIN at t=0 with a ten-second validity period. At t=10 the PD erases the PIN. A poll receives ACK because no operation is pending. A status query reports an empty cache; a subsequent verification request ends with no cached PIN. The PD sends no VERIFY. This example models the timing and credential state.

Example ID: pin-expiration-idle.

Table 105: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 04 00 00 00 0A 00</u> <u>CD 30</u>	<u>t=0 s. Acquire a PIN; Cache</u> <u>Validity is 0x000A seconds.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>t=0 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>t=0 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 06 86 00 05 10</u>	<u>t=0 s. Result 0x00: requested</u> <u>PIN operation completed.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>t=10 s. Poll for completion.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 07 40 5B 53</u>	<u>t=10 s. No completion pending.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 00 00 00 00 00 00</u> <u>A7 D9</u>	<u>t=10 s. Report empty cache and</u> <u>no PIN verification.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 05 86 00 55 49</u>	<u>t=10 s. Cache empty; credential</u> <u>PIN verification absent.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 06 AA 03 00 00 00 00 00</u> <u>32 DF</u>	<u>t=10 s. Request verification</u> <u>using the expired cache.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 06 40 6A 60</u>	<u>t=10 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>t=10 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 07 86 0A 7F 86</u>	<u>t=10 s. Result 0x0A: no cached</u> <u>PIN.</u>

D.6 Expiration while waiting for a credential

The ACU arms cached-PIN verification with Auto Execute flag 0x02. The credential is absent when the cache expires at t=10. The PD ends the pending request with PIVSPER 0x0A (no cached PIN). This example models the waiting state.

Example ID: pin-expiration-waiting.

Table 106: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 04 00 00 00 0A 00</u> <u>CD 30</u>	<u>t=0 s. Acquire a PIN; Cache</u> <u>Validity is 0x000A seconds.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>t=0 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>t=0 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 06 86 00 05 10</u>	<u>t=0 s. Result 0x00: requested</u> <u>PIN operation completed.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 07 AA 03 00 00 02 00 00</u> <u>81 F6</u>	<u>t=0 s. Arm cached-PIN</u> <u>verification for credential</u> <u>presentation.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 07 40 5B 53</u>	<u>t=0 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>t=0 s. Poll for completion.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>t=0 s. No completion pending.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>t=10 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 06 86 0A 4F B1</u>	<u>t=10 s. Result 0x0A: no cached</u> <u>PIN.</u>

D.7 Expiration before VERIFY submission

The PD accepts verification at t=9. Before the PD submits VERIFY, the deadline is reached at t=10. The PD returns PIVSPER 0x0A (no cached PIN) and sends no VERIFY. Acceptance alone does not reserve the PIN beyond its deadline. Timing is modeled.

Example ID: pin-expiration-boundary.

Table 107: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 04 00 00 00 0A 00</u> <u>CD 30</u>	<u>t=0 s. Acquire a PIN; Cache</u> <u>Validity is 0x000A seconds.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>t=0 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>t=0 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 06 86 00 05 10</u>	<u>t=0 s. Result 0x00: requested</u> <u>PIN operation completed.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 07 AA 03 00 00 00 00 00</u> <u>E1 98</u>	<u>t=9 s. Accept verification at t=9,</u> <u>before the cache deadline.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 07 40 5B 53</u>	<u>t=9 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>t=10 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 05 86 0A 1F E8</u>	<u>t=10 s. Result 0x0A: no cached</u> <u>PIN.</u>

D.8 Expiration after VERIFY submission

The PD submits VERIFY at t=9, before the deadline. The cache expires at t=10 while the exchange is completing. The PD reports successful verification, then reports an empty cache with the credential still PIN verified. Credential APDUs come from the contactless capture; the timing is modeled.

Example ID: pin-expiration-submitted.

Table 108: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 04 00 00 00 0A 00</u> <u>CD 30</u>	<u>t=0 s. Acquire a PIN; Cache</u> <u>Validity is 0x000A seconds.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>t=0 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>t=0 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 06 86 00 05 10</u>	<u>t=0 s. Result 0x00: requested</u> <u>PIN operation completed.</u>
<u>ACU</u> <u>PIVXMITPAIRING</u>	<u>53 01 10 00 07 AD 30 30 30 30 30 30</u> <u>30 32 AE E4</u>	<u>t=0 s. Supply the captured</u> <u>pairing code after SM</u> <u>preparation.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 07 40 5B 53</u>	<u>t=0 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>t=0 s. Poll for completion.</u>
<u>PD</u> <u>CARDSTATUSR</u>	<u>53 81 2A 00 05 85 01 02 01 01 B6 10</u> <u>37 92 27 0F BD 08 B6 10 37 92 27 0F</u> <u>BD 08 48 00 0B A0 00 00 03 08 00 00</u> <u>10 00 01 00 D4 6F</u>	<u>t=0 s. Status 0x01: SM and VCI</u> <u>established.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 06 AA 02 00 00 00 00 00</u> <u>92 9A</u>	<u>t=9 s. Accept verification at t=9,</u> <u>before the cache deadline.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 06 40 6A 60</u>	<u>t=9 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>t=10 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 07 86 00 35 27</u>	<u>t=10 s. Result 0x00: requested</u> <u>PIN operation completed.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 00 00 00 00 00 00</u> <u>A7 D9</u>	<u>t=10 s. Empty cache; credential</u> <u>PIN verified.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 05 86 02 17 69</u>	<u>t=10 s. Cache empty; credential</u> <u>PIN verified.</u>

D.9 PIN reuse preserves the original deadline

Two explicit contact verification requests use the same cached PIN at t=1 and t=2. Both succeed. At t=10, the cache expires and the credential remains PIN verified. The successes and timing are modeled; the example demonstrates the PD state rules.

Example ID: pin-expiration-reuse.

Table 109: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 04 00 00 00 0A 00</u> <u>CD 30</u>	<u>t=0 s. Acquire a PIN; Cache</u> <u>Validity is 0x000A seconds.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>t=0 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>t=0 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 06 86 00 05 10</u>	<u>t=0 s. Result 0x00: requested</u> <u>PIN operation completed.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 07 AA 03 00 00 00 00 00</u> <u>E1 98</u>	<u>t=1 s. Explicit contact</u> <u>verification using the same</u> <u>cached PIN.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 07 40 5B 53</u>	<u>t=1 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>t=1 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 05 86 00 55 49</u>	<u>t=1 s. Result 0x00: requested</u> <u>PIN operation completed.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 06 AA 03 00 00 00 00 00</u> <u>32 DF</u>	<u>t=2 s. Explicit contact</u> <u>verification using the same</u> <u>cached PIN.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 06 40 6A 60</u>	<u>t=2 s. Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>t=2 s. Poll for completion.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 07 86 00 35 27</u>	<u>t=2 s. Result 0x00: requested</u> <u>PIN operation completed.</u>
<u>ACU</u> <u>PIVSPE</u>	<u>53 01 0E 00 05 AA 00 00 00 00 00 00</u> <u>A7 D9</u>	<u>t=10 s. Cache expired;</u> <u>credential PIN verification</u> <u>remains valid.</u>
<u>PD</u> <u>PIVSPER</u>	<u>53 81 09 00 05 86 02 17 69</u>	<u>t=10 s. Cache empty; credential</u> <u>PIN verified.</u>

D.10 Credential failure status across two fragments

PIV Auto reports a modeled GENERAL AUTHENTICATE rejection: Result 0x06, Detail 0x00. The credential status is 0x6982 (security status not satisfied). The first fragment ends with SW1, 0x69; the second carries SW2, 0x82. The ACU reassembles the complete record before interpreting the status word.

Example ID: auto-status-word.

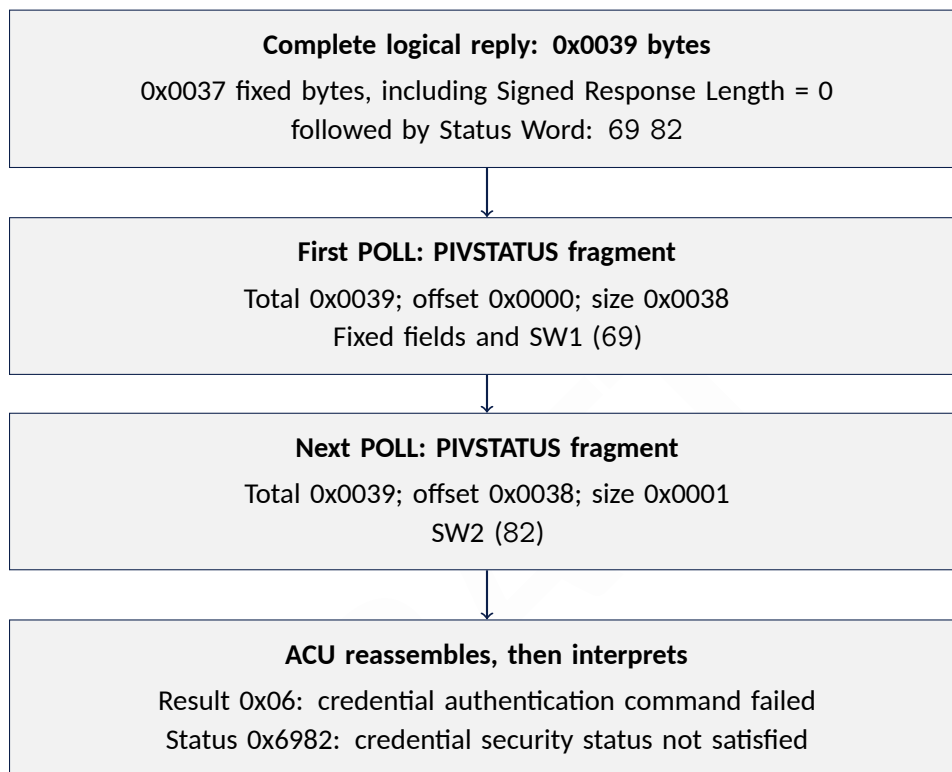


Figure 9: OSDP 3.0 multipart fields describe the entire logical reply. A fragment boundary can fall between SW1 and SW2.

Table 110: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll for completion.</u>
<u>PD</u> <u>PIVSTATUS</u>	<u>53 81 46 00 05 89 39 00 00 00 38 00</u> <u>06 00 01 00 00 00 02 00 00 00 00 00</u> <u>00 00 00 00 00 00 00 00 00 00 00 00</u> <u>00 00 00 00 00 00 00 00 00 00 00 00</u> <u>00 00 00 00 00 00 00 00 00 00 00 00</u> <u>00 00 00 9A 07 00 00 69 E4 69</u>	<u>PIV Auto credential rejection;</u> <u>SW1/SW2 cross the fragment</u> <u>boundary.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll for completion.</u>

Message	Packet bytes (hex)	Notes
PD PIVSTATUS	53 81 0F 00 06 89 39 00 38 00 01 00 82 9D EB	PIV Auto credential rejection; SW1/SW2 cross the fragment boundary.

D.11 Incorrect PIN preserves SM and VCI

This modeled case starts with SM, VCI, and a cached PIN available. The credential returns an authenticated 0x63C2 (incorrect PIN, two attempts remain). The PD reports PIN mismatch and clears the cache and PIN-verification state. SM and VCI remain established. The SPE reply carries the mismatch result; a separate Mode 0x06 query retrieves remaining attempts. The ACU can acquire a corrected PIN and request verification again using the existing session.

Example ID: sm-credential-rejection.

Table 111: OSDP command and reply exchange

Message	Packet bytes (hex)	Notes
ACU PIVSPE	53 01 0E 00 05 AA 02 00 00 00 00 00 E7 52	Verify a cached PIN over an established VCI; initial state is modeled.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll for completion.
PD PIVSPER	53 81 09 00 06 86 03 66 20	Result 0x03: PIN mismatch.
ACU PIVSPE	53 01 0E 00 07 AA 00 00 00 00 00 00 01 56	Cache erased and PIN-verification state cleared after PIN mismatch.
PD PIVSPER	53 81 09 00 07 86 00 35 27	Cache empty; credential PIN verification absent.
ACU CARDSTATUS	53 01 09 00 05 A9 01 8C 9B	Read credential status after ordinary rejection.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll for completion.
PD CARDSTATUSR	53 81 2A 00 06 85 01 02 01 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 FF FF 0B A0 00 00 03 08 00 00 10 00 01 00 A9 03	Status 0x01: SM and VCI established.

D.12 Recovering trust for contactless PIV Auto

This example uses a credential that supports SM and requires a pairing code for VCI. PIV Auto is configured for contactless authentication with key 0x9A, but the PD's trust store is empty. The ACU already holds the roots and validation policy authorized for the deployment. Figure 10 shows the recovery sequence.

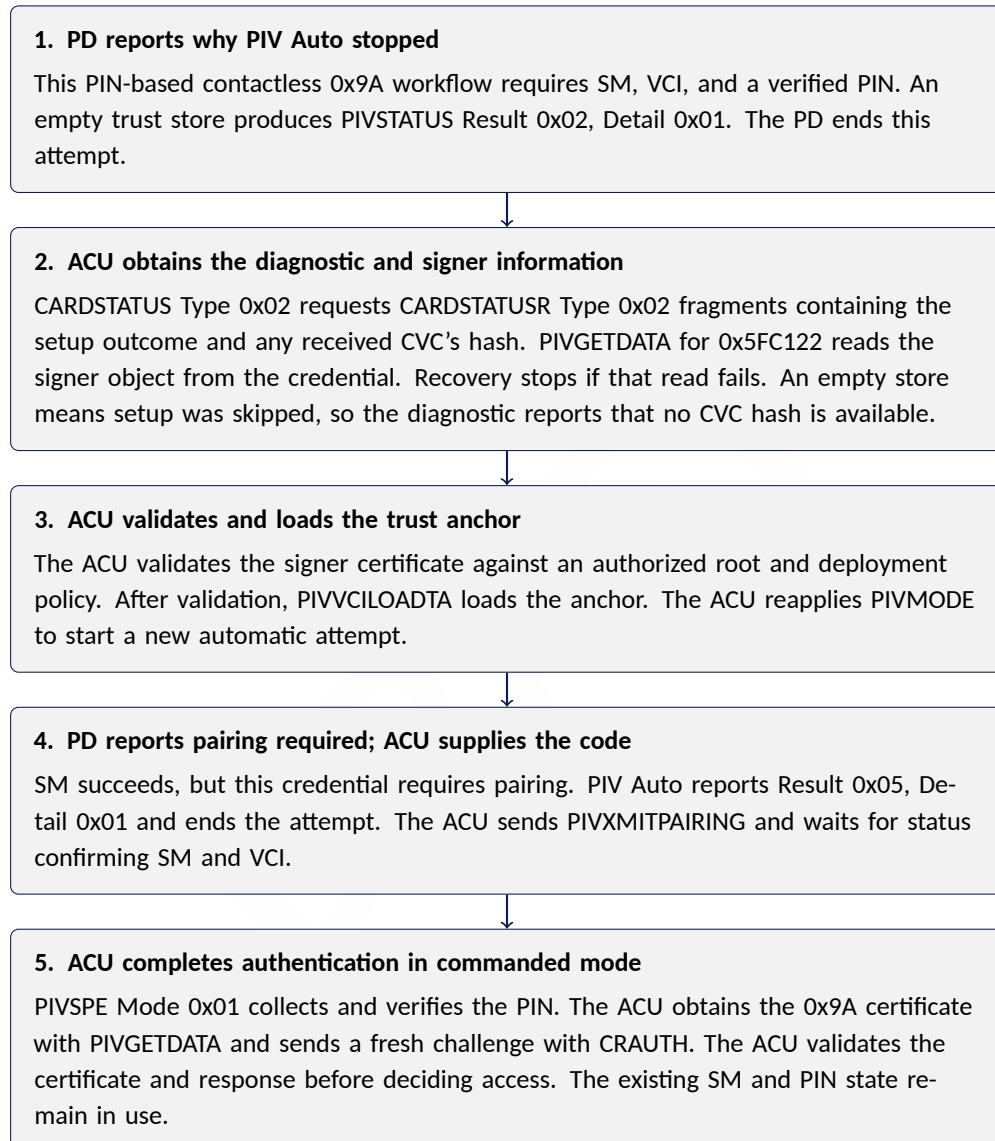


Figure 10: Recovery from an empty PD trust store for contactless PIV authentication with key 0x9A. Every step depends on a successful result from the preceding step. Unsupported SM or VCI requires a different ACU-selected workflow.

The first PIV Auto attempt ends with Result 0x02, Detail 0x01 (trust store empty). The ACU requests the SM diagnostic record, then reads the Secure Messaging Certificate Signer object with PIVGETDATA, Object Identifier 0x5FC122, Tag Length zero, Offset zero, and Requested Length zero. When the read succeeds, the reply contains the complete object, including signer certificate tag 0x70 and any intermediate CVC in tag 0x7F21. The PD reads the credential through the contactless interface. If the object is absent or

the credential denies access, the PD reports the corresponding PIVERROR. The ACU then selects another supported workflow or ends the transaction. Recovery relies on the credential returning the object.

The ACU validates the signer certificate's chain to an authorized root, validity period, revocation status, and PIV or PIV-I content-signing extended key usage. The ACU checks any intermediate CVC's signature and linkage as specified under PIVVCILOADTA. The anchor record contains the validated signer's public key and the IIN derived from that certificate's subjectKeyIdentifier. The ACU loads an anchor only after these checks succeed. An IIN match alone provides no authorization to trust a certificate.

After the load succeeds, the ACU reapplies the PIVMODE configuration. The PD starts a new attempt, establishes SM, and reports Result 0x05, Detail 0x01 (pairing code required). The ACU sends PIVXMITPAIRING and waits for CARDSTATUSR with security status 0x01 (SM and VCI established). The ACU then sends PIVSPE Mode 0x01, with Auto Execute clear, to collect and verify the PIN. After successful verification, the ACU reads the PIV Authentication certificate (Object Identifier 0x5FC105) and uses CRAUTH with key 0x9A and a fresh challenge. Certificate and challenge-response validation remain the ACU's responsibility.

The ACU waits for a successful result at each step before issuing the next command. An error ends this recovery sequence. These are commanded operations after the automatic attempt has ended. The PD returns each command's defined reply; the sequence generates no second PIV Auto result. PIVMODE remains configured for the next credential presentation. Reapplying PIVMODE after pairing or PIN verification would clear the state needed for the commanded authentication.

If SM or VCI is unsupported, the ACU chooses a supported interface or authentication method explicitly, or rejects the transaction. If a signature or cryptogram check fails, the PD blocks credential operations and retains the diagnostic record. The ACU investigates that failure before requesting a new attempt. Trust recovery is appropriate for missing authorized trust material; failed cryptographic checks are security failures.

This is a control-flow example. The preceding captured SM/VCI packet examples cover pairing and PIN verification. The companion trust-recovery example records the command sequence and validation decisions.

E CRAUTH Worked Examples (Informative)

This example sends a captured P-256 challenge to key 0x9E and returns the captured signature template. The first command fragment contains only Format Marker 0xFF. The second contains Algorithm 0x11, Reference Identifier 0x9E, and the 0x20-byte challenge. The complete request record is 0x0023 bytes. The PD removes the marker and uses the algorithm, key reference, and challenge to construct GENERALAUTHENTICATE.

The credential data comes from the existing contactless card04 SM capture. Credential SM is assumed established as in that capture; the example shows the plaintext at the PD's SM boundary. The packets omit OSDP Secure Channel for clarity. The two failure cases are modeled to show terminal error handling. Full exchanges and source references are in the CRAUTH example file. The command and reply codes remain 0xA5 and 0x82.

Table 112: CRAUTH command and reply exchange

Message	Packet bytes (hex)	Notes
ACU CRAUTH	53 01 0F 00 05 A5 23 00 00 00 01 00 FF 6B 63	Total 0x0023; offset 0x0000; fragment 0x0001. Format marker only.
PD ACK	53 81 08 00 05 40 39 35	Fragment accepted.
ACU CRAUTH	53 01 30 00 06 A5 23 00 01 00 22 00 11 9E 9B C1 80 EF 9A 61 6F 9F 05 8B F4 60 44 C8 9A 30 1B BD 90 13 7A DF 12 F9 D9 A9 91 7B 09 4A EF A0 5C FF	Total 0x0023; offset 0x0001; fragment 0x0022.
PD ACK	53 81 08 00 06 40 6A 60	Fragment accepted.
ACU POLL	53 01 08 00 07 60 E9 55	Poll for the result.
PD CRAUTHR	53 81 36 00 07 82 4B 00 00 00 28 00 7C 49 82 47 30 45 02 21 00 83 6F 34 70 E4 CD 87 3C 6C A3 E3 B6 5A D3 6B 17 DF FC E2 7B 56 A8 25 EC 39 A9 DC E3 D4 CB B0 6F 70	Total 0x004B; offset 0x0000; fragment 0x0028.
ACU POLL	53 01 08 00 05 60 8B 33	Poll for the result.
PD CRAUTHR	53 81 31 00 05 82 4B 00 28 00 23 00 95 02 20 61 1E FE 89 BF A8 13 B1 34 BD 11 1D 64 00 01 8B 50 71 1E 6C 9C 5D C8 22 DC 75 24 0B FA 60 D6 EE D9 FA	Total 0x004B; offset 0x0028; fragment 0x0023.

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll for the result.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 06 40 6A 60</u>	<u>No completion pending.</u>

Table 113: Modeled failure completions

Failure	Packet bytes (hex)	ACU action
<u>Credential rejection</u>	<u>53 81 0C 00 07 8A 23 10 69 82</u> <u>2A D5</u>	<u>PIVERROR 0x1023 (security</u> <u>conditions unmet), SW1/SW2</u> <u>0x6982. End the request.</u>
<u>Failure after a data</u> <u>fragment</u>	<u>53 81 0A 00 05 8A 02 00 0E 51</u>	<u>PIVERROR 0x0002 (smart-card</u> <u>timeout). Discard the partial</u> <u>response.</u>

F Deprecated osdp_GENAUTH

The osdp_GENAUTH command is deprecated and expected to be removed in a future revision of this specification. Using a PIV credential to decrypt or process arbitrary data is unsafe and unnecessary for credential verification; implementations **SHOULD** use osdp_CAUTH flows instead. Controllers and PDs **SHOULD** plan migrations away from osdp_GENAUTH.